

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
pd.set_option('display.float_format', '{:.2f}'.format)
```

```
In [2]: merchant = pd.read_csv("C:\\Users\\visha\\Downloads\\Datasets\\Financial Transac
transaction = pd.read_csv("C:\\Users\\visha\\Downloads\\Datasets\\Financial Tran
cards = pd.read_csv("C:\\Users\\visha\\Downloads\\Datasets\\Financial Transactio
customer = pd.read_csv("C:\\Users\\visha\\Downloads\\Datasets\\Financial Transac
```

```
In [3]: merchant.head()
```

```
Out[3]:
```

	Merchant_ID	Merchant_Name	Merchant_Category	State	City	Merchant_F
0	MER100001	Paytm Utilities	Utilities	Haryana	Gurugram	
1	MER100002	Coursera	Education	Bihar	Patna	
2	MER100003	Redbus	Travel	Bihar	Muzaffarpur	
3	MER100004	Air India	Airline	Tamil Nadu	Salem	
4	MER100005	IndiGo	Airline	Punjab	Jalandhar	

```
In [4]: merchant.describe()
```

```
Out[4]:
```

	Merchant_Rating
count	500.00
mean	4.29
std	0.42
min	3.50
25%	4.00
50%	4.30
75%	4.70
max	5.00

```
In [5]: merchant.duplicated().sum()
```

```
Out[5]: np.int64(0)
```

```
In [6]: merchant.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Merchant_ID           500 non-null   str
1   Merchant_Name         500 non-null   str
2   Merchant_Category     500 non-null   str
3   State                 500 non-null   str
4   City                  500 non-null   str
5   Merchant_Risk_Level   500 non-null   str
6   Merchant_Rating       500 non-null   float64
7   Merchant_Status       500 non-null   str
8   Merchant_Since        500 non-null   str
dtypes: float64(1), str(8)
memory usage: 35.3 KB
```

```
In [7]: merchant.shape
```

```
Out[7]: (500, 9)
```

```
In [8]: merchant['Merchant_Since'] = pd.to_datetime(merchant['Merchant_Since'])
```

```
In [9]: merchant.isnull().sum()
```

```
Out[9]: Merchant_ID           0
Merchant_Name             0
Merchant_Category         0
State                    0
City                     0
Merchant_Risk_Level       0
Merchant_Rating           0
Merchant_Status           0
Merchant_Since            0
dtype: int64
```

```
In [10]: transaction.head()
```

Out[10]:

	Transaction_ID	Customer_ID	Card_ID	Merchant_ID	Transaction_Date	Transactio
0	TXN1000001	CUST100660	CARD500875	MER100380	2024-03-03	(
1	TXN1000002	CUST111388	CARD514772	MER100017	2023-11-08	(
2	TXN1000003	CUST115872	CARD520625	MER100143	2023-01-11	(
3	TXN1000004	CUST109049	CARD511773	MER100053	2023-05-25	(
4	TXN1000005	CUST114459	CARD518771	MER100064	2023-10-22	(

```
In [11]: transaction.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 250000 entries, 0 to 249999
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Transaction_ID         250000 non-null  str
1   Customer_ID           250000 non-null  str
2   Card_ID               250000 non-null  str
3   Merchant_ID           250000 non-null  str
4   Transaction_Date       250000 non-null  str
5   Transaction_Time       250000 non-null  str
6   Transaction_Amount     250000 non-null  float64
7   Payment_Method         250000 non-null  str
8   Transaction_Channel    250000 non-null  str
9   Device_Type           250000 non-null  str
10  Transaction_Status     250000 non-null  str
11  Is_International       250000 non-null  int64
12  Fraud_Flag            250000 non-null  int64
13  Fraud_Reason          13473 non-null   str
14  Merchant_Risk_Level    250000 non-null  str
15  Merchant_Category     250000 non-null  str
16  Customer_State        250000 non-null  str
17  Customer_City         250000 non-null  str
18  Merchant_State        250000 non-null  str
19  Merchant_City         250000 non-null  str
dtypes: float64(1), int64(2), str(17)
memory usage: 38.1 MB

```

In [12]: `transaction.head()`

Out[12]:

	Transaction_ID	Customer_ID	Card_ID	Merchant_ID	Transaction_Date	Transactio
0	TXN1000001	CUST100660	CARD500875	MER100380	2024-03-03	(
1	TXN1000002	CUST111388	CARD514772	MER100017	2023-11-08	(
2	TXN1000003	CUST115872	CARD520625	MER100143	2023-01-11	(
3	TXN1000004	CUST109049	CARD511773	MER100053	2023-05-25	(
4	TXN1000005	CUST114459	CARD518771	MER100064	2023-10-22	(

In [13]: `transaction[(transaction['Transaction_Amount'] > 1000000) & (transaction['Fraud_`

Out[13]:

	Transaction_ID	Customer_ID	Card_ID	Merchant_ID	Transaction_Date	Trar
672	TXN1000673	CUST116590	CARD521533	MER100144	2024-03-03	
1486	TXN1001487	CUST110892	CARD514137	MER100321	2026-03-11	
4439	TXN1004440	CUST107706	CARD510015	MER100149	2025-11-05	
5307	TXN1005308	CUST106644	CARD508648	MER100055	2025-11-29	
5972	TXN1005973	CUST119597	CARD525448	MER100334	2024-03-10	
...	...	...	...	...	...	...
246837	TXN1246838	CUST118422	CARD523914	MER100480	2023-07-26	
247869	TXN1247870	CUST107788	CARD510128	MER100499	2023-10-03	
248096	TXN1248097	CUST106744	CARD508781	MER100065	2026-02-27	
248690	TXN1248691	CUST105536	CARD507208	MER100423	2025-09-01	
249903	TXN1249904	CUST106744	CARD508782	MER100484	2023-05-30	

200 rows × 20 columns

In [14]:

```
transaction.shape
```

Out[14]:

```
(250000, 20)
```

```
In [15]: transaction['Transaction_Date'] = pd.to_datetime(transaction['Transaction_Date'])
```

```
In [16]: transaction.duplicated().sum()
```

```
Out[16]: np.int64(0)
```

```
In [17]: transaction.isnull().sum()
```

```
Out[17]: Transaction_ID      0
Customer_ID      0
Card_ID          0
Merchant_ID      0
Transaction_Date  0
Transaction_Time  0
Transaction_Amount  0
Payment_Method   0
Transaction_Channel  0
Device_Type      0
Transaction_Status  0
Is_International  0
Fraud_Flag       0
Fraud_Reason     236527
Merchant_Risk_Level  0
Merchant_Category  0
Customer_State   0
Customer_City    0
Merchant_State   0
Merchant_City    0
dtype: int64
```

```
In [18]: transaction.describe()
```

```
Out[18]:
```

	Transaction_Date	Transaction_Amount	Is_International	Fraud_Flag
count	250000	250000.00	250000.00	250000.00
mean	2024-11-14 06:12:02.995199	16607.53	0.05	0.05
min	2023-01-01 00:00:00	100.00	0.00	0.00
25%	2023-11-11 00:00:00	1000.00	0.00	0.00
50%	2024-12-01 00:00:00	3155.26	0.00	0.00
75%	2025-11-27 00:00:00	18309.91	0.00	0.00
max	2026-08-18 00:00:00	1887057.16	1.00	1.00
std	NaN	53421.57	0.22	0.23

```
In [19]: cards.head()
```

Out[19]:

	Card_ID	Customer_ID	Card_Type	Card_Network	Credit_Limit	Card_Status	Con
0	CARD500001	CUST100001	Gold	RuPay	587317	Active	
1	CARD500002	CUST100002	Silver	Visa	150720	Active	
2	CARD500003	CUST100003	Gold	RuPay	631005	Active	
3	CARD500004	CUST100004	Silver	Mastercard	285015	Expired	
4	CARD500005	CUST100005	Gold	Mastercard	348591	Active	



In [20]: `cards.duplicated().sum()`

Out[20]: `np.int64(0)`

In [21]: `cards.info()`

```
<class 'pandas.DataFrame'>
RangeIndex: 32457 entries, 0 to 32456
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Card_ID         32457 non-null  str
1   Customer_ID     32457 non-null  str
2   Card_Type       32457 non-null  str
3   Card_Network    32457 non-null  str
4   Credit_Limit    32457 non-null  int64
5   Card_Status     32457 non-null  str
6   Contactless     32457 non-null  str
7   Card_Mode       32457 non-null  str
8   Issue_Date      32457 non-null  str
9   Expiry_Date     32457 non-null  str
dtypes: int64(1), str(9)
memory usage: 2.5 MB
```

In [22]: `cards.describe()`

Out[22]:

Credit_Limit	
count	32457.00
mean	444777.54
std	368956.54
min	50002.00
25%	162030.00
50%	297402.00
75%	618074.00
max	1499927.00

In [23]: cards[(cards['Credit_Limit'] > 700000) & (cards['Card_Type'] == 'Platinum')]

Out[23]:

	Card_ID	Customer_ID	Card_Type	Card_Network	Credit_Limit	Card_Status
7	CARD500008	CUST100007	Platinum	Mastercard	764925	Active
9	CARD500010	CUST100009	Platinum	Visa	971876	Active
16	CARD500017	CUST100012	Platinum	Mastercard	966151	Active
18	CARD500019	CUST100014	Platinum	Mastercard	980464	Active
24	CARD500025	CUST100020	Platinum	RuPay	1410557	Active
...	...	...	...	...	...	...
32440	CARD532441	CUST124987	Platinum	American Express	1445509	Active
32449	CARD532450	CUST124995	Platinum	RuPay	1064239	Active
32450	CARD532451	CUST124995	Platinum	American Express	1389939	Active
32452	CARD532453	CUST124997	Platinum	Visa	1161328	Active
32456	CARD532457	CUST125000	Platinum	Mastercard	1097473	Active

6082 rows × 10 columns

In [24]: cards.shape

Out[24]: (32457, 10)

```
In [25]: cards['Issue_Date'] = pd.to_datetime(cards['Issue_Date'])
cards['Expiry_Date'] = pd.to_datetime(cards['Expiry_Date'])
```

```
In [26]: cards.isnull().sum()
```

```
Out[26]: Card_ID      0
Customer_ID    0
Card_Type      0
Card_Network   0
Credit_Limit   0
Card_Status    0
Contactless    0
Card_Mode      0
Issue_Date     0
Expiry_Date    0
dtype: int64
```

```
In [27]: customer.head()
```

```
Out[27]:
```

	Customer_ID	Customer_Name	Gender	Age	Marital_Status	Occupation	Annual_Inc
0	CUST100001	Omaja Majumdar	Female	27	Single	Police Officer	116
1	CUST100002	Jason Agrawal	Female	41	Married	Teacher	49
2	CUST100003	Amrita Dora	Male	61	Divorced	Business	186
3	CUST100004	Radhika Batra	Male	26	Single	Software Engineer	80
4	CUST100005	Gautam Mukhopadhyay	Female	24	Single	Software Engineer	158



```
In [28]: customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer_ID           25000 non-null  str
1   Customer_Name         25000 non-null  str
2   Gender                25000 non-null  str
3   Age                   25000 non-null  int64
4   Marital_Status        25000 non-null  str
5   Occupation            25000 non-null  str
6   Annual_Income         25000 non-null  int64
7   Customer_Segment      25000 non-null  str
8   State                 25000 non-null  str
9   City                  25000 non-null  str
10  Account_Type          25000 non-null  str
11  Customer_Since        25000 non-null  str
dtypes: int64(2), str(10)
memory usage: 2.3 MB
```

```
In [29]: customer.describe()
```


Out[29]:

	Age	Annual_Income
count	25000.00	25000.00
mean	41.81	1611153.23
std	13.06	1727786.25
min	18.00	2.00
25%	31.00	570477.50
50%	41.00	969189.00
75%	52.00	1882414.00
max	70.00	9999931.00

In [30]: `customer[customer['Annual_Income'] > 5000000]`

Out[30]:

	Customer_ID	Customer_Name	Gender	Age	Marital_Status	Occupation	Annua
29	CUST100030	Manan Chander	Male	62	Married	Business	
61	CUST100062	Onveer De	Male	33	Married	Business	
113	CUST100114	Janaki Samra	Female	69	Married	Business	
118	CUST100119	Tamanna Bahri	Male	32	Married	Business	
143	CUST100144	Rachit Ramanathan	Male	41	Married	Business	
...	...	...	...	...	...	...	...
24965	CUST124966	Vincent Goyal	Female	53	Married	Business	
24973	CUST124974	Sathvik Shere	Male	58	Married	Business	
24975	CUST124976	Samesh Sheth	Male	54	Married	Business	
24984	CUST124985	Udyati Bajaj	Male	61	Married	Business	
24990	CUST124991	Anamika Lal	Female	57	Married	Business	

1118 rows × 12 columns

In [31]: `customer['Customer_Since'] = pd.to_datetime(customer['Customer_Since'])`

In [32]: `customer.isnull().sum()`

```
Out[32]: Customer_ID      0
Customer_Name      0
Gender             0
Age               0
Marital_Status     0
Occupation         0
Annual_Income      0
Customer_Segment   0
State             0
City              0
Account_Type       0
Customer_Since     0
dtype: int64
```

```
In [33]: customer.duplicated().sum()
```

```
Out[33]: np.int64(0)
```

```
In [34]: customer.shape
```

```
Out[34]: (25000, 12)
```

Exploratory Data Analysis

```
In [35]: customer.head()
```

```
Out[35]:
```

	Customer_ID	Customer_Name	Gender	Age	Marital_Status	Occupation	Annual_Inc
0	CUST100001	Omaja Majumdar	Female	27	Single	Police Officer	116
1	CUST100002	Jason Agrawal	Female	41	Married	Teacher	49
2	CUST100003	Amrita Dora	Male	61	Divorsed	Business	186
3	CUST100004	Radhika Batra	Male	26	Single	Software Engineer	80
4	CUST100005	Gautam Mukhopadhyay	Female	24	Single	Software Engineer	158



```
In [36]: customer.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer_ID           25000 non-null  str
1   Customer_Name         25000 non-null  str
2   Gender                25000 non-null  str
3   Age                   25000 non-null  int64
4   Marital_Status        25000 non-null  str
5   Occupation            25000 non-null  str
6   Annual_Income         25000 non-null  int64
7   Customer_Segment      25000 non-null  str
8   State                 25000 non-null  str
9   City                  25000 non-null  str
10  Account_Type          25000 non-null  str
11  Customer_Since        25000 non-null  datetime64[us]
dtypes: datetime64[us](1), int64(2), str(9)
memory usage: 2.3 MB

```

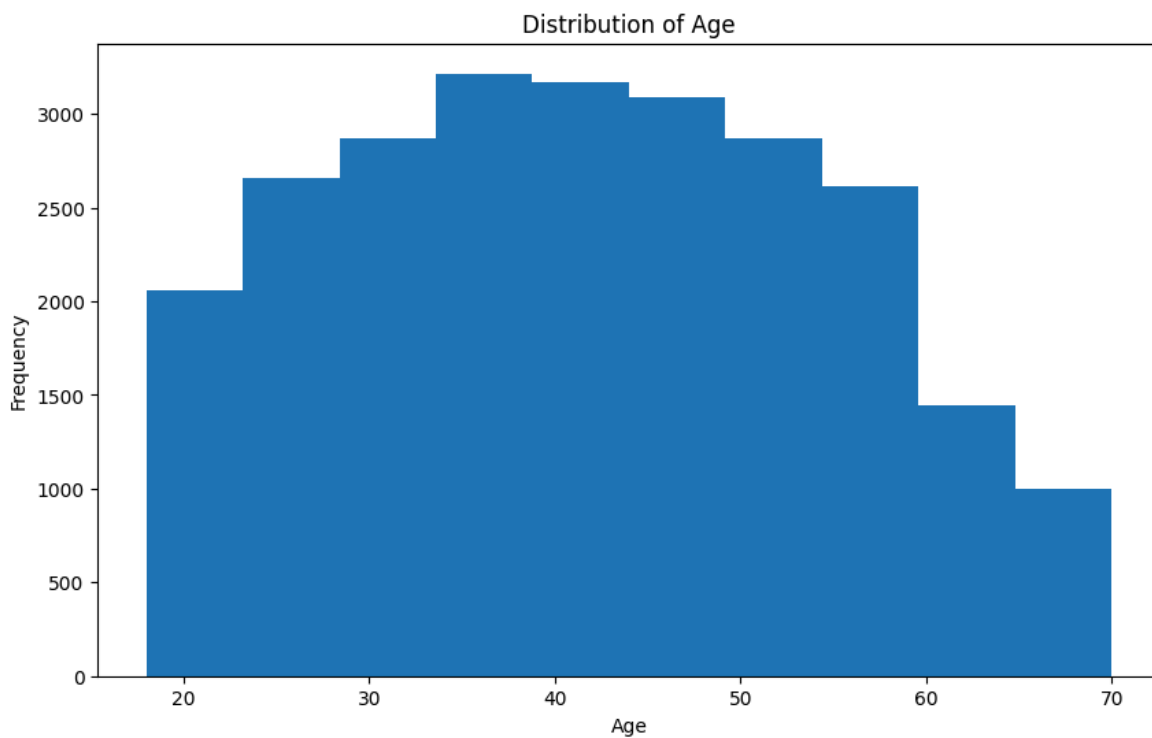
```
In [37]: customer.describe()
```

```
Out[37]:
```

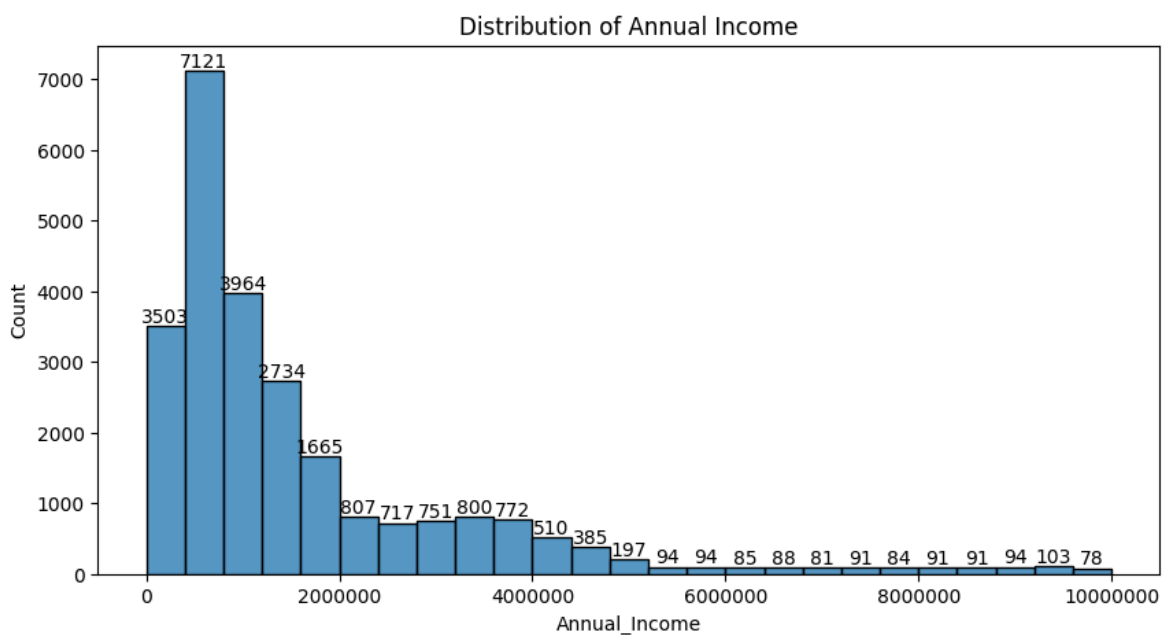
	Age	Annual_Income	Customer_Since
count	25000.00	25000.00	25000
mean	41.81	1611153.23	2021-07-11 11:46:00.192000
min	18.00	2.00	2016-07-19 00:00:00
25%	31.00	570477.50	2018-12-29 00:00:00
50%	41.00	969189.00	2021-07-11 00:00:00
75%	52.00	1882414.00	2024-01-04 06:00:00
max	70.00	9999931.00	2026-07-19 00:00:00
std	13.06	1727786.25	NaN

Univariate Analysis

```
In [38]: plt.figure(figsize=(10,6))
plt.hist(data=customer, x='Age')
plt.title('Distribution of Age')
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



```
In [39]: plt.figure(figsize=(10,5))
ax = sns.histplot(customer['Annual_Income'], bins= 25)
ax.ticklabel_format(style = 'plain', axis = 'x')
plt.bar_label(plt.gca().containers[0])
plt.title("Distribution of Annual Income")
plt.show()
```



```
In [40]: customer.head()
```

Out[40]:

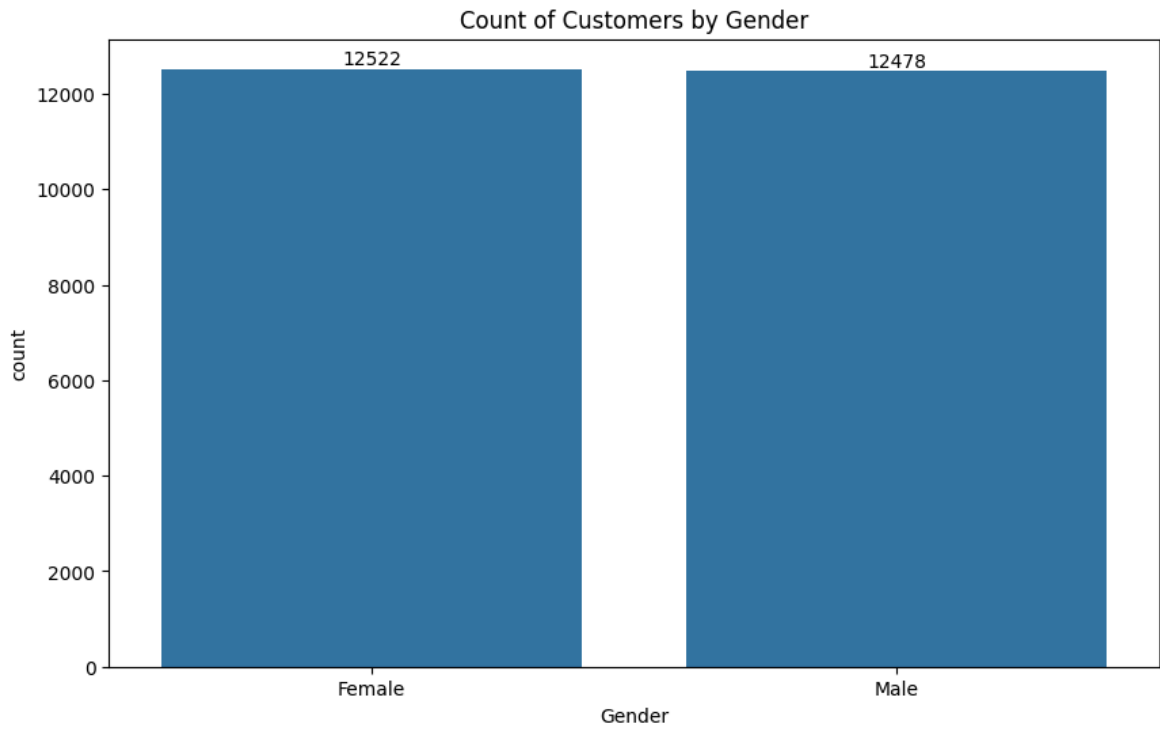
	Customer_ID	Customer_Name	Gender	Age	Marital_Status	Occupation	Annual_Inc
0	CUST100001	Omaja Majumdar	Female	27	Single	Police Officer	116
1	CUST100002	Jason Agrawal	Female	41	Married	Teacher	49
2	CUST100003	Amrita Dora	Male	61	Divorsed	Business	186
3	CUST100004	Radhika Batra	Male	26	Single	Software Engineer	80
4	CUST100005	Gautam Mukhopadhyay	Female	24	Single	Software Engineer	158

In [41]: `customer.info()`

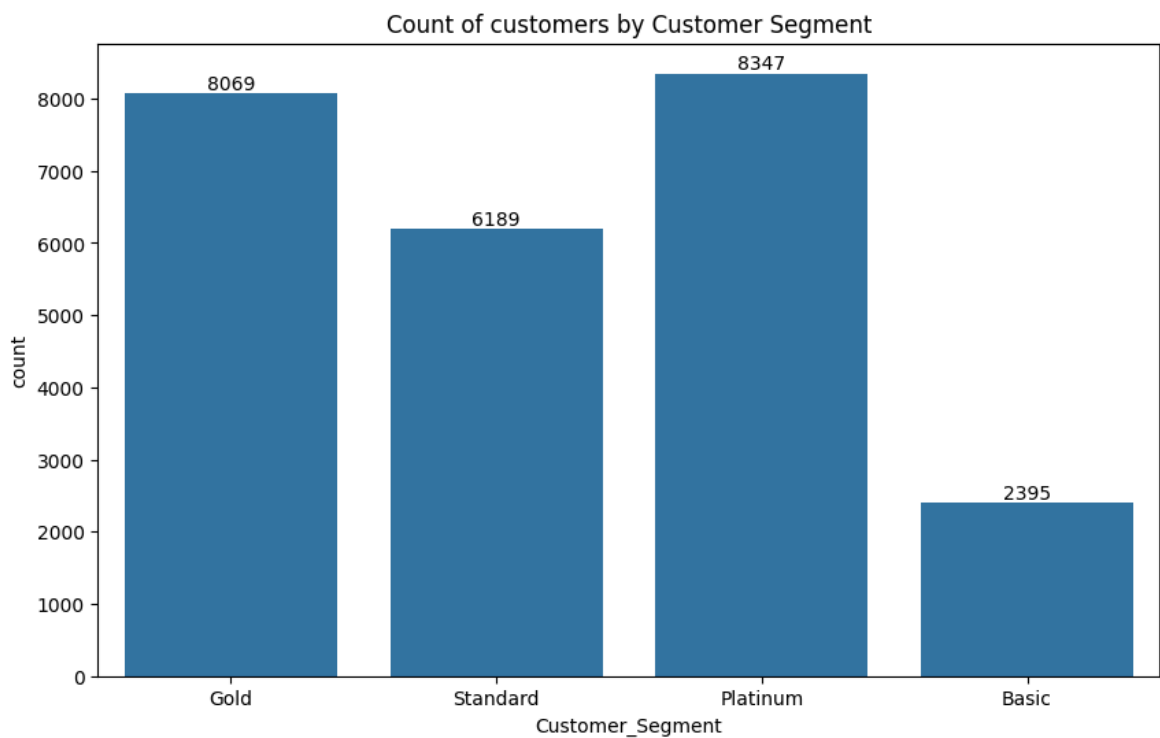
```
<class 'pandas.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer_ID           25000 non-null  str
1   Customer_Name         25000 non-null  str
2   Gender                25000 non-null  str
3   Age                   25000 non-null  int64
4   Marital_Status        25000 non-null  str
5   Occupation            25000 non-null  str
6   Annual_Income         25000 non-null  int64
7   Customer_Segment      25000 non-null  str
8   State                 25000 non-null  str
9   City                  25000 non-null  str
10  Account_Type          25000 non-null  str
11  Customer_Since        25000 non-null  datetime64[us]
dtypes: datetime64[us](1), int64(2), str(9)
memory usage: 2.3 MB
```

In [42]:

```
plt.figure(figsize=(10,6))
sns.countplot(data=customer,x='Gender')
plt.title("Count of Customers by Gender")
plt.bar_label(plt.gca().containers[0])
plt.show()
```



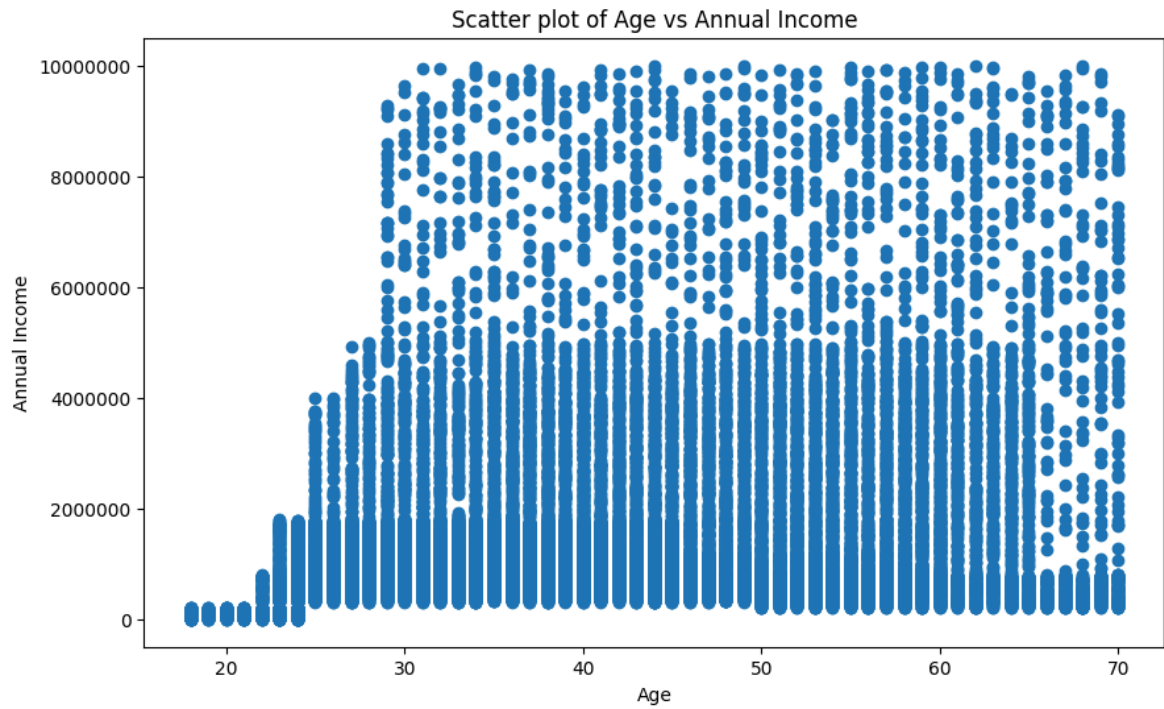
```
In [43]: plt.figure(figsize=(10,6))
plt.title("Count of customers by Customer Segment")
sns.countplot(data=customer,x='Customer_Segment')
plt.bar_label(plt.gca().containers[0])
plt.show()
```



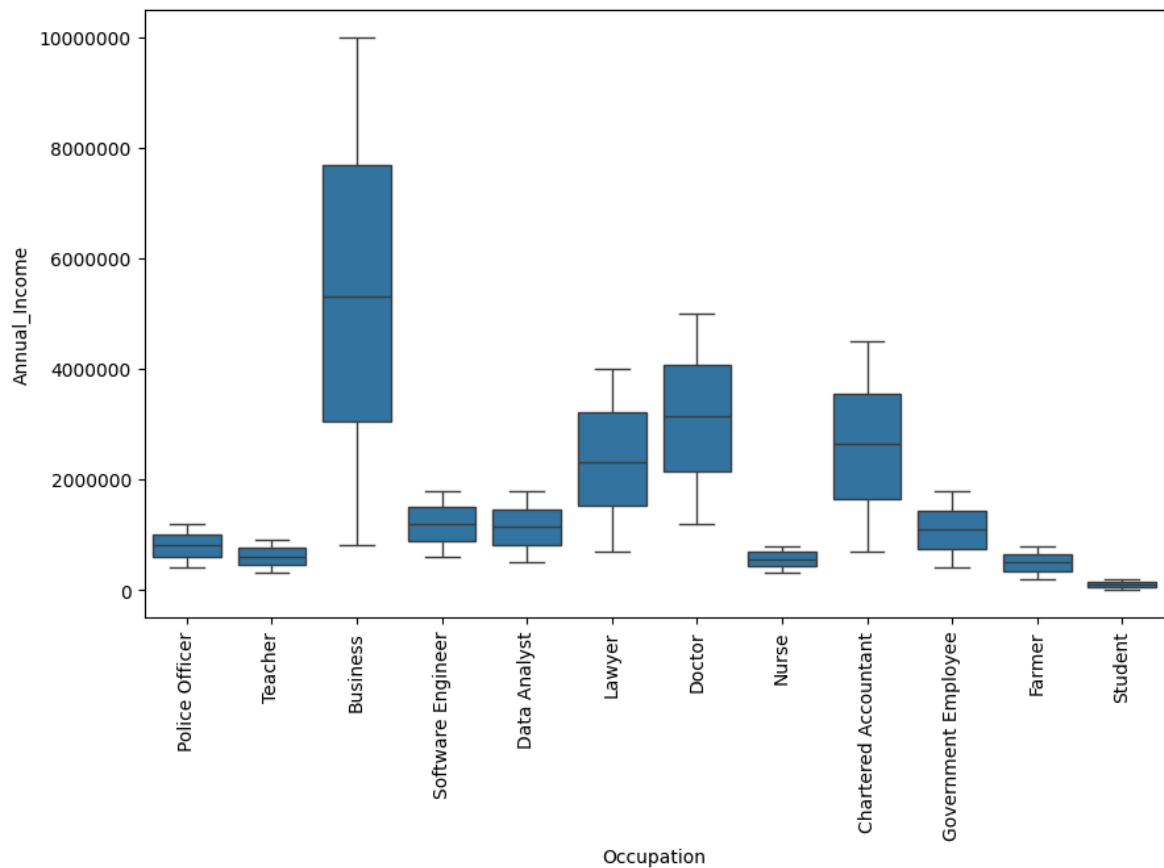
Bivariate Analysis

```
In [44]: plt.figure(figsize=(10,6))
plt.scatter(data=customer, x='Age', y='Annual_Income')
plt.ticklabel_format(style='plain', axis='y')
plt.title("Scatter plot of Age vs Annual Income")
plt.xlabel("Age")
```

```
plt.ylabel("Annual Income")
plt.show()
```



```
In [45]: plt.figure(figsize=(10,6))
sns.boxplot(data=customer, x='Occupation', y='Annual_Income')
plt.xticks(rotation=90)
plt.ticklabel_format(style='plain', axis='y')
plt.show()
```

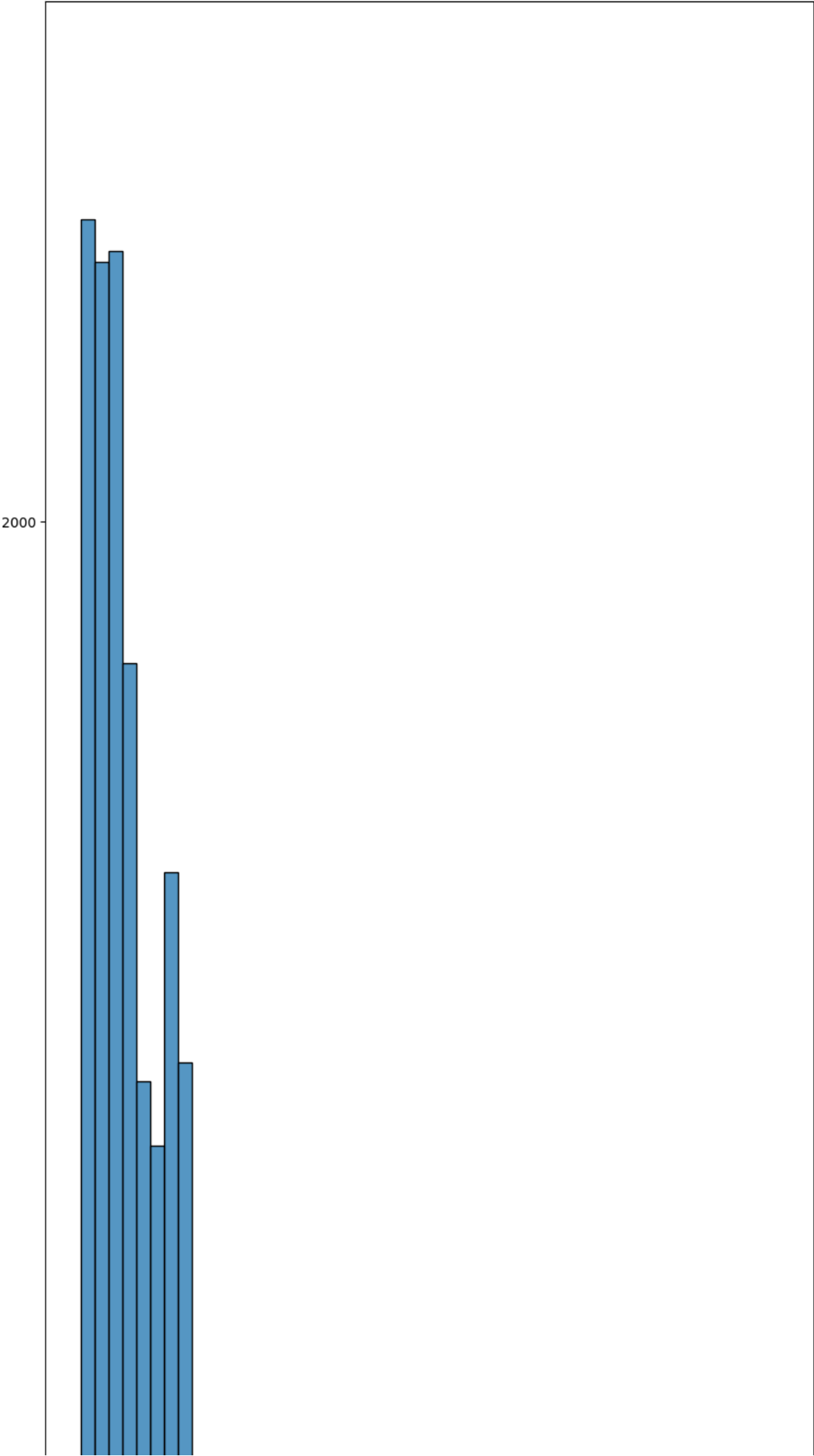


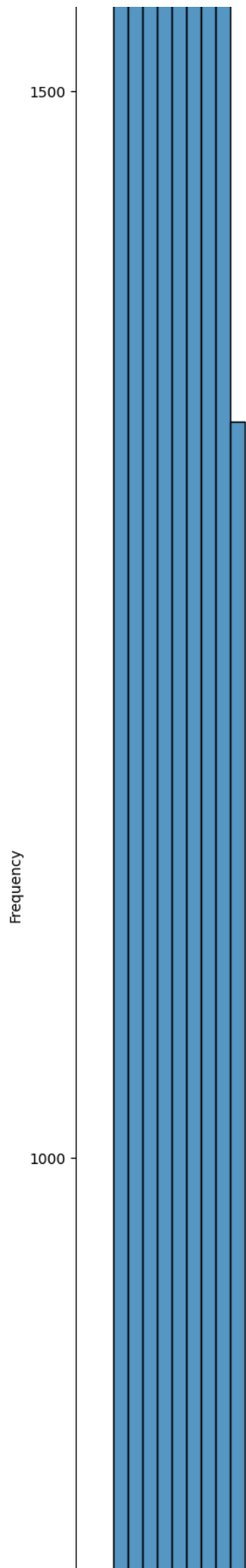
Card Table

Univariate Analysis

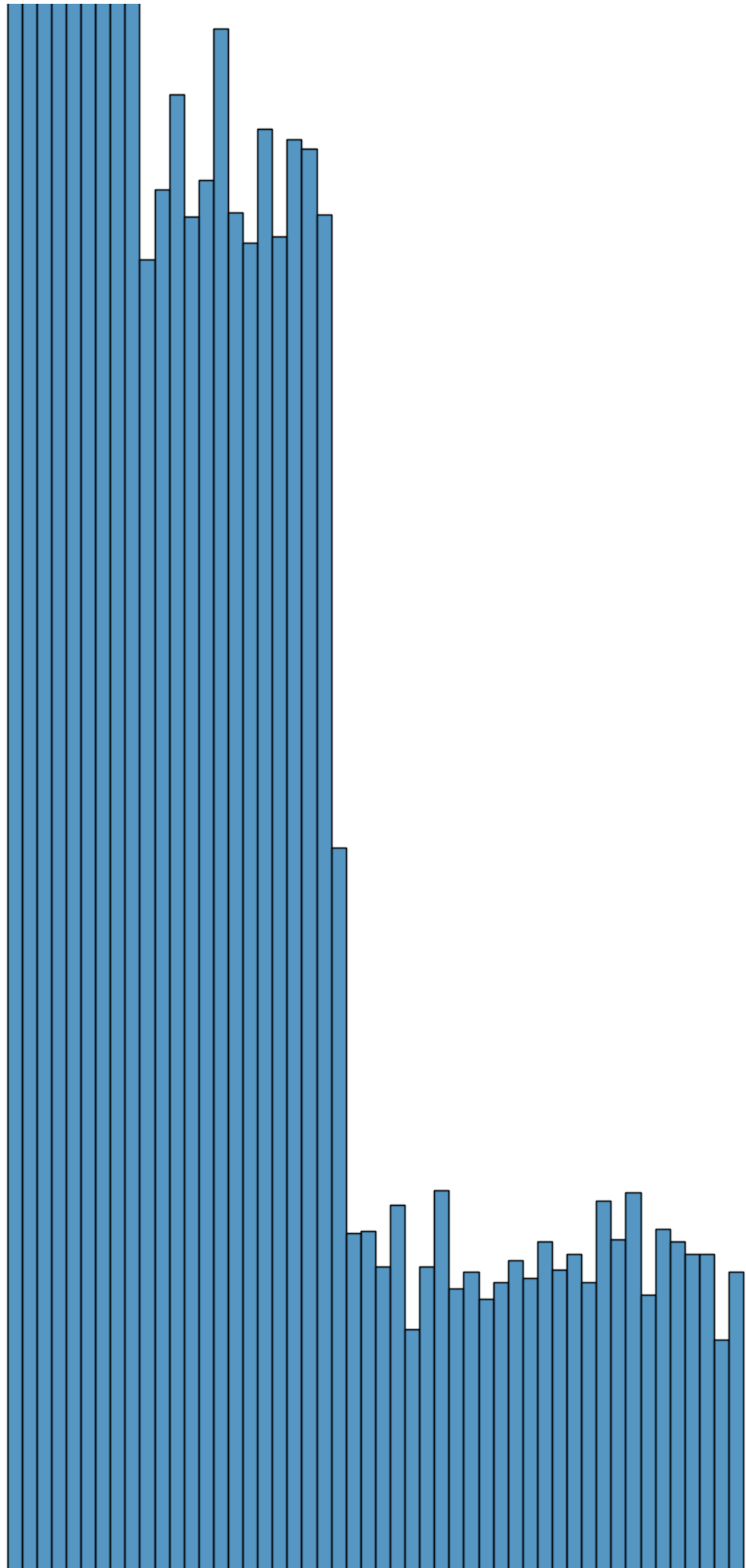
```
In [46]: plt.figure(figsize=(10,60))
sns.histplot(data=cards, x='Credit_Limit', bins=50)
plt.ticklabel_format(style='plain', axis='x')
plt.title("Distribution of credit Limit")
plt.xlabel("Credit Limit")
plt.ylabel("Frequency")
plt.show()
```

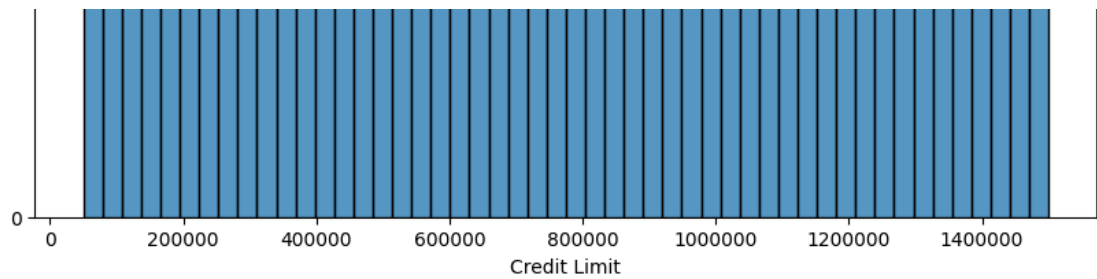

Distribution of credit Limit



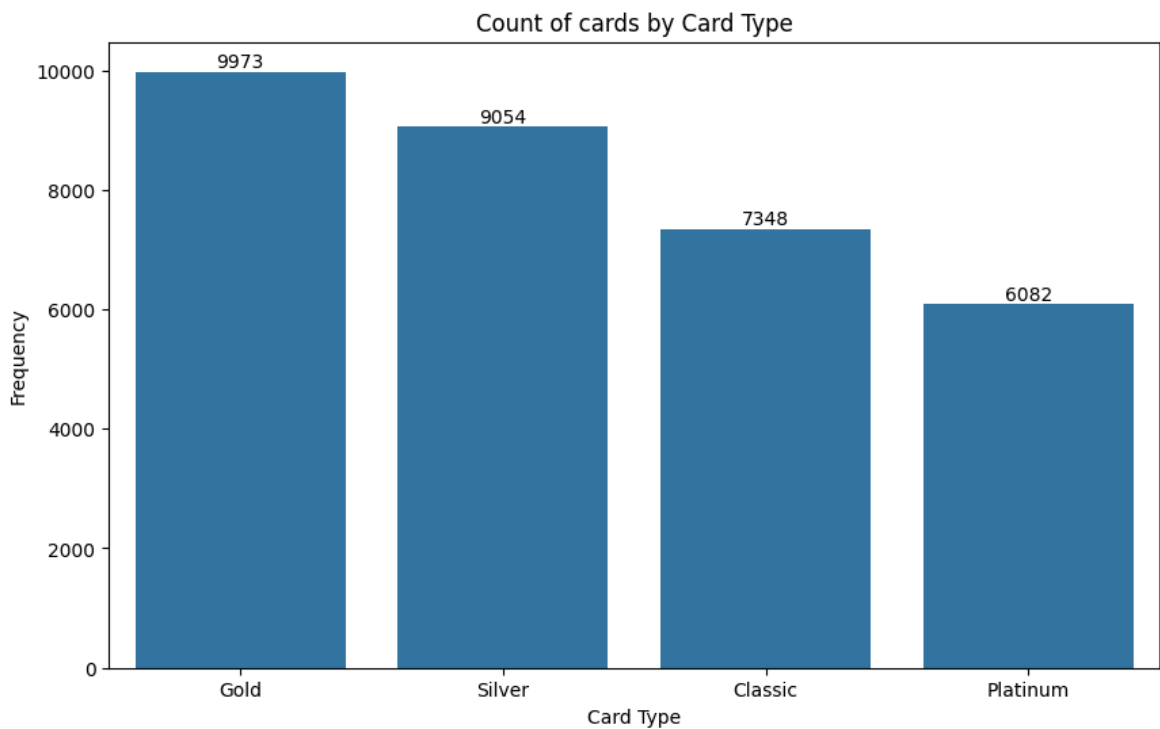


500

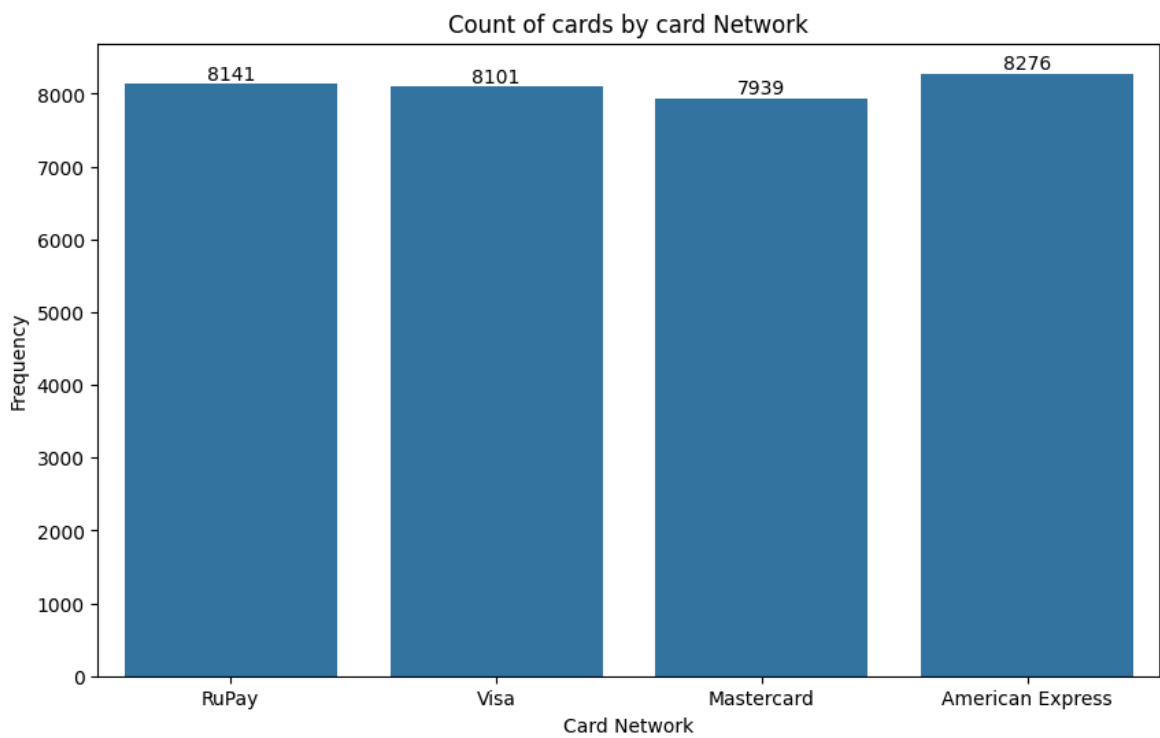




```
In [47]: plt.figure(figsize=(10,6))
sns.countplot(data=cards, x='Card_Type')
plt.title("Count of cards by Card Type")
plt.xlabel("Card Type")
plt.ylabel("Frequency")
plt.bar_label(plt.gca().containers[0])
plt.show()
```

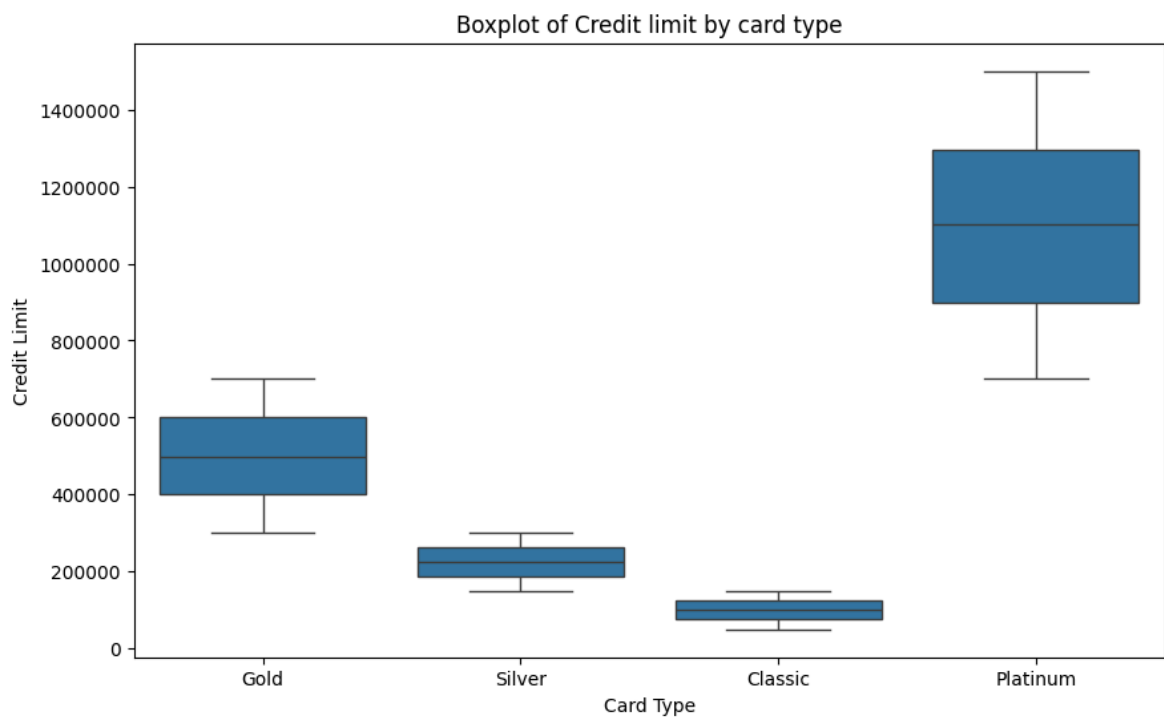


```
In [48]: plt.figure(figsize=(10,6))
sns.countplot(data=cards, x='Card_Network')
plt.title("Count of cards by card Network")
plt.xlabel("Card Network")
plt.ylabel("Frequency")
plt.bar_label(plt.gca().containers[0])
plt.show()
```



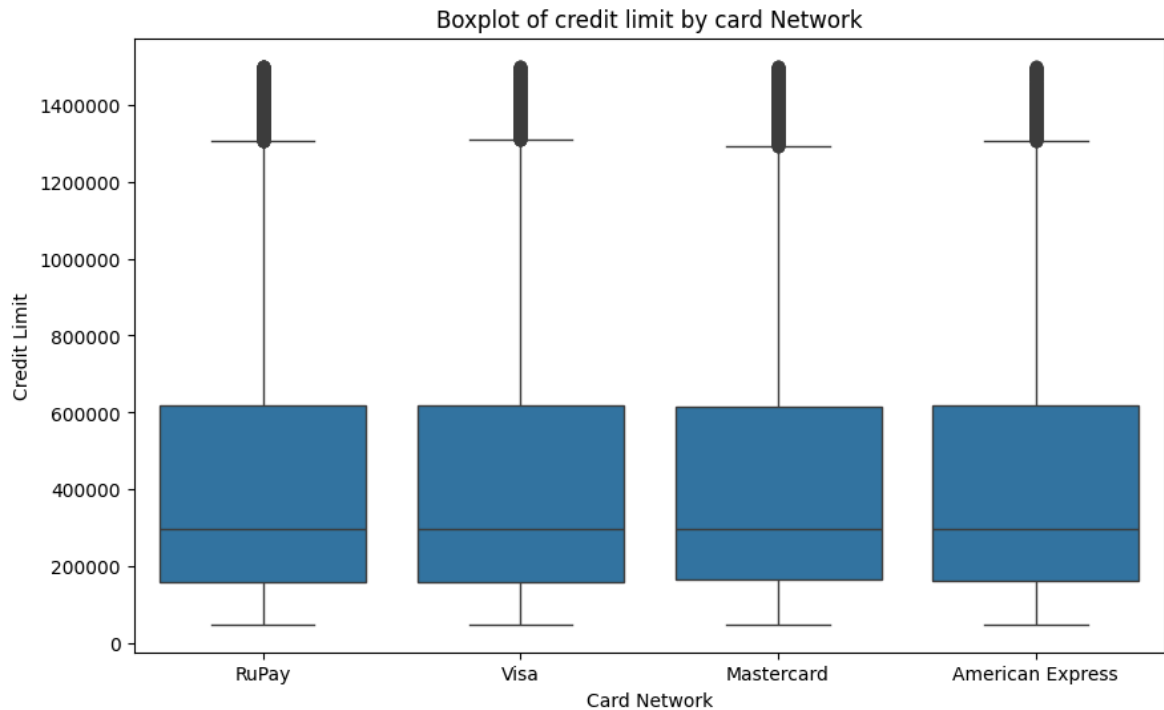
Bivariate Analysis

```
In [49]: plt.figure(figsize=(10,6))
sns.boxplot(data=cards, x='Card_Type', y='Credit_Limit')
plt.ticklabel_format(style='plain', axis='y')
plt.title("Boxplot of Credit limit by card type")
plt.xlabel("Card Type")
plt.ylabel("Credit Limit")
plt.show()
```



```
In [50]: plt.figure(figsize=(10,6))
sns.boxplot(data=cards, x = 'Card_Network', y='Credit_Limit')
plt.title("Boxplot of credit limit by card Network")
plt.xlabel("Card Network")
```

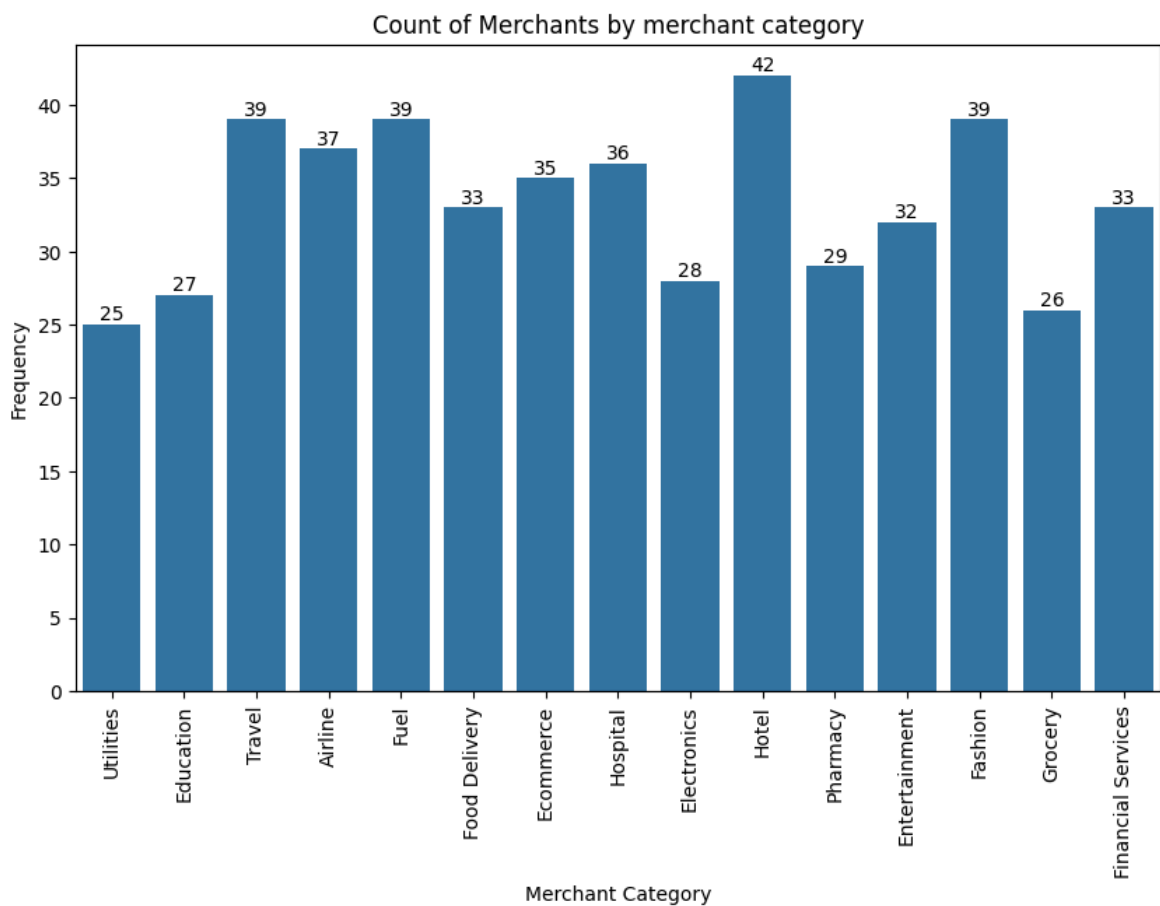
```
plt.ylabel("Credit Limit")
plt.ticklabel_format(style='plain', axis = 'y')
plt.show()
```



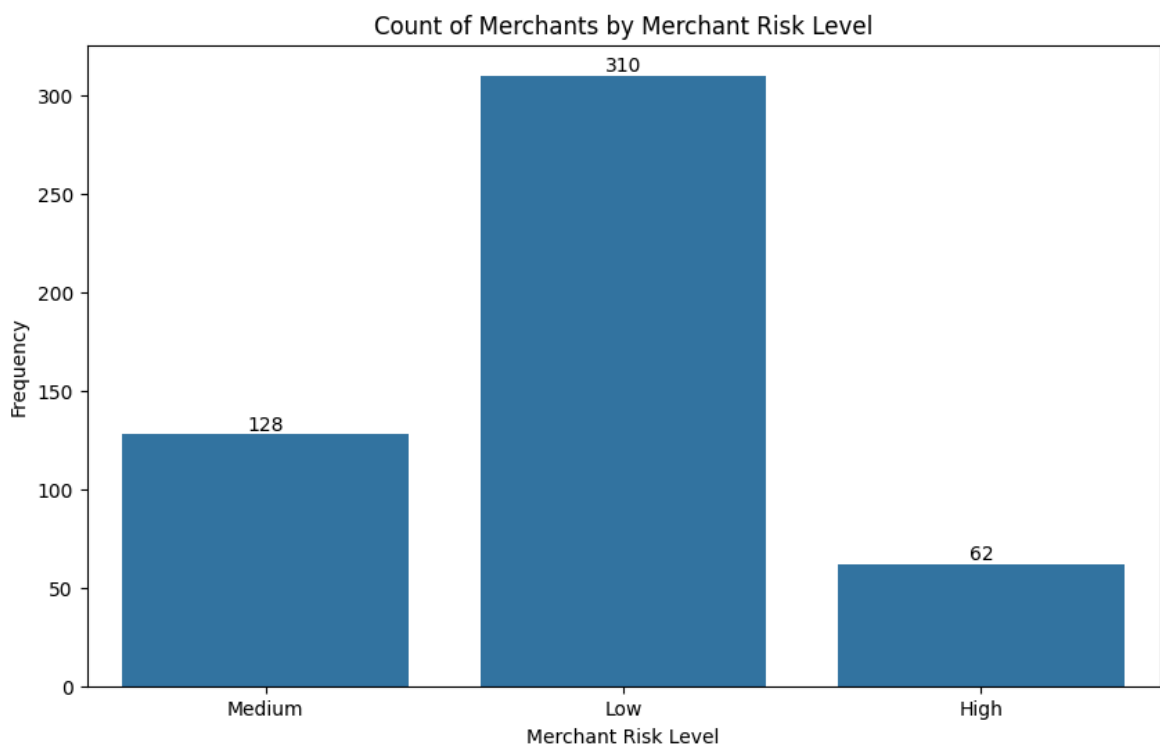
Merchant Table

Univariate Analysis

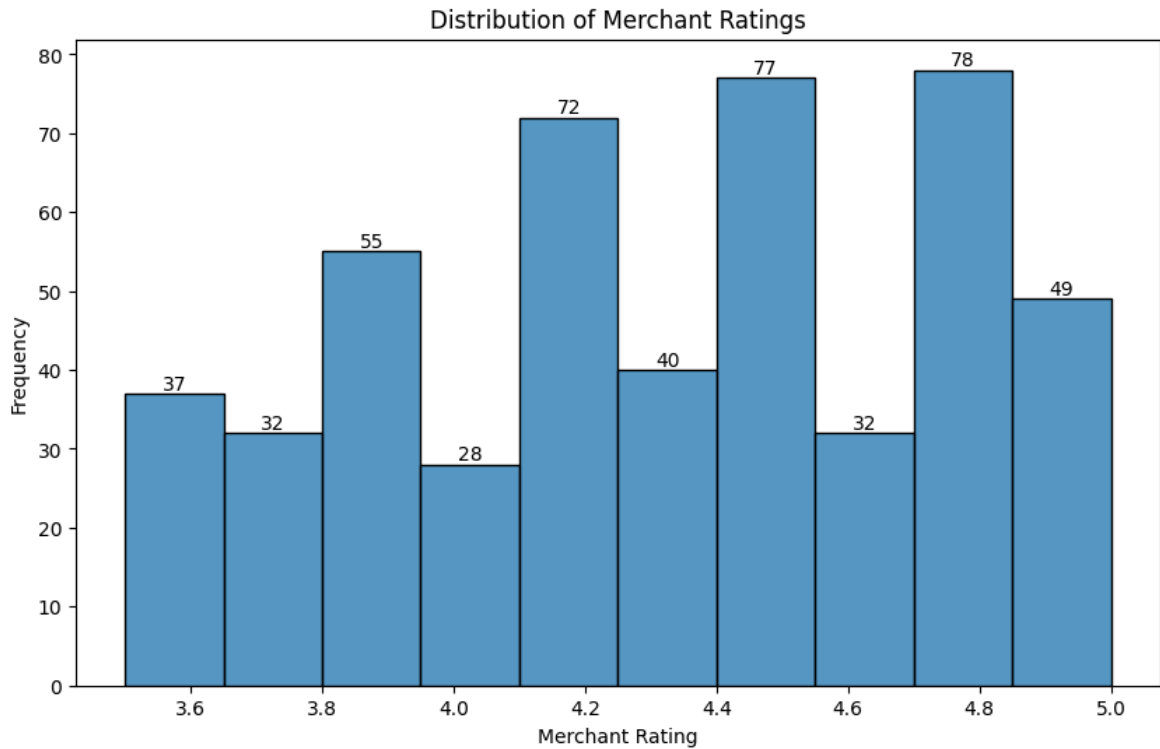
```
In [51]: plt.figure(figsize=(10,6))
sns.countplot(data=merchant, x='Merchant_Category')
plt.title("Count of Merchants by merchant category")
plt.xlabel("Merchant Category")
plt.ylabel("Frequency")
plt.xticks(rotation=90)
plt.bar_label(plt.gca().containers[0])
plt.show()
```



```
In [52]: plt.figure(figsize=(10,6))
sns.countplot(data=merchant, x='Merchant_Risk_Level')
plt.title("Count of Merchants by Merchant Risk Level")
plt.xlabel("Merchant Risk Level")
plt.ylabel("Frequency")
plt.bar_label(plt.gca().containers[0])
plt.show()
```

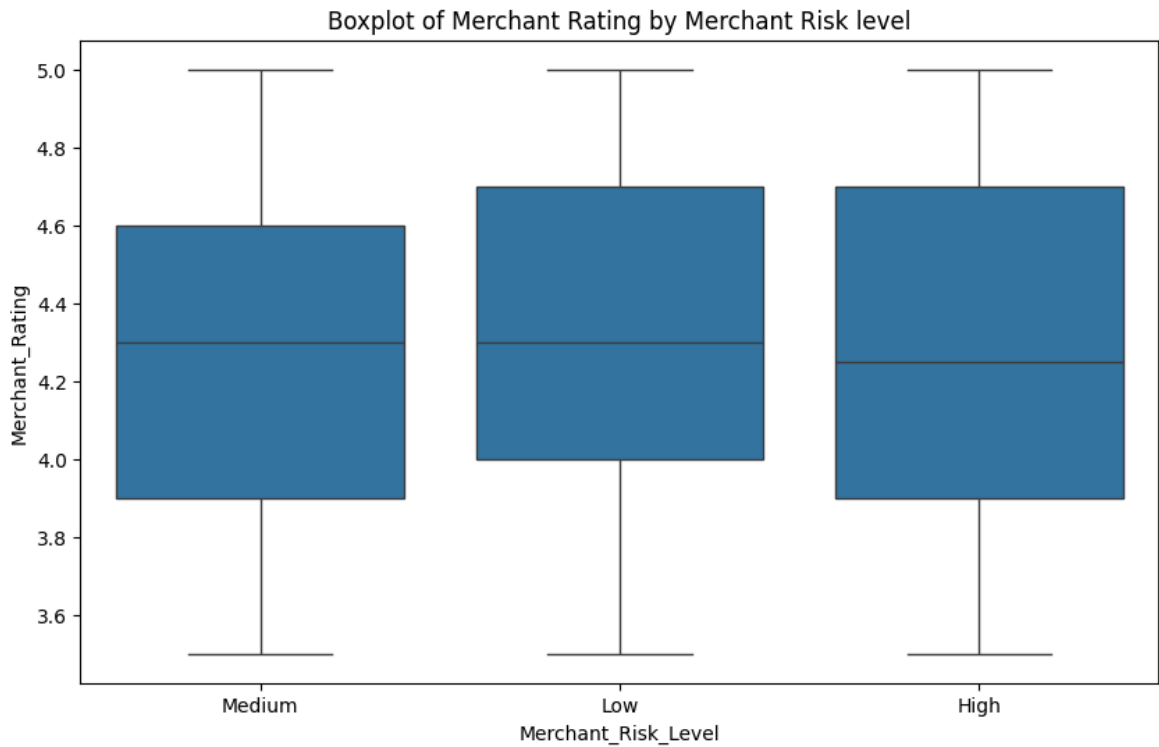


```
In [53]: plt.figure(figsize=(10,6))
sns.histplot(data=merchant, x='Merchant_Rating')
plt.title("Distribution of Merchant Ratings")
plt.xlabel("Merchant Rating")
plt.ylabel("Frequency")
plt.ticklabel_format(style='plain', axis='x')
plt.bar_label(plt.gca().containers[0])
plt.show()
```



Bivariate Analysis

```
In [54]: plt.figure(figsize=(10,6))
sns.boxplot(data=merchant, x="Merchant_Risk_Level", y="Merchant_Rating")
plt.title("Boxplot of Merchant Rating by Merchant Risk level")
plt.show()
```

Transaction Table

In [55]: `transaction.head()`

Out[55]:

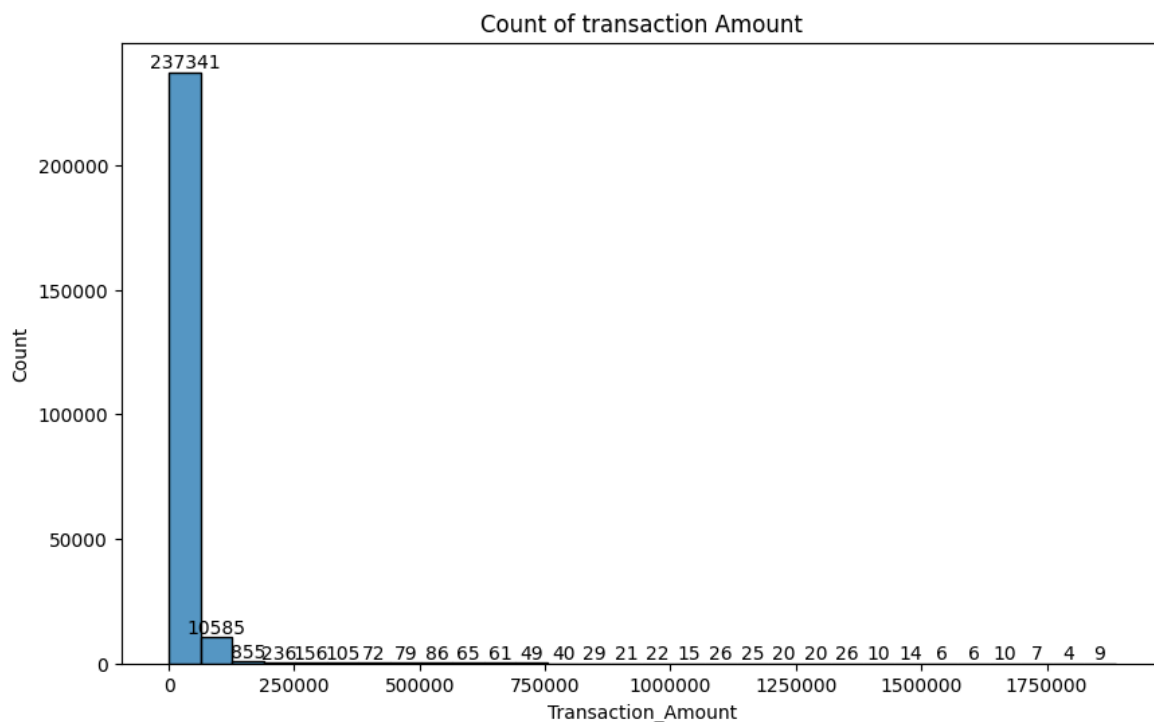
	Transaction_ID	Customer_ID	Card_ID	Merchant_ID	Transaction_Date	Transactio
--	----------------	-------------	---------	-------------	------------------	------------

0	TXN1000001	CUST100660	CARD500875	MER100380	2024-03-03	(
1	TXN1000002	CUST111388	CARD514772	MER100017	2023-11-08	(
2	TXN1000003	CUST115872	CARD520625	MER100143	2023-01-11	(
3	TXN1000004	CUST109049	CARD511773	MER100053	2023-05-25	(
4	TXN1000005	CUST114459	CARD518771	MER100064	2023-10-22	(



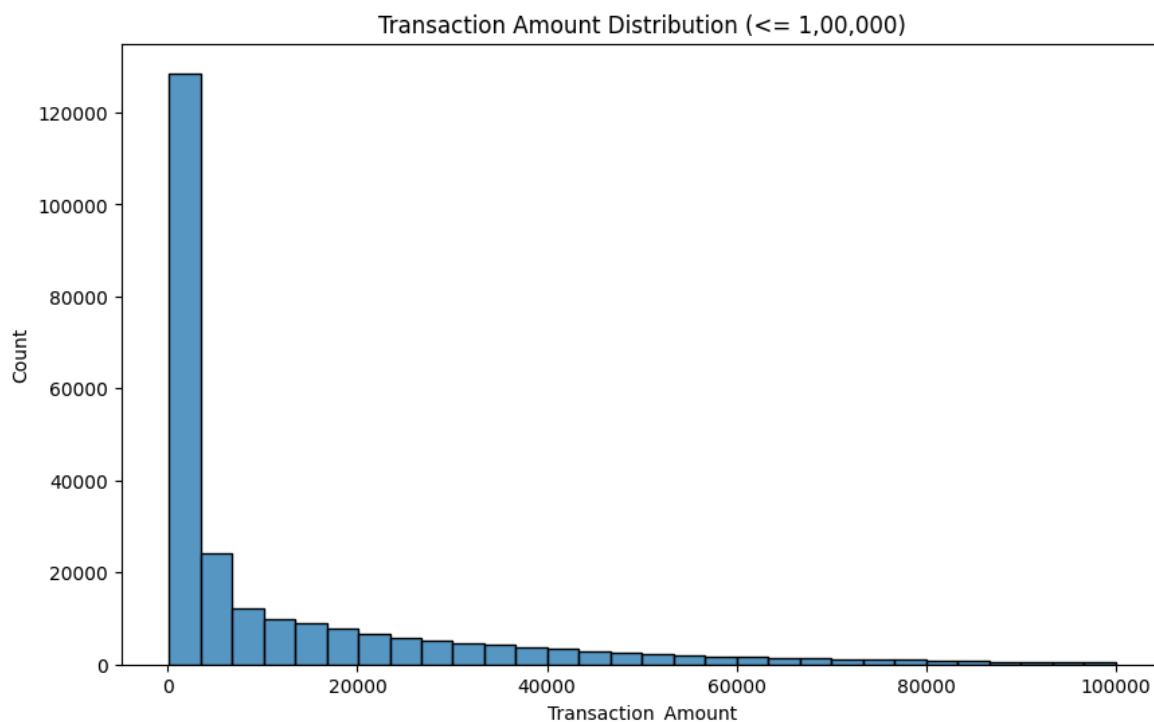
Univariate Analysis

```
In [56]: plt.figure(figsize=(10,6))
sns.histplot(data=transaction, x='Transaction_Amount', bins=30)
plt.ticklabel_format(style='plain', axis='x')
plt.title("Count of transaction Amount")
plt.bar_label(plt.gca().containers[0])
plt.show()
```

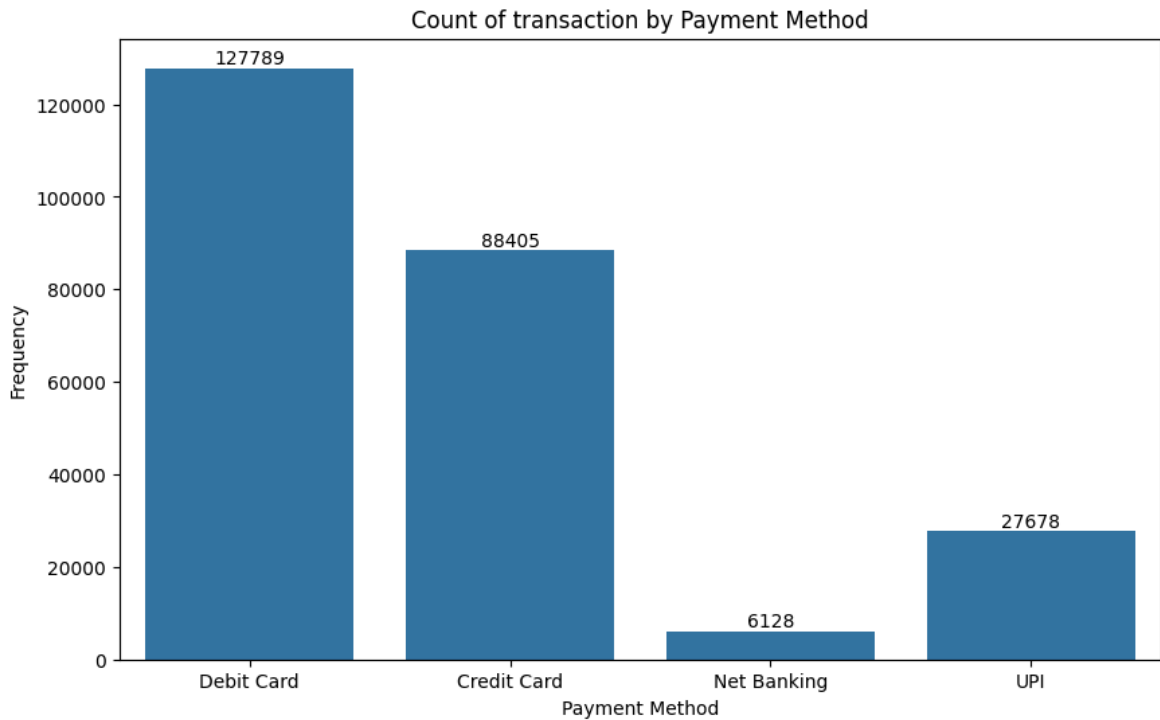


```
In [57]: plt.figure(figsize=(10,6))

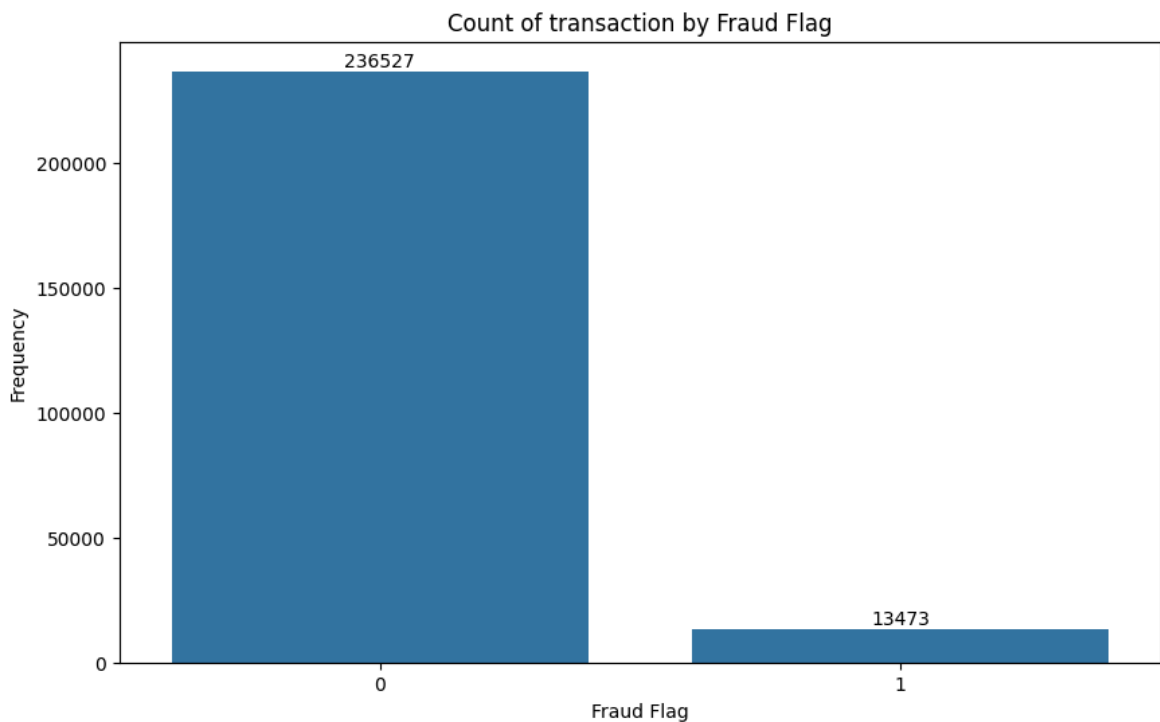
sns.histplot(transaction[transaction['Transaction_Amount'] <= 100000]['Transaction_Amount'])
plt.title("Transaction Amount Distribution (<= 1,00,000)")
plt.show()
```



```
In [58]: plt.figure(figsize=(10,6))
sns.countplot(data=transaction, x='Payment_Method')
plt.title('Count of transaction by Payment Method')
plt.bar_label(plt.gca().containers[0])
plt.xlabel("Payment Method")
plt.ylabel("Frequency")
plt.show()
```

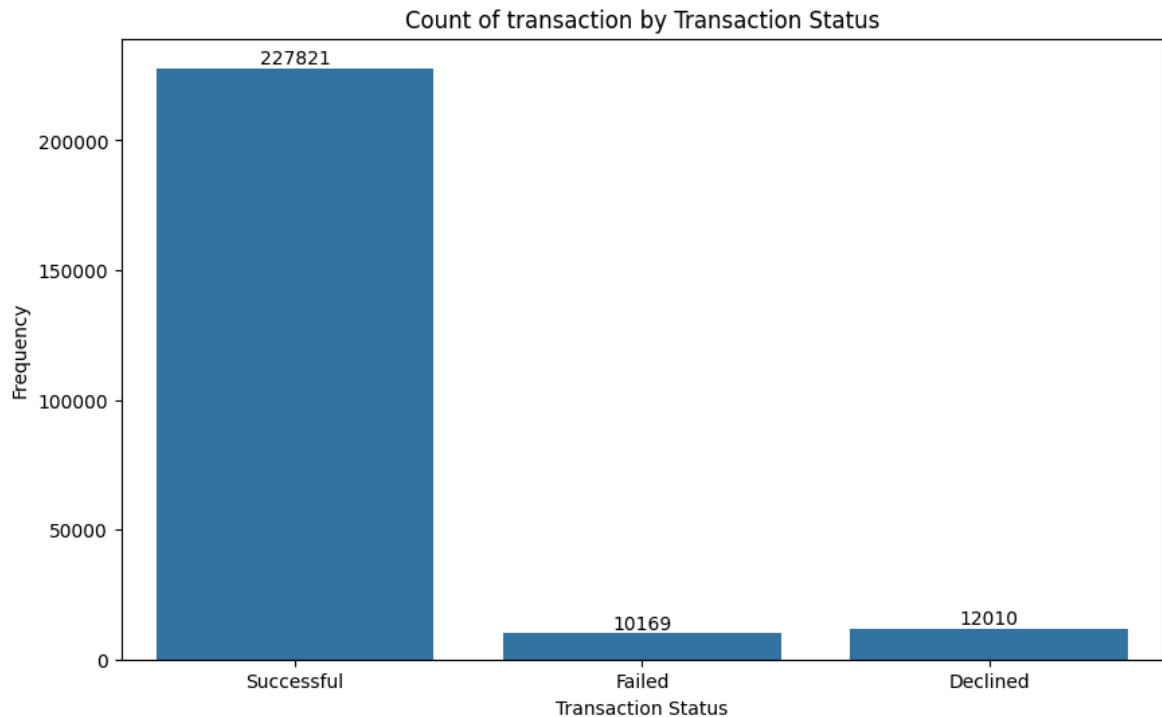


```
In [59]: plt.figure(figsize=(10,6))
sns.countplot(data=transaction, x='Fraud_Flag')
plt.title("Count of transaction by Fraud Flag")
plt.bar_label(plt.gca().containers[0])
plt.xlabel("Fraud Flag")
plt.ylabel("Frequency")
plt.show()
```



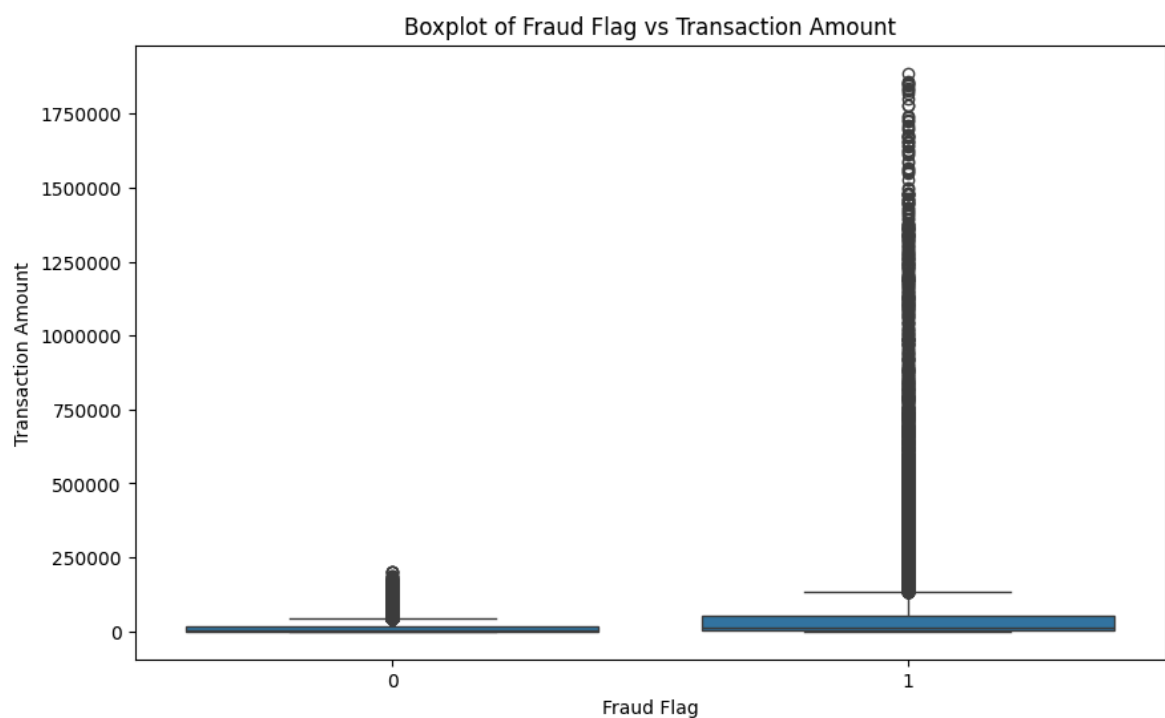
```
In [60]: plt.figure(figsize=(10,6))
sns.countplot(data=transaction, x='Transaction_Status')
plt.title("Count of transaction by Transaction Status")
plt.bar_label(plt.gca().containers[0])
plt.xlabel("Transaction Status")
```

```
plt.ylabel("Frequency")
plt.show()
```



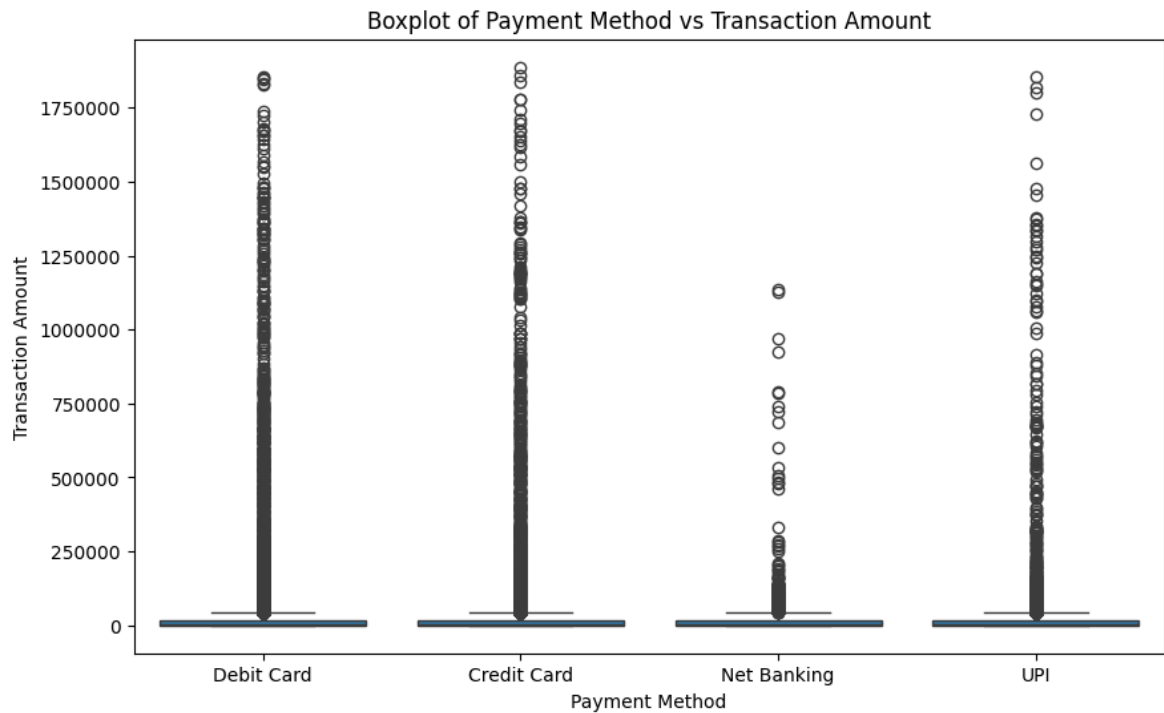
Bivariate Analysis

```
In [61]: plt.figure(figsize=(10,6))
sns.boxplot(data=transaction, x='Fraud_Flag', y='Transaction_Amount')
plt.title("Boxplot of Fraud Flag vs Transaction Amount")
plt.xlabel("Fraud Flag")
plt.ylabel("Transaction Amount")
plt.ticklabel_format(style='plain', axis='y')
plt.show()
```



```
In [62]: plt.figure(figsize=(10,6))
sns.boxplot(data=transaction, x='Payment_Method', y='Transaction_Amount')
```

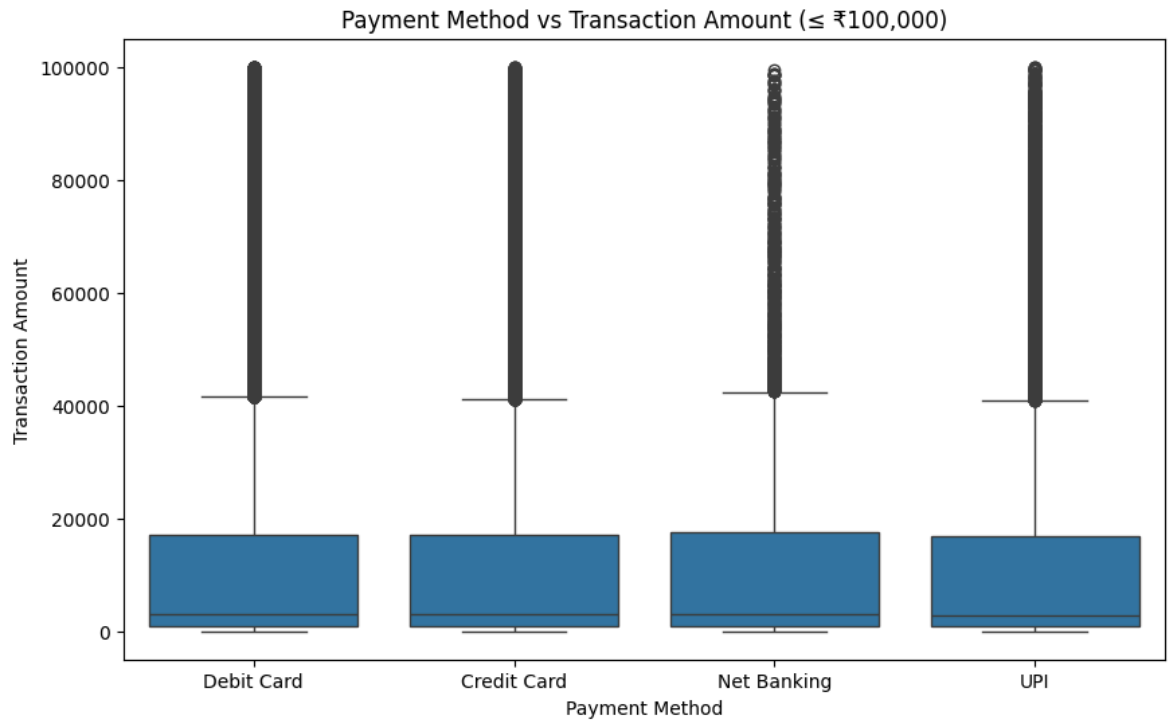
```
plt.title("Boxplot of Payment Method vs Transaction Amount")
plt.ticklabel_format(style='plain', axis='y')
plt.xlabel("Payment Method")
plt.ylabel("Transaction Amount")
plt.show()
```



```
In [63]: # Zoomed version for better clearly and visualization
plt.figure(figsize=(10,6))

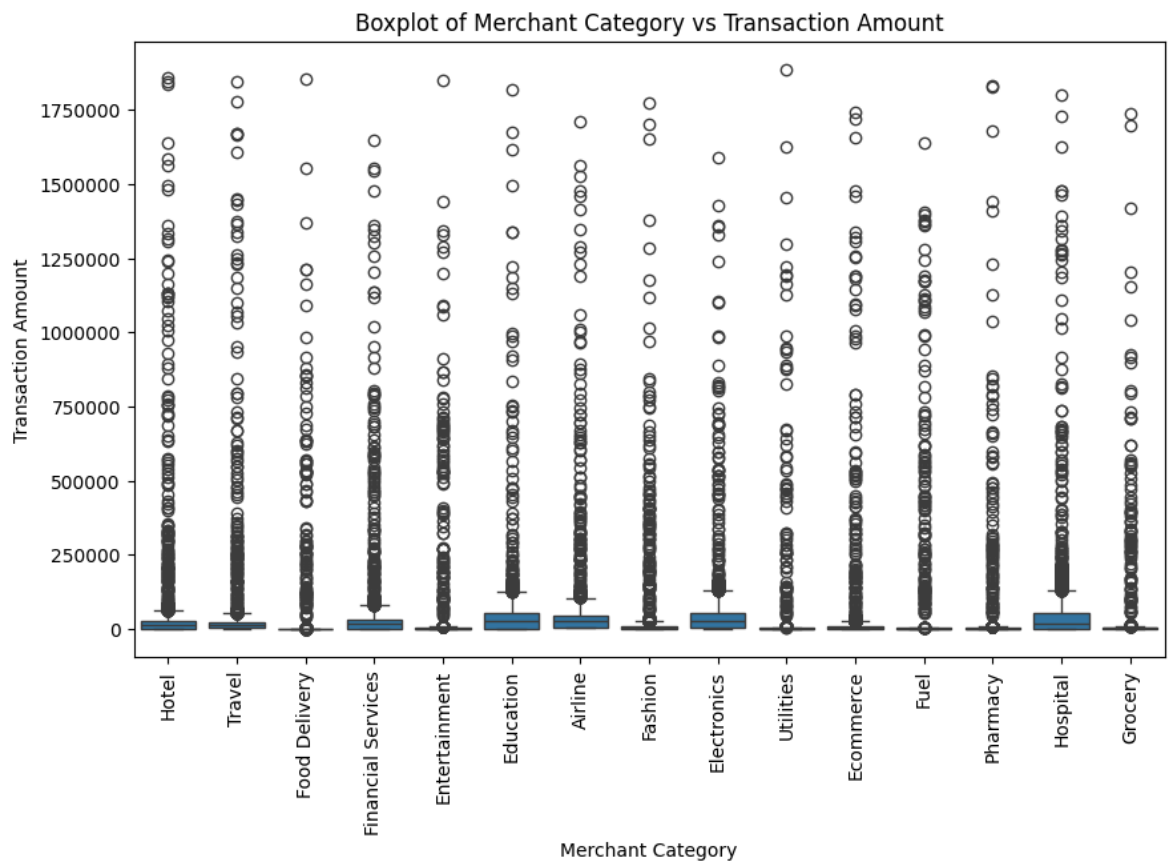
sns.boxplot(
    data=transaction[transaction['Transaction_Amount'] <= 100000],
    x='Payment_Method',
    y='Transaction_Amount'
)

plt.title('Payment Method vs Transaction Amount (≤ ₹100,000)')
plt.xlabel('Payment Method')
plt.ylabel('Transaction Amount')
plt.show()
```



```
In [64]: plt.figure(figsize=(10,6))

sns.boxplot(data=transaction, x='Merchant_Category', y='Transaction_Amount')
plt.title("Boxplot of Merchant Category vs Transaction Amount")
plt.ticklabel_format(style='plain', axis='y')
plt.xticks(rotation=90)
plt.xlabel("Merchant Category")
plt.ylabel("Transaction Amount")
plt.show()
```



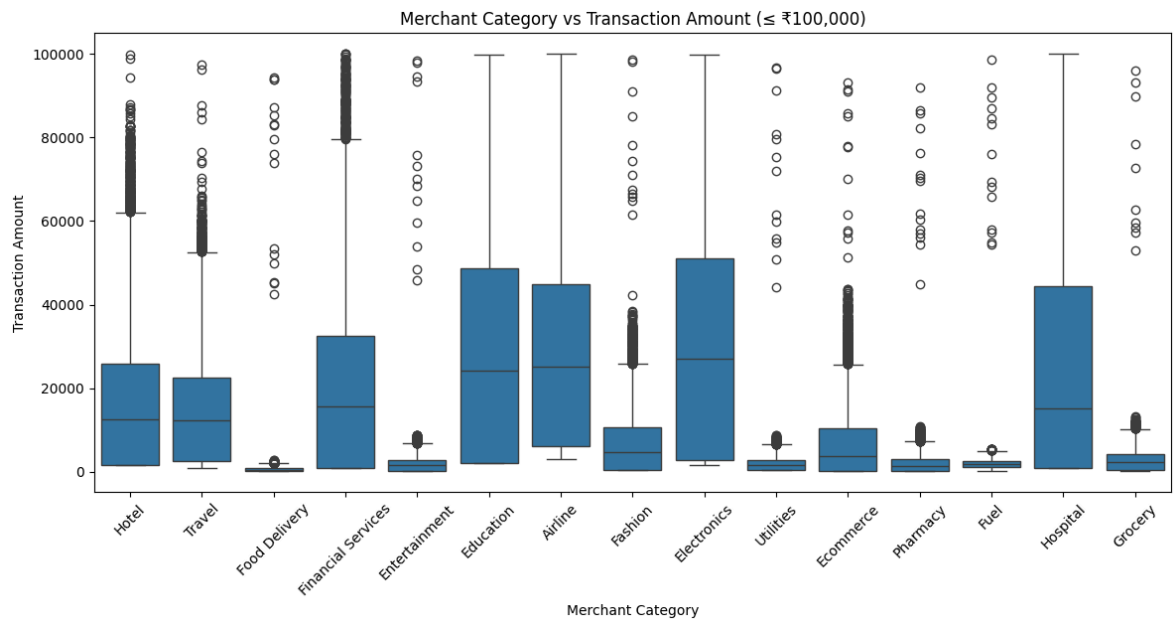
```

In [65]: plt.figure(figsize=(14,6))

sns.boxplot(
    data=transaction[transaction['Transaction_Amount'] <= 100000],
    x='Merchant_Category',
    y='Transaction_Amount'
)

plt.xticks(rotation=45)
plt.title('Merchant Category vs Transaction Amount (≤ ₹100,000)')
plt.xlabel('Merchant Category')
plt.ylabel('Transaction Amount')
plt.show()

```



```

In [66]: merge_cc = customer.merge(cards,
    on = 'Customer_ID',
    how = 'inner')

```

```

In [67]: merge_cc.shape

```

```

Out[67]: (32457, 21)

```

```

In [68]: merge_cc.isnull().sum()

```

```
Out[68]: Customer_ID      0
Customer_Name      0
Gender              0
Age                0
Marital_Status     0
Occupation          0
Annual_Income       0
Customer_Segment    0
State              0
City               0
Account_Type        0
Customer_Since      0
Card_ID             0
Card_Type           0
Card_Network        0
Credit_Limit        0
Card_Status         0
Contactless         0
Card_Mode           0
Issue_Date          0
Expiry_Date         0
dtype: int64
```

```
In [69]: merge_cc.head()
```

```
Out[69]:
```

	Customer_ID	Customer_Name	Gender	Age	Marital_Status	Occupation	Annual_Inc
0	CUST100001	Omaja Majumdar	Female	27	Single	Police Officer	116
1	CUST100002	Jason Agrawal	Female	41	Married	Teacher	49
2	CUST100003	Amrita Dora	Male	61	Divorsed	Business	186
3	CUST100004	Radhika Batra	Male	26	Single	Software Engineer	80
4	CUST100005	Gautam Mukhopadhyay	Female	24	Single	Software Engineer	158

5 rows × 21 columns



```
In [70]: merge_cct = merge_cc.merge(transaction,
on=['Customer_ID', 'Card_ID'],
how='inner')
```

```
In [71]: merge_cct.shape
```

```
Out[71]: (250000, 39)
```

```
In [72]: merge_cct.isnull().sum()
```



```
Out[72]: Customer_ID      0
Customer_Name      0
Gender             0
Age               0
Marital_Status     0
Occupation         0
Annual_Income      0
Customer_Segment   0
State              0
City               0
Account_Type       0
Customer_Since     0
Card_ID            0
Card_Type          0
Card_Network       0
Credit_Limit      0
Card_Status        0
Contactless        0
Card_Mode          0
Issue_Date         0
Expiry_Date        0
Transaction_ID     0
Merchant_ID        0
Transaction_Date    0
Transaction_Time    0
Transaction_Amount  0
Payment_Method      0
Transaction_Channel 0
Device_Type        0
Transaction_Status  0
Is_International   0
Fraud_Flag         0
Fraud_Reason       236527
Merchant_Risk_Level 0
Merchant_Category   0
Customer_State      0
Customer_City       0
Merchant_State      0
Merchant_City       0
dtype: int64
```

```
In [73]: merge_cct.head()
```

Out[73]:

	Customer_ID	Customer_Name	Gender	Age	Marital_Status	Occupation	Annual_Inc
0	CUST100001	Omaja Majumdar	Female	27	Single	Police Officer	116
1	CUST100001	Omaja Majumdar	Female	27	Single	Police Officer	116
2	CUST100001	Omaja Majumdar	Female	27	Single	Police Officer	116
3	CUST100001	Omaja Majumdar	Female	27	Single	Police Officer	116
4	CUST100001	Omaja Majumdar	Female	27	Single	Police Officer	116

5 rows × 39 columns



In [74]: `final_merge = merge_cct.merge(merchant,
on='Merchant_ID',
how='inner')`

In [75]: `final_merge.shape`

Out[75]: (250000, 47)

In [76]: `final_merge.isnull().sum()`

```

Out[76]: Customer_ID          0
         Customer_Name        0
         Gender               0
         Age                  0
         Marital_Status       0
         Occupation           0
         Annual_Income        0
         Customer_Segment     0
         State_x              0
         City_x               0
         Account_Type         0
         Customer_Since       0
         Card_ID              0
         Card_Type            0
         Card_Network         0
         Credit_Limit         0
         Card_Status          0
         Contactless          0
         Card_Mode            0
         Issue_Date           0
         Expiry_Date          0
         Transaction_ID       0
         Merchant_ID          0
         Transaction_Date     0
         Transaction_Time     0
         Transaction_Amount   0
         Payment_Method       0
         Transaction_Channel   0
         Device_Type          0
         Transaction_Status   0
         Is_International     0
         Fraud_Flag           0
         Fraud_Reason         236527
         Merchant_Risk_Level_x 0
         Merchant_Category_x   0
         Customer_State        0
         Customer_City         0
         Merchant_State        0
         Merchant_City         0
         Merchant_Name         0
         Merchant_Category_y   0
         State_y              0
         City_y               0
         Merchant_Risk_Level_y 0
         Merchant_Rating       0
         Merchant_Status       0
         Merchant_Since        0
         dtype: int64

```

```

In [77]: final_merge.head()

```

```
Out[77]:
```

	Customer_ID	Customer_Name	Gender	Age	Marital_Status	Occupation	Annual_Inc
0	CUST100001	Omaja Majumdar	Female	27	Single	Police Officer	116
1	CUST100001	Omaja Majumdar	Female	27	Single	Police Officer	116
2	CUST100001	Omaja Majumdar	Female	27	Single	Police Officer	116
3	CUST100001	Omaja Majumdar	Female	27	Single	Police Officer	116
4	CUST100001	Omaja Majumdar	Female	27	Single	Police Officer	116

5 rows × 47 columns

```
In [78]: final_merge.columns
```

```
Out[78]: Index(['Customer_ID', 'Customer_Name', 'Gender', 'Age', 'Marital_Status',
               'Occupation', 'Annual_Income', 'Customer_Segment', 'State_x', 'City_x',
               'Account_Type', 'Customer_Since', 'Card_ID', 'Card_Type',
               'Card_Network', 'Credit_Limit', 'Card_Status', 'Contactless',
               'Card_Mode', 'Issue_Date', 'Expiry_Date', 'Transaction_ID',
               'Merchant_ID', 'Transaction_Date', 'Transaction_Time',
               'Transaction_Amount', 'Payment_Method', 'Transaction_Channel',
               'Device_Type', 'Transaction_Status', 'Is_International', 'Fraud_Flag',
               'Fraud_Reason', 'Merchant_Risk_Level_x', 'Merchant_Category_x',
               'Customer_State', 'Customer_City', 'Merchant_State', 'Merchant_City',
               'Merchant_Name', 'Merchant_Category_y', 'State_y', 'City_y',
               'Merchant_Risk_Level_y', 'Merchant_Rating', 'Merchant_Status',
               'Merchant_Since'],
              dtype='str')
```

```
In [79]: (final_merge['Merchant_Category_x'] == final_merge['Merchant_Category_y']).all()
```

```
Out[79]: np.True_
```

```
In [80]: (final_merge['Merchant_Risk_Level_x'] == final_merge['Merchant_Risk_Level_y']).a
```

```
Out[80]: np.True_
```

```
In [81]: (final_merge['State_x'] == final_merge['State_y']).all()
```

```
Out[81]: np.False_
```

```
In [82]: (final_merge['City_x'] == final_merge['City_y']).all()
```

```
Out[82]: np.False_
```

```
In [83]: # Drop the unnecessary duplicate columns
         final_merge = final_merge.drop(columns=[
             'State_x',
             'City_x',
             'State_y',
```

```
    'City_y',  
    'Merchant_Category_y',  
    'Merchant_Risk_Level_y'  
])
```

```
In [84]: #Rename the remaining duplicates columns  
final_merge.rename(columns={  
    'Merchant_Category_x': 'Merchant_Category',  
    'Merchant_Risk_Level_x': 'Merchant_Risk_Level'  
}, inplace=True)
```

```
In [85]: final_merge.columns
```

```
Out[85]: Index(['Customer_ID', 'Customer_Name', 'Gender', 'Age', 'Marital_Status',  
    'Occupation', 'Annual_Income', 'Customer_Segment', 'Account_Type',  
    'Customer_Since', 'Card_ID', 'Card_Type', 'Card_Network',  
    'Credit_Limit', 'Card_Status', 'Contactless', 'Card_Mode', 'Issue_Date',  
    'Expiry_Date', 'Transaction_ID', 'Merchant_ID', 'Transaction_Date',  
    'Transaction_Time', 'Transaction_Amount', 'Payment_Method',  
    'Transaction_Channel', 'Device_Type', 'Transaction_Status',  
    'Is_International', 'Fraud_Flag', 'Fraud_Reason', 'Merchant_Risk_Level',  
    'Merchant_Category', 'Customer_State', 'Customer_City',  
    'Merchant_State', 'Merchant_City', 'Merchant_Name', 'Merchant_Rating',  
    'Merchant_Status', 'Merchant_Since'],  
    dtype='str')
```

```
In [86]: final_merge.dtypes
```

```
Out[86]: Customer_ID          str
         Customer_Name       str
         Gender              str
         Age                 int64
         Marital_Status      str
         Occupation          str
         Annual_Income       int64
         Customer_Segment    str
         Account_Type        str
         Customer_Since      datetime64[us]
         Card_ID             str
         Card_Type           str
         Card_Network        str
         Credit_Limit        int64
         Card_Status         str
         Contactless         str
         Card_Mode           str
         Issue_Date          datetime64[us]
         Expiry_Date         datetime64[us]
         Transaction_ID       str
         Merchant_ID         str
         Transaction_Date     datetime64[us]
         Transaction_Time     str
         Transaction_Amount   float64
         Payment_Method       str
         Transaction_Channel  str
         Device_Type         str
         Transaction_Status   str
         Is_International     int64
         Fraud_Flag          int64
         Fraud_Reason        str
         Merchant_Risk_Level  str
         Merchant_Category    str
         Customer_State       str
         Customer_City        str
         Merchant_State       str
         Merchant_City        str
         Merchant_Name        str
         Merchant_Rating      float64
         Merchant_Status      str
         Merchant_Since      datetime64[us]
         dtype: object
```

```
In [87]: final_merge.duplicated().sum()
```

```
Out[87]: np.int64(0)
```

```
In [88]: final_merge.describe()
```

Out[88]:

	Age	Annual_Income	Customer_Since	Credit_Limit	Issue_Date	Exp
count	250000.00	250000.00	250000	250000.00	250000	
mean	41.83	1608372.22	2021-07-09 05:47:11.385599	443544.51	2022-07-20 16:00:57.484800	2021-07-20 21:55:21.385599
min	18.00	2.00	2016-07-19 00:00:00	50002.00	2018-07-20 00:00:00	2016-07-19 00:00:00
25%	31.00	570206.00	2018-12-26 00:00:00	161491.00	2020-07-18 00:00:00	2018-12-26 00:00:00
50%	41.00	963103.00	2021-07-04 00:00:00	296589.00	2022-07-25 00:00:00	2021-07-04 00:00:00
75%	53.00	1861271.00	2024-01-05 00:00:00	615720.00	2024-07-23 00:00:00	2024-01-05 00:00:00
max	70.00	9999931.00	2026-07-19 00:00:00	1499927.00	2026-07-19 00:00:00	2026-07-19 00:00:00
std	13.07	1732134.04	NaN	368654.67	NaN	

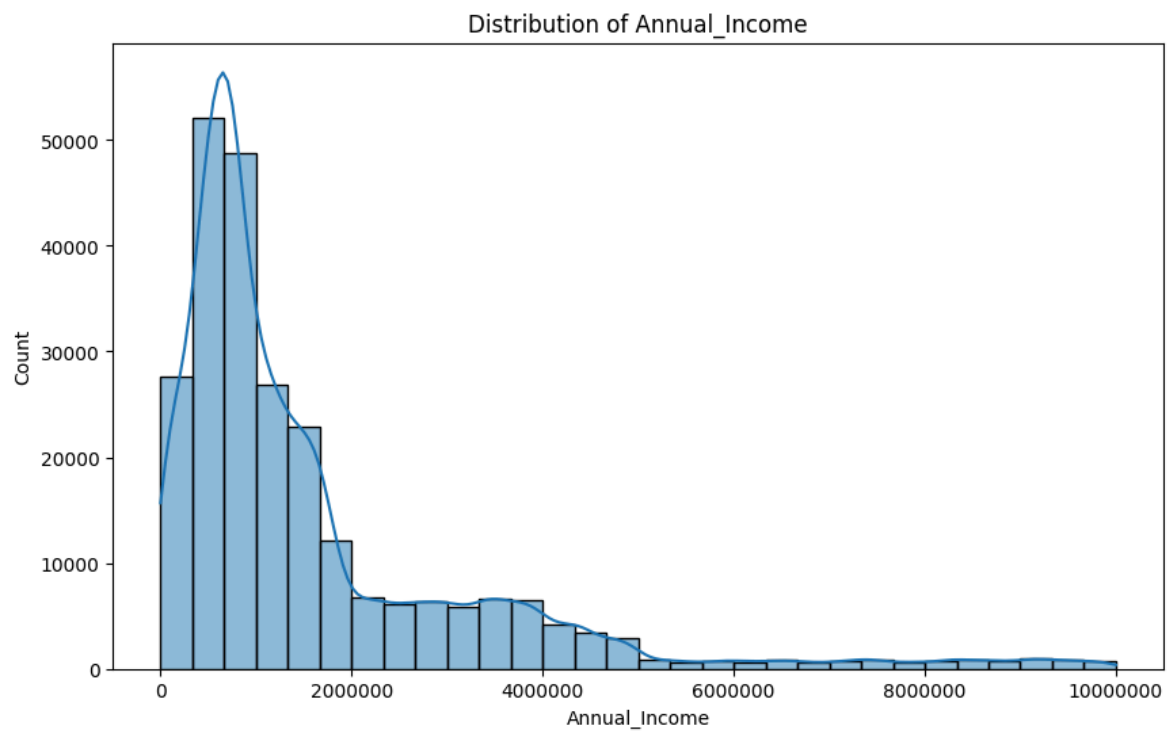
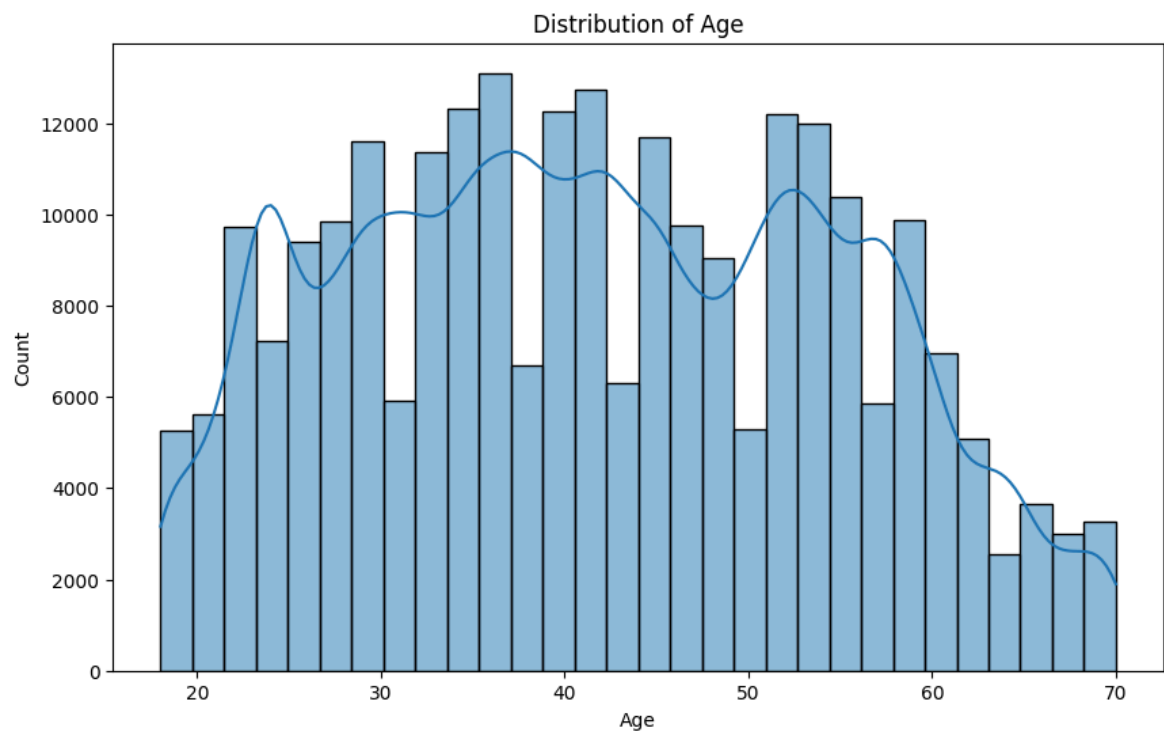
Final Merge Table EDA

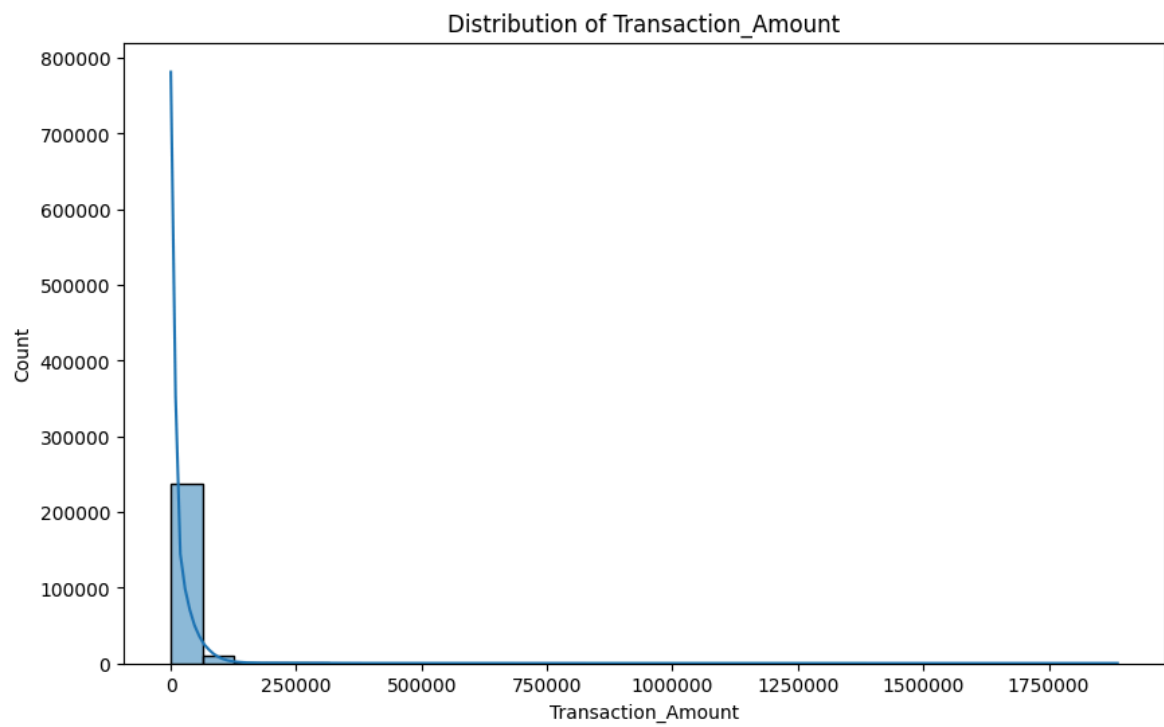
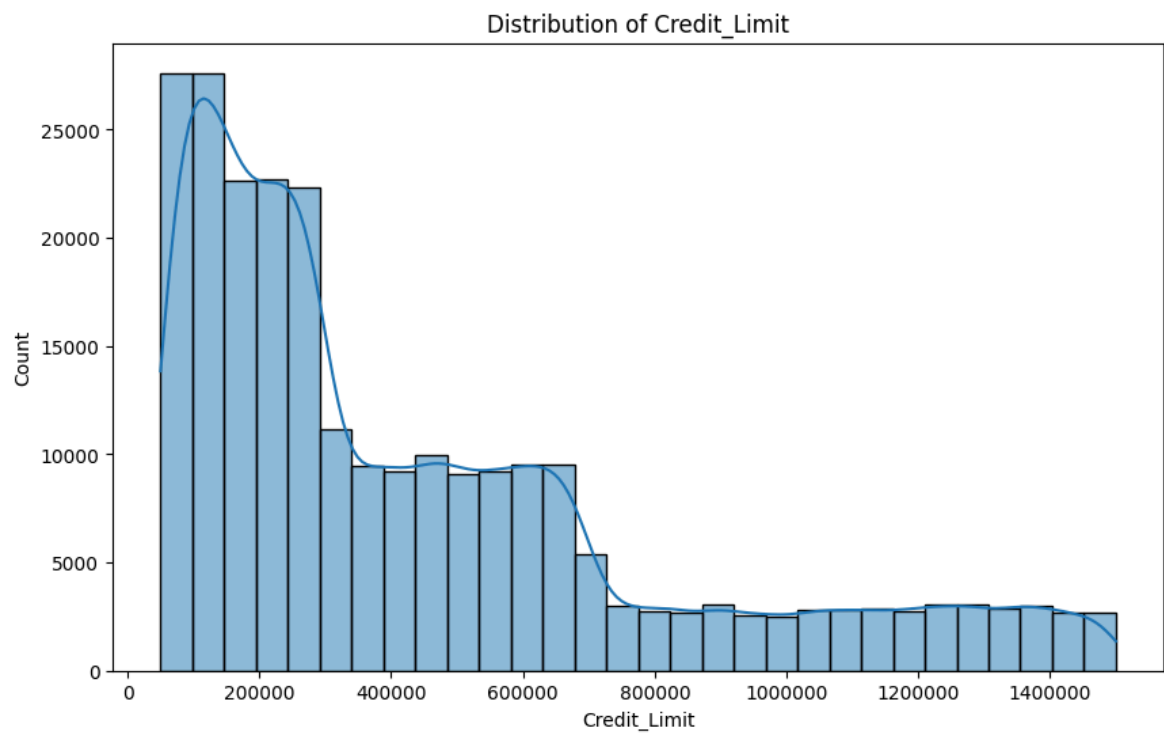
In [89]:

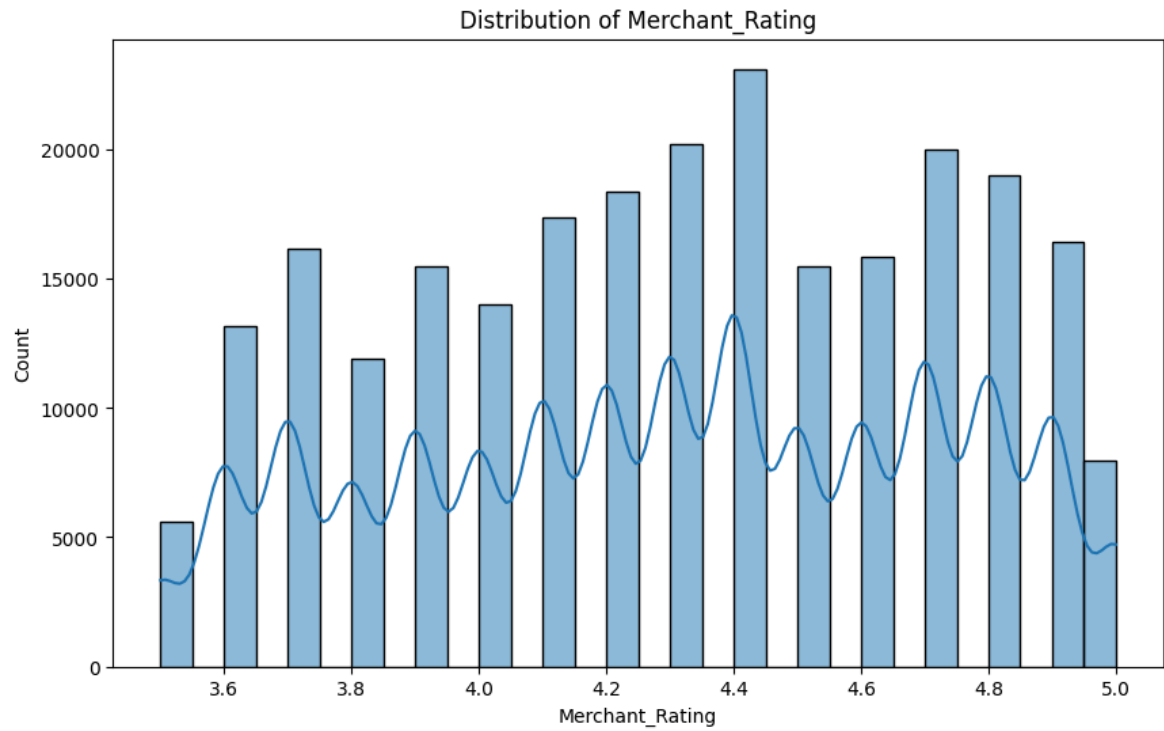
```
num_cols =[
    'Age',
    'Annual_Income',
    'Credit_Limit',
    'Transaction_Amount',
    'Merchant_Rating'
]
```

In [90]:

```
for col in num_cols:
    plt.figure(figsize=(10,6))
    sns.histplot(final_merge[col], bins=30, kde=True)
    plt.title(f'Distribution of {col}')
    plt.ticklabel_format(style='plain', axis = 'x')
    plt.show()
```

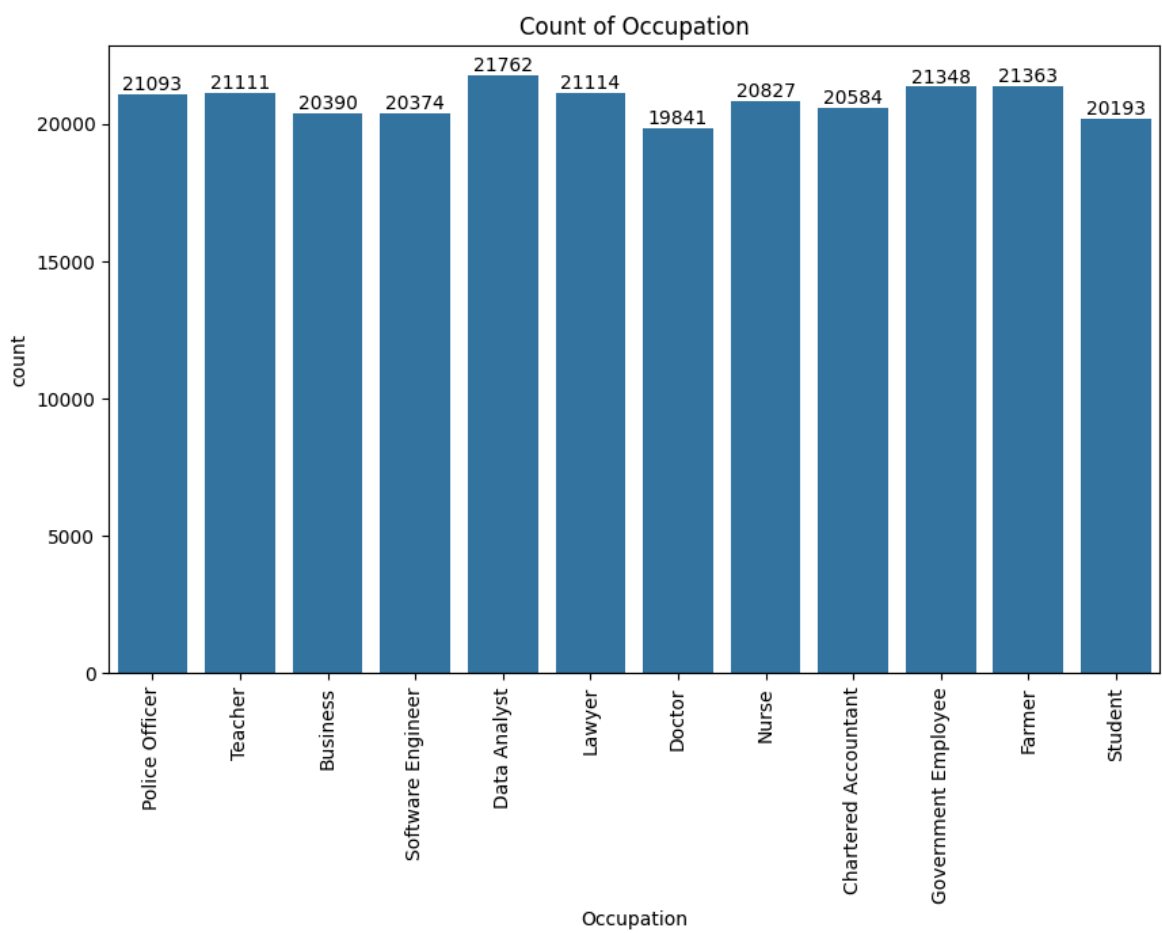
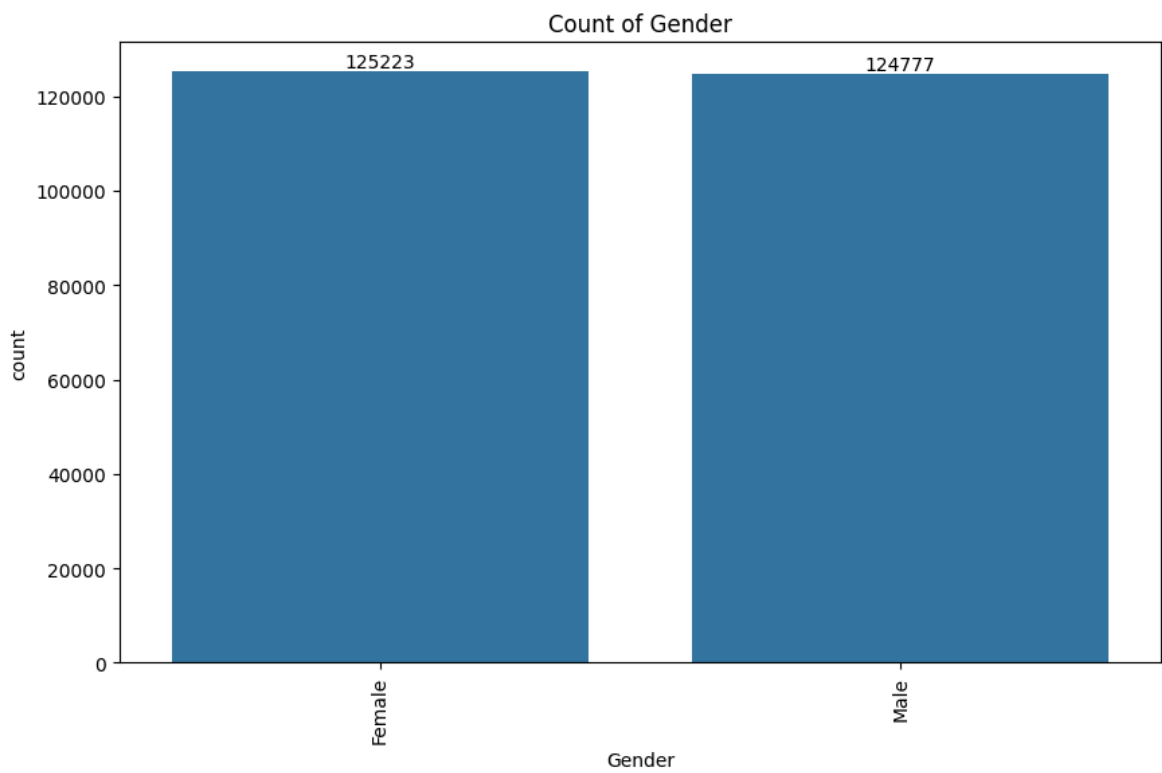


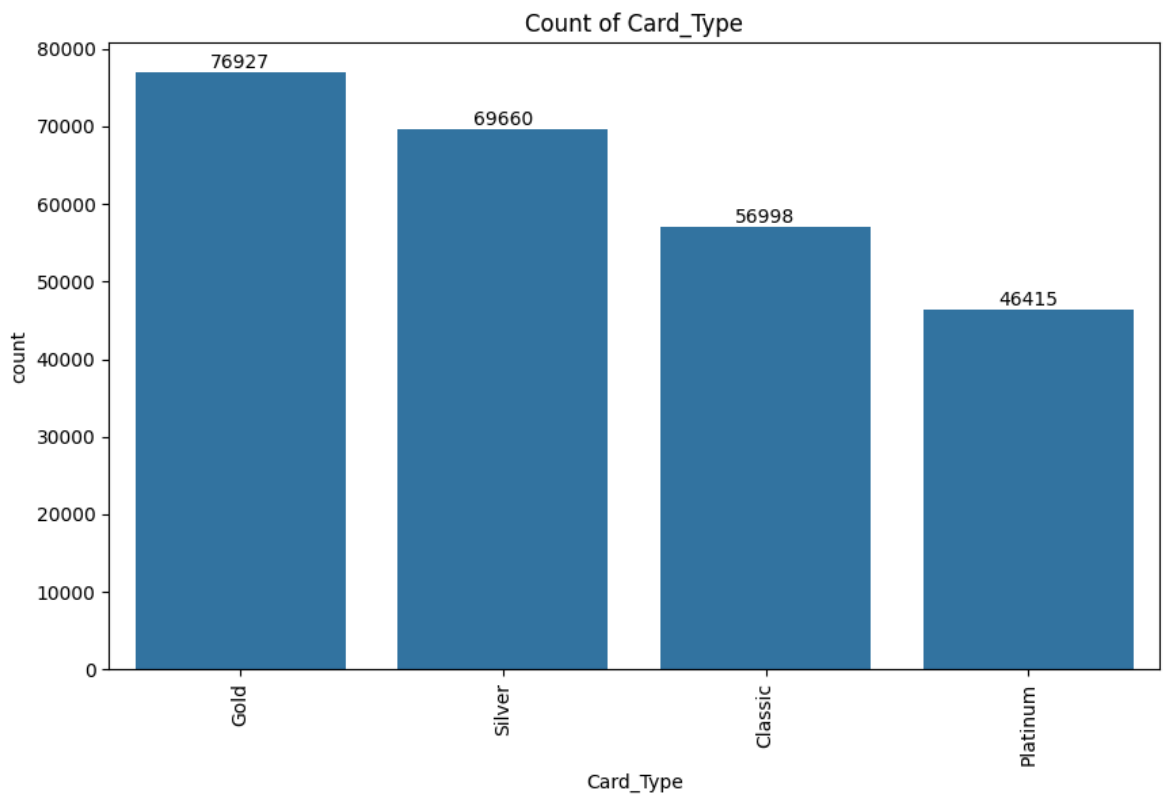
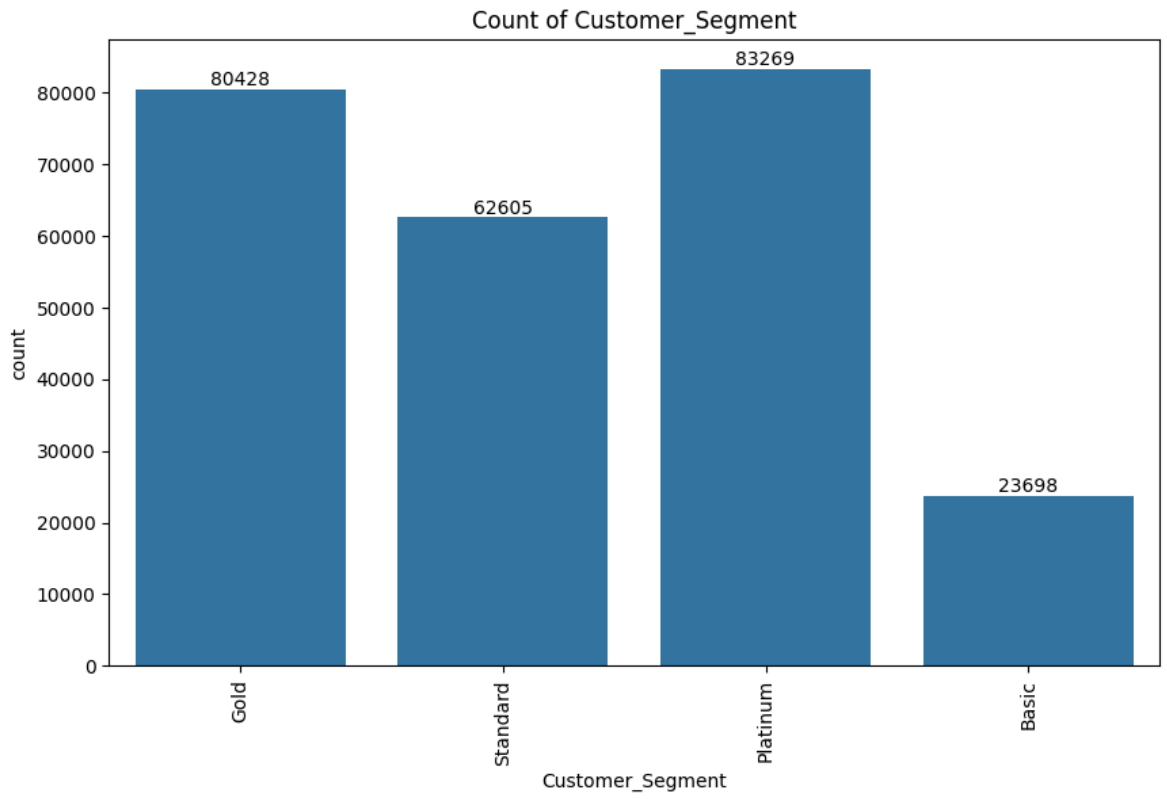


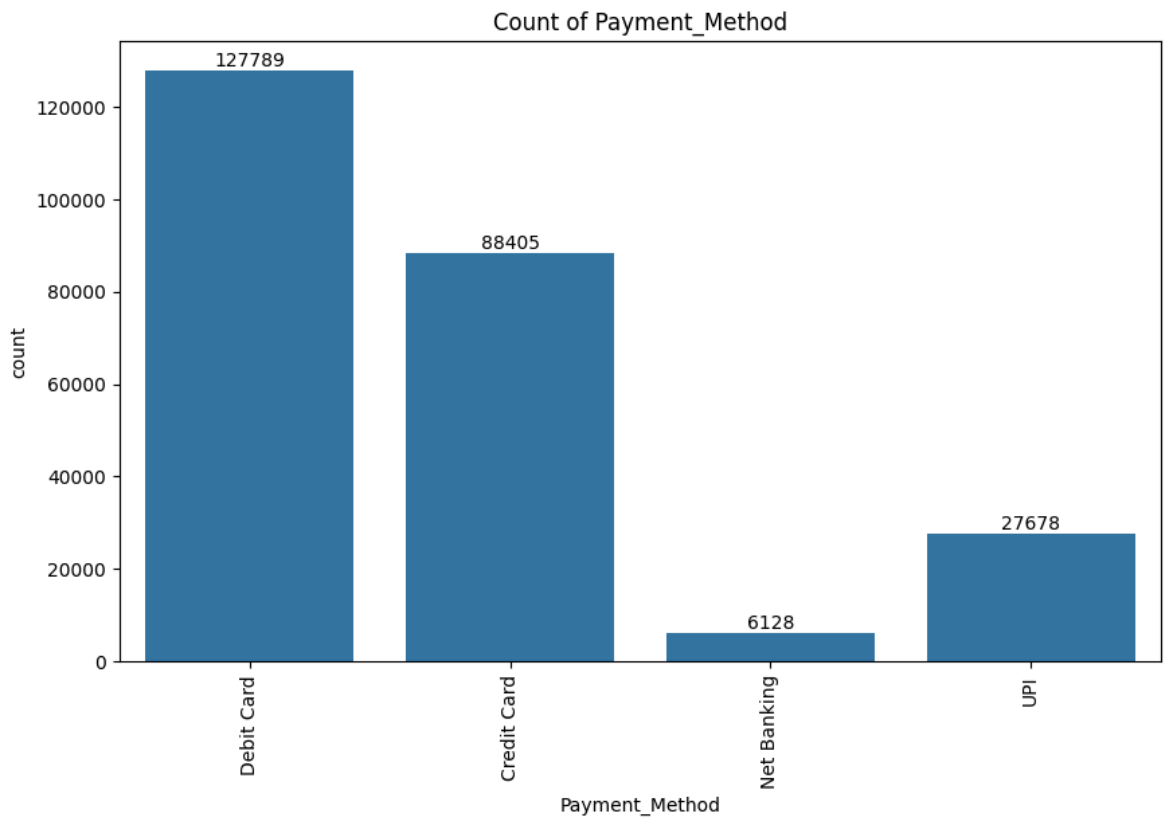
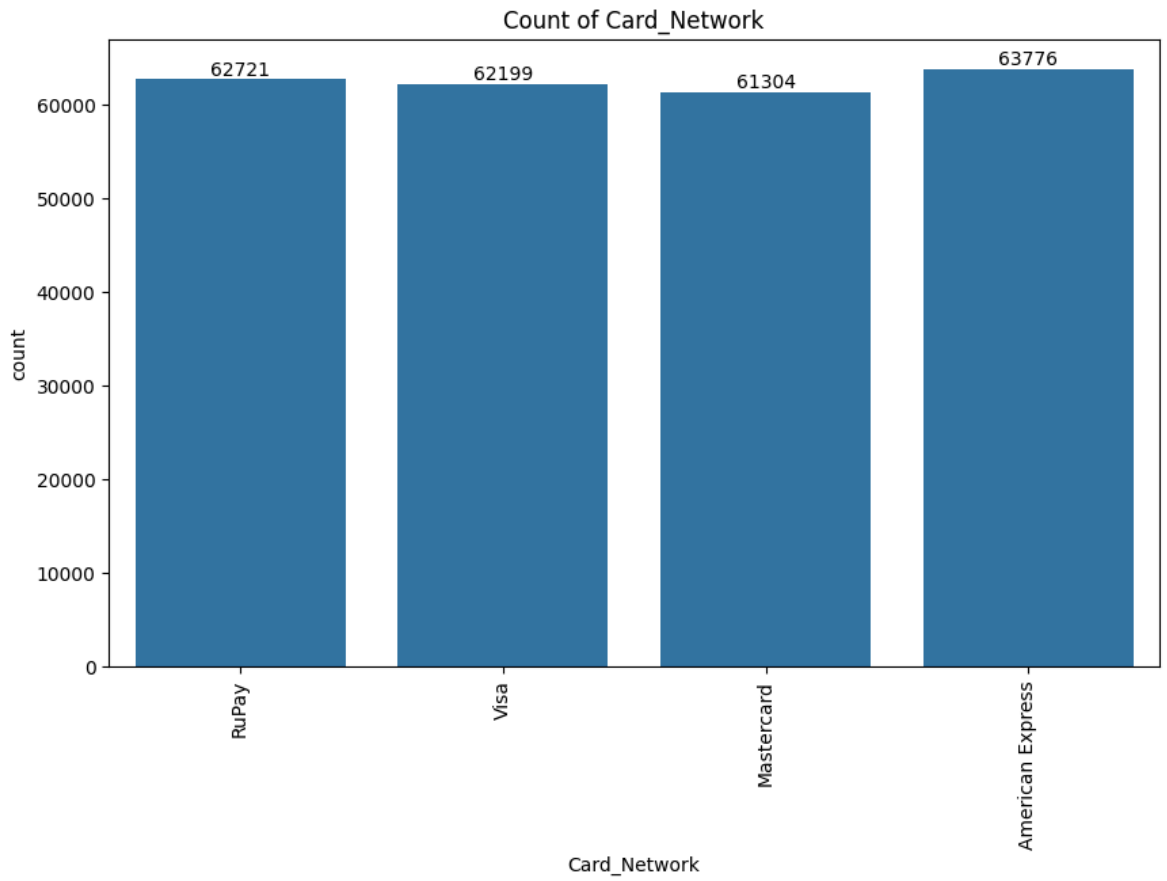


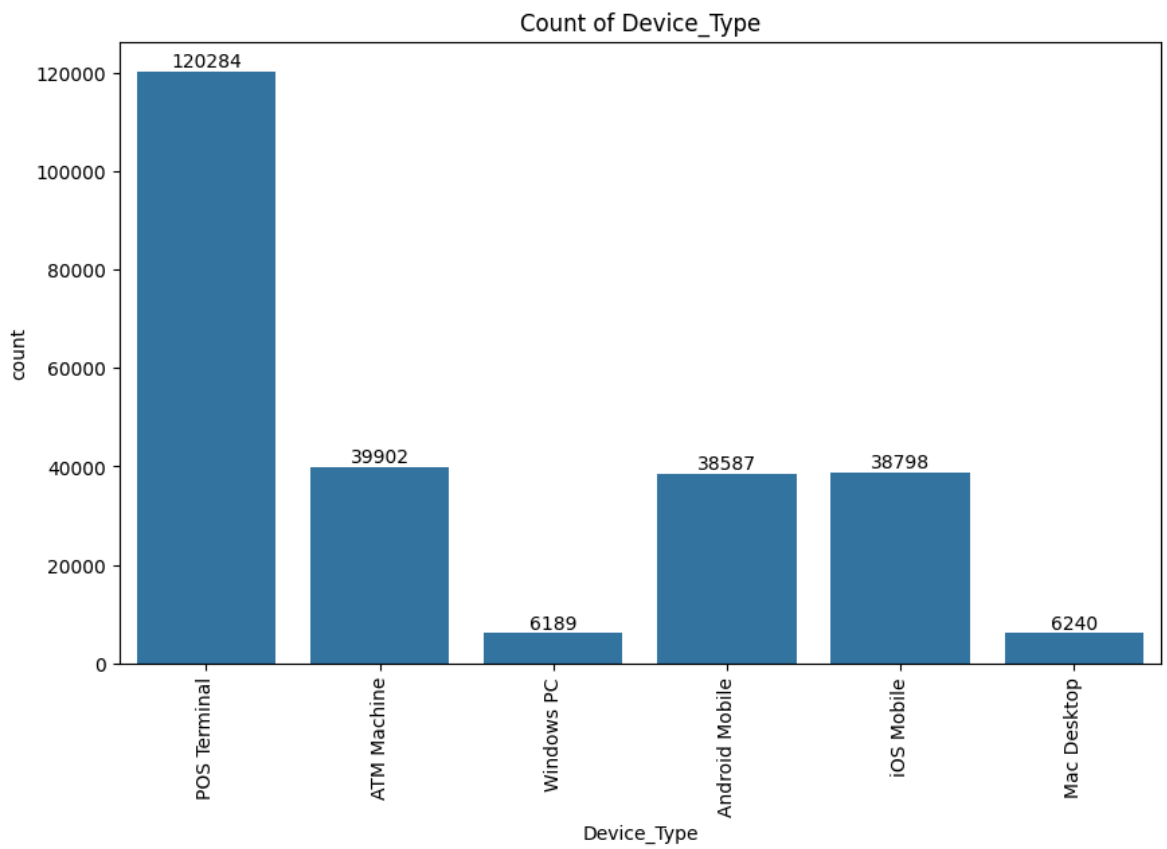
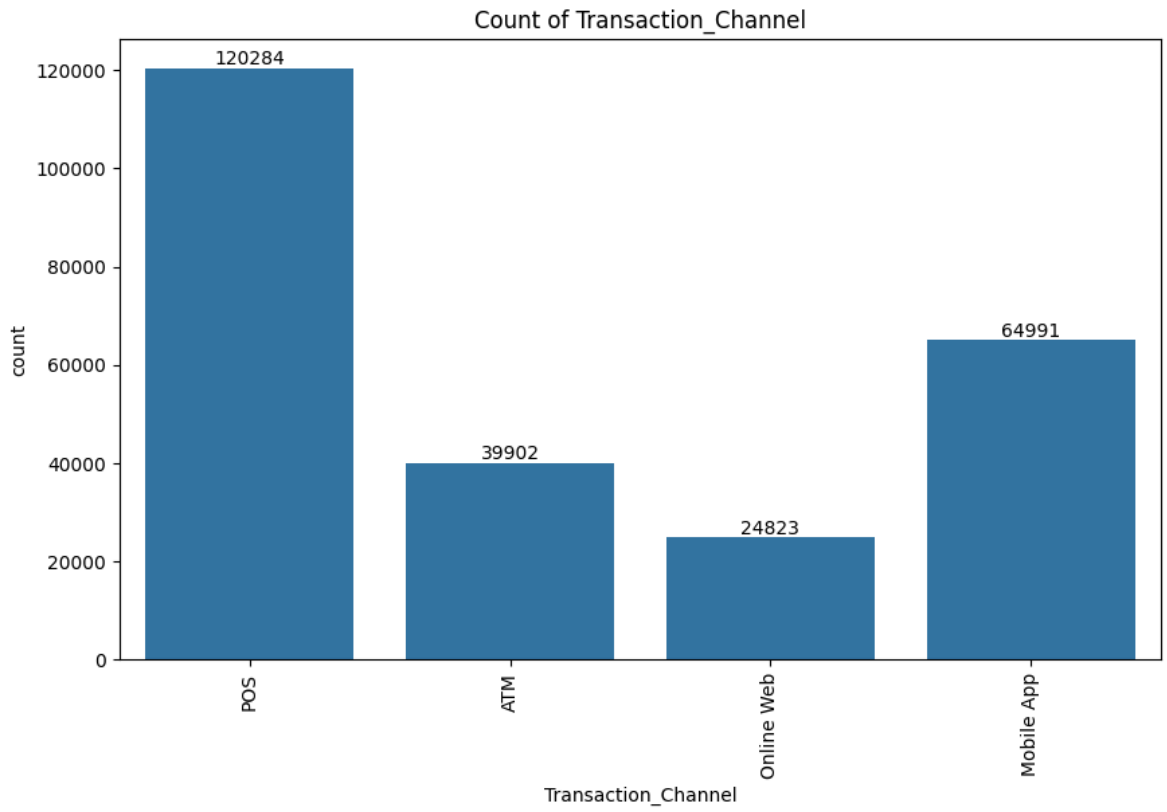
```
In [91]: cat_cols = [
    'Gender',
    'Occupation',
    'Customer_Segment',
    'Card_Type',
    'Card_Network',
    'Payment_Method',
    'Transaction_Channel',
    'Device_Type',
    'Merchant_Category',
    'Merchant_Risk_Level',
    'Fraud_Flag'
]
```

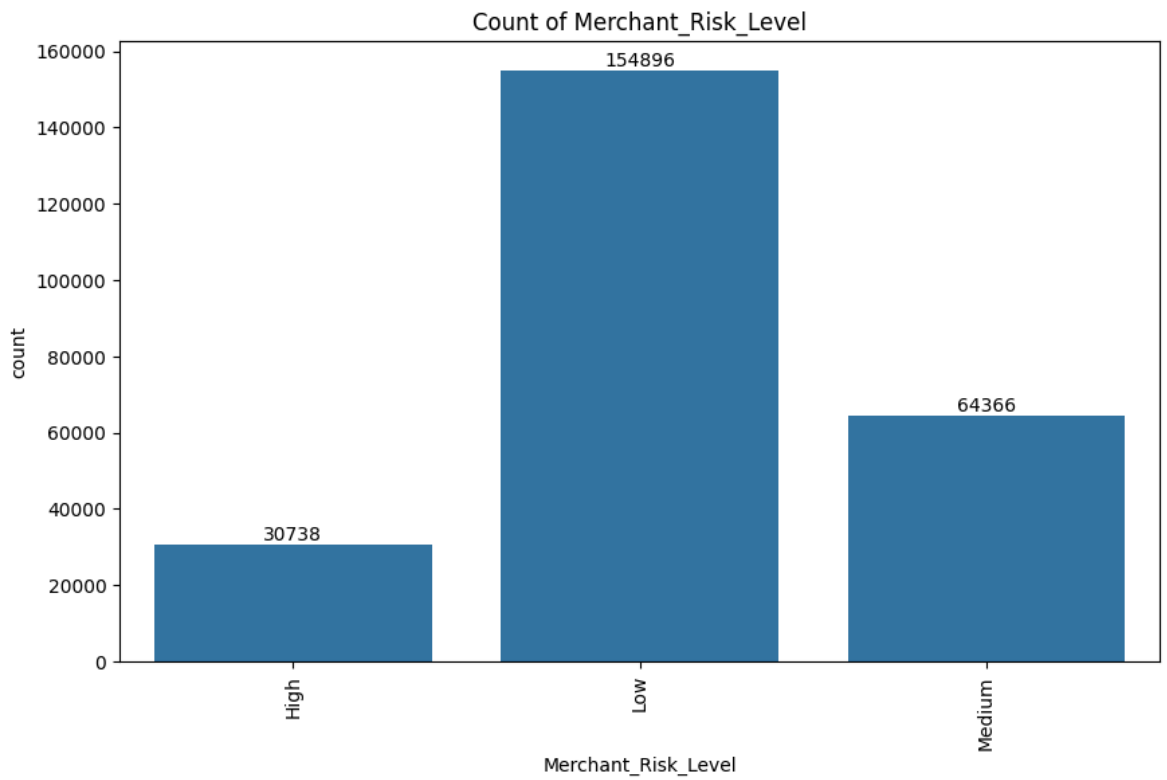
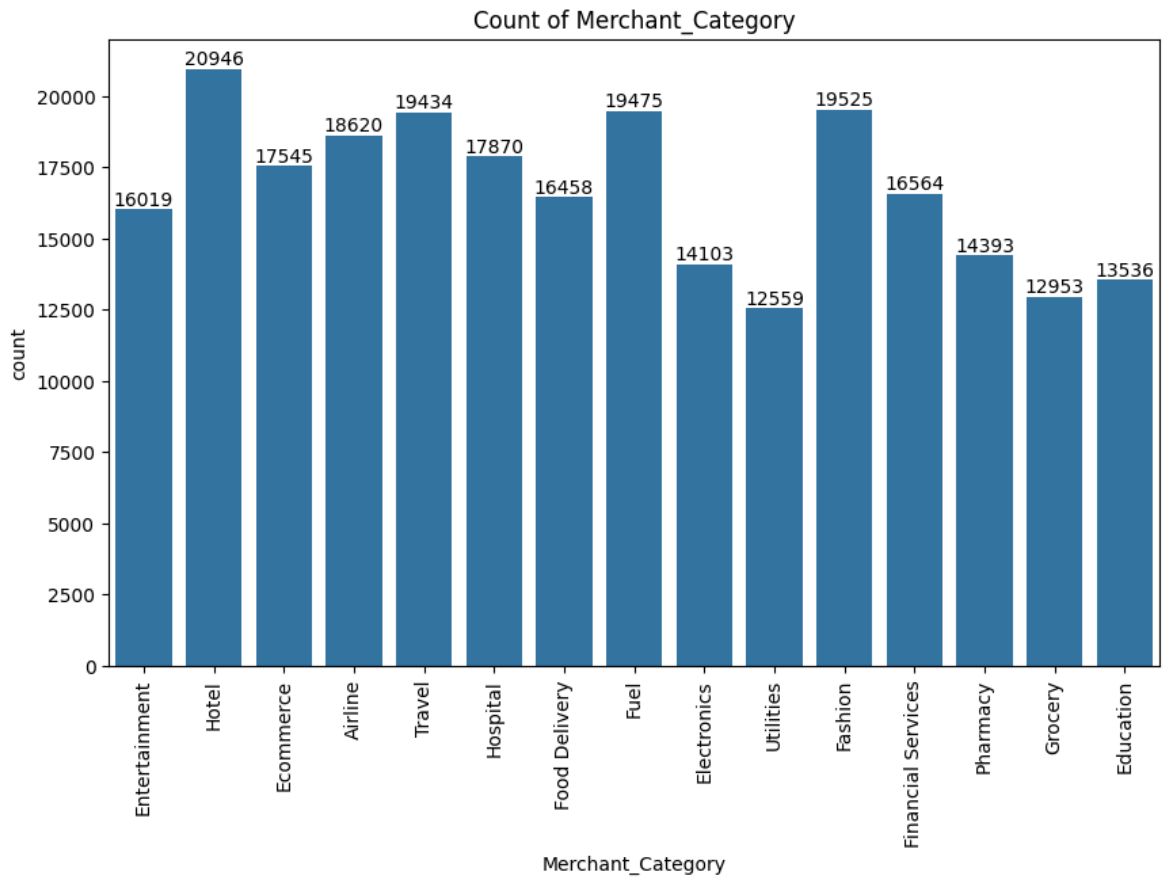
```
In [92]: for col in cat_cols:
    plt.figure(figsize=(10,6))
    sns.countplot(data=final_merge, x = col)
    plt.title(f'Count of {col}')
    plt.xticks(rotation=90)
    plt.bar_label(plt.gca().containers[0])
    plt.show()
```

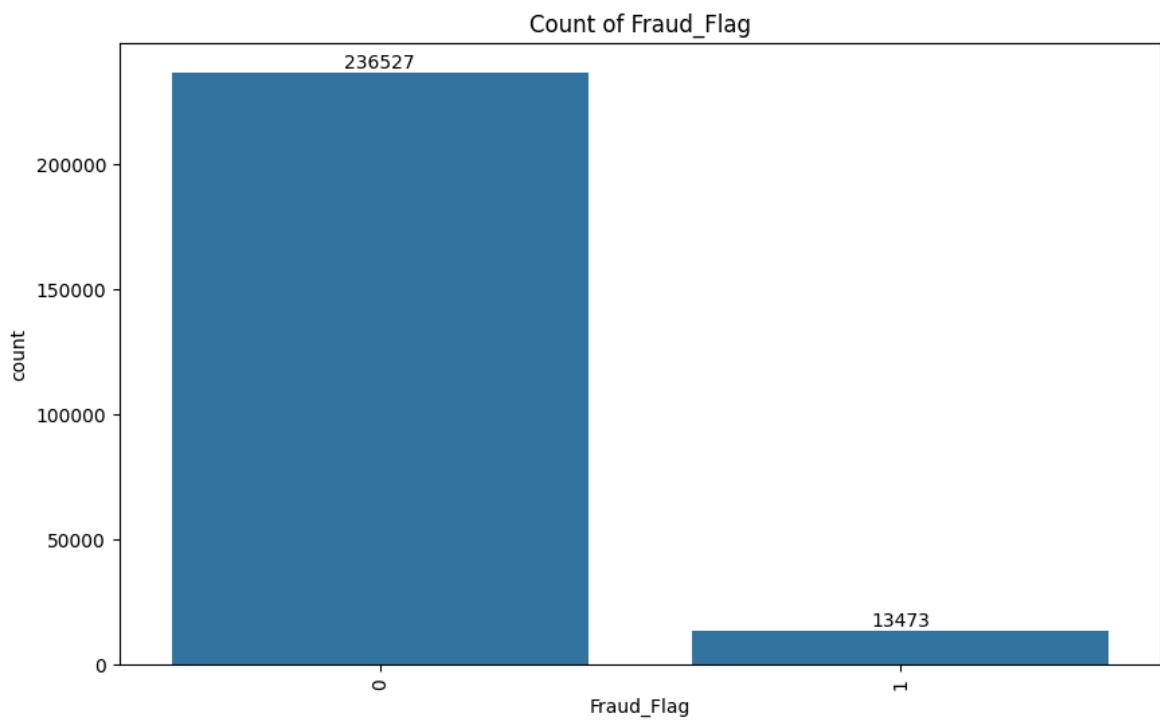












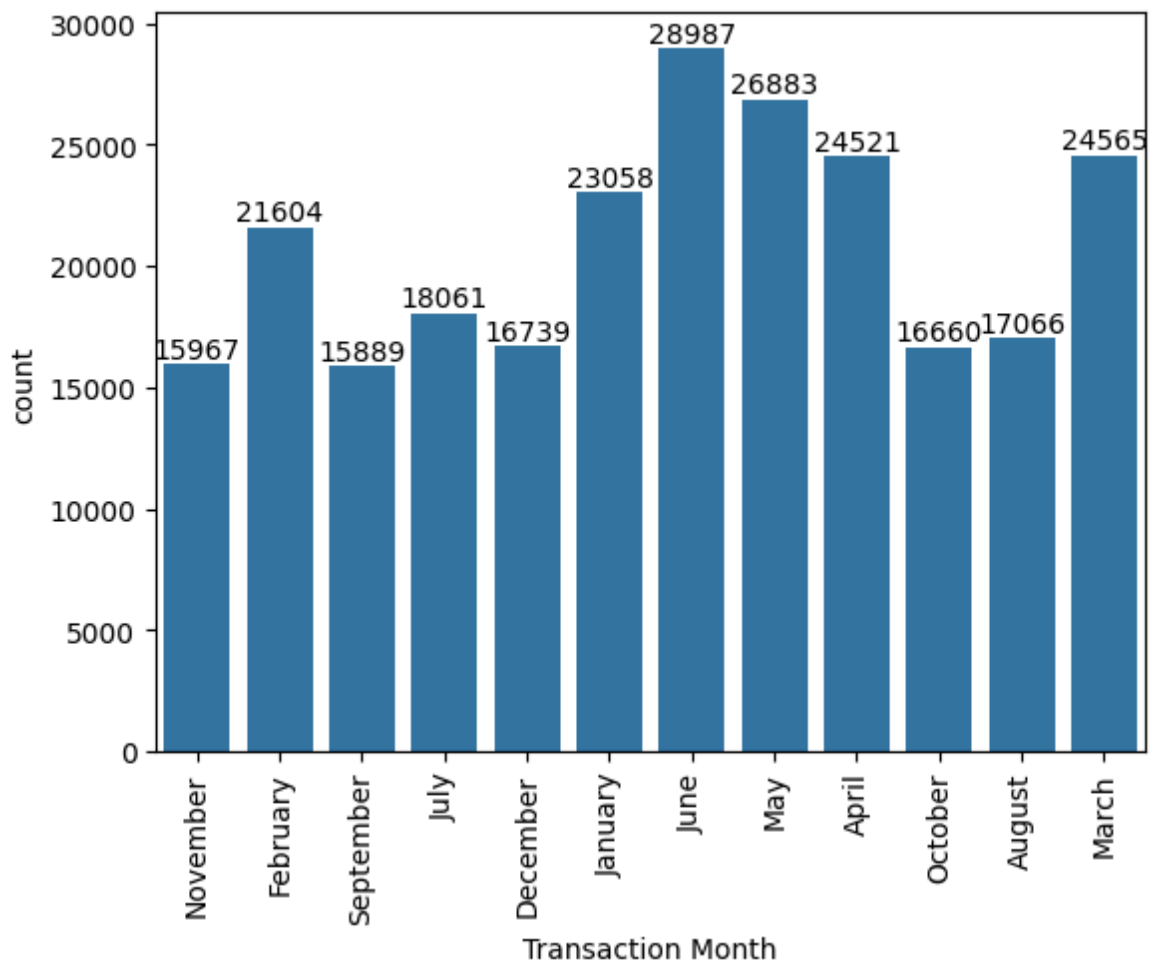
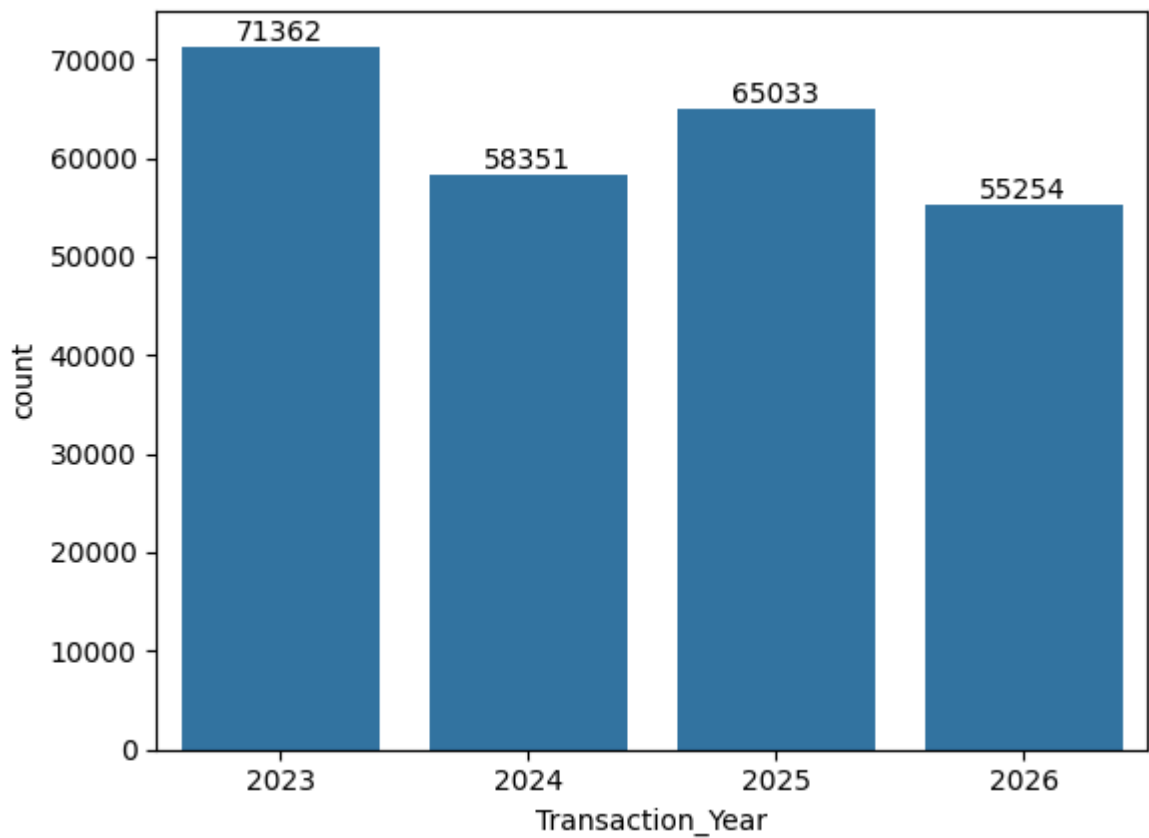
Date Columns

```
In [93]: final_merge['Transaction_Year'] = final_merge['Transaction_Date'].dt.year
```

```
In [94]: final_merge['Transaction_Month'] = final_merge['Transaction_Date'].dt.month_name
```

```
In [95]: sns.countplot(data=final_merge, x = 'Transaction_Year')
plt.bar_label(plt.gca().containers[0])
plt.show()

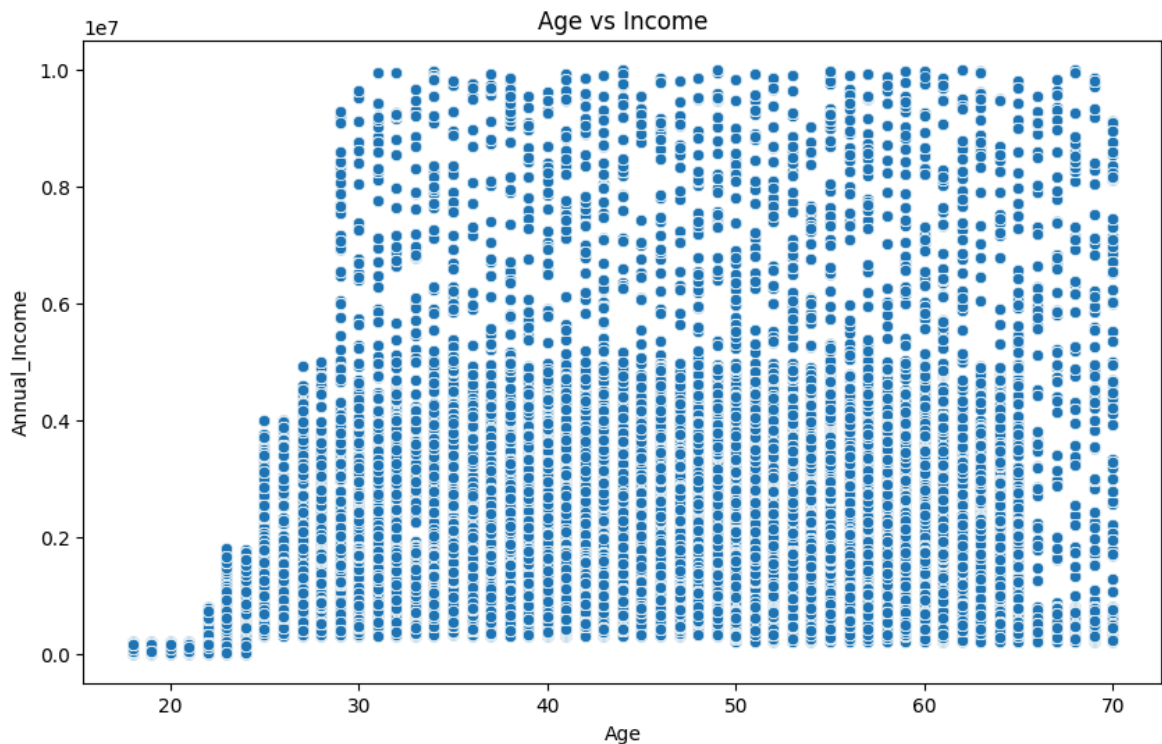
sns.countplot(data=final_merge, x='Transaction_Month')
plt.xticks(rotation = 90)
plt.bar_label(plt.gca().containers[0])
plt.show()
```

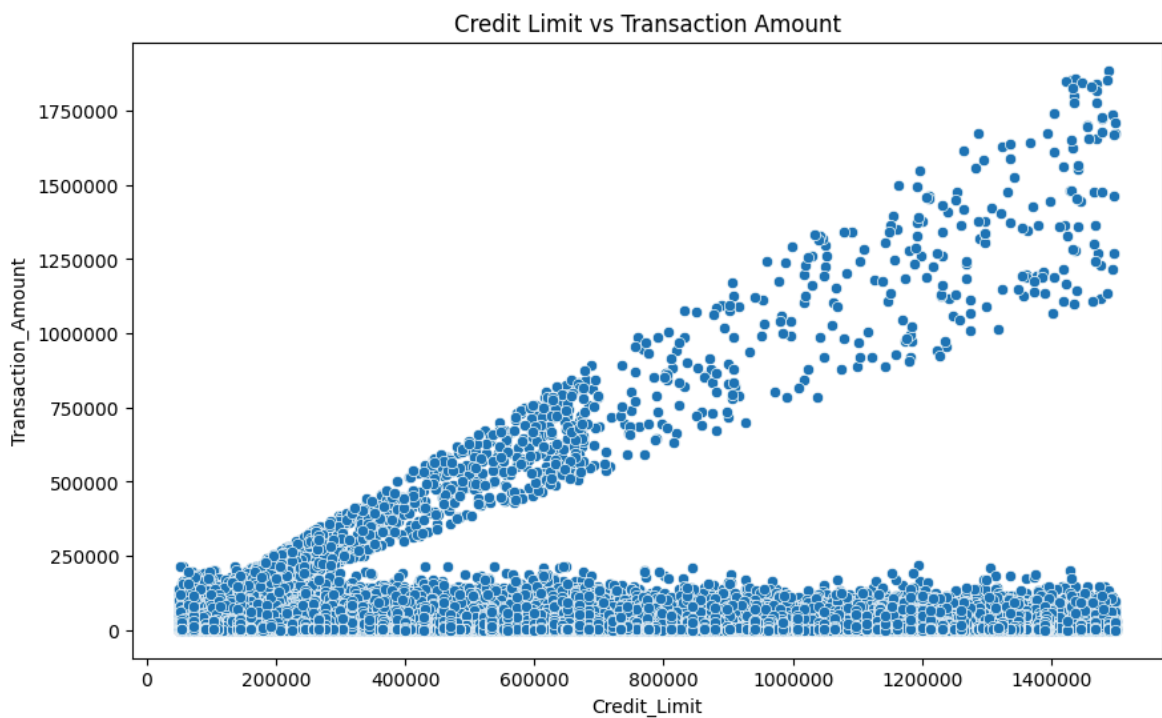
Bivariate Analysis

```
In [96]: plt.figure(figsize=(10,6))
sns.scatterplot(data=final_merge, x = 'Age', y = 'Annual_Income')
```

```
plt.title("Age vs Income")
plt.show()
```

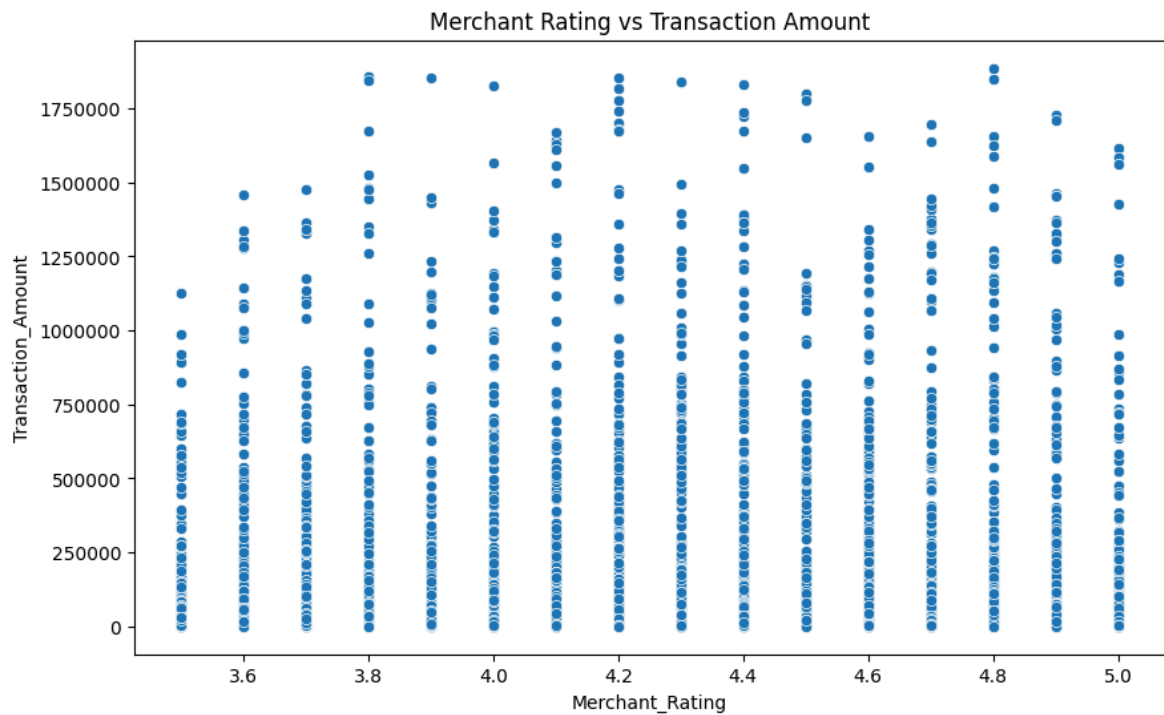


```
In [97]: plt.figure(figsize=(10,6))
sns.scatterplot(data=final_merge, x = 'Credit_Limit', y = 'Transaction_Amount')
plt.title("Credit Limit vs Transaction Amount")
plt.ticklabel_format(style='plain', axis = 'x')
plt.ticklabel_format(style='plain', axis = 'y')
plt.show()
```



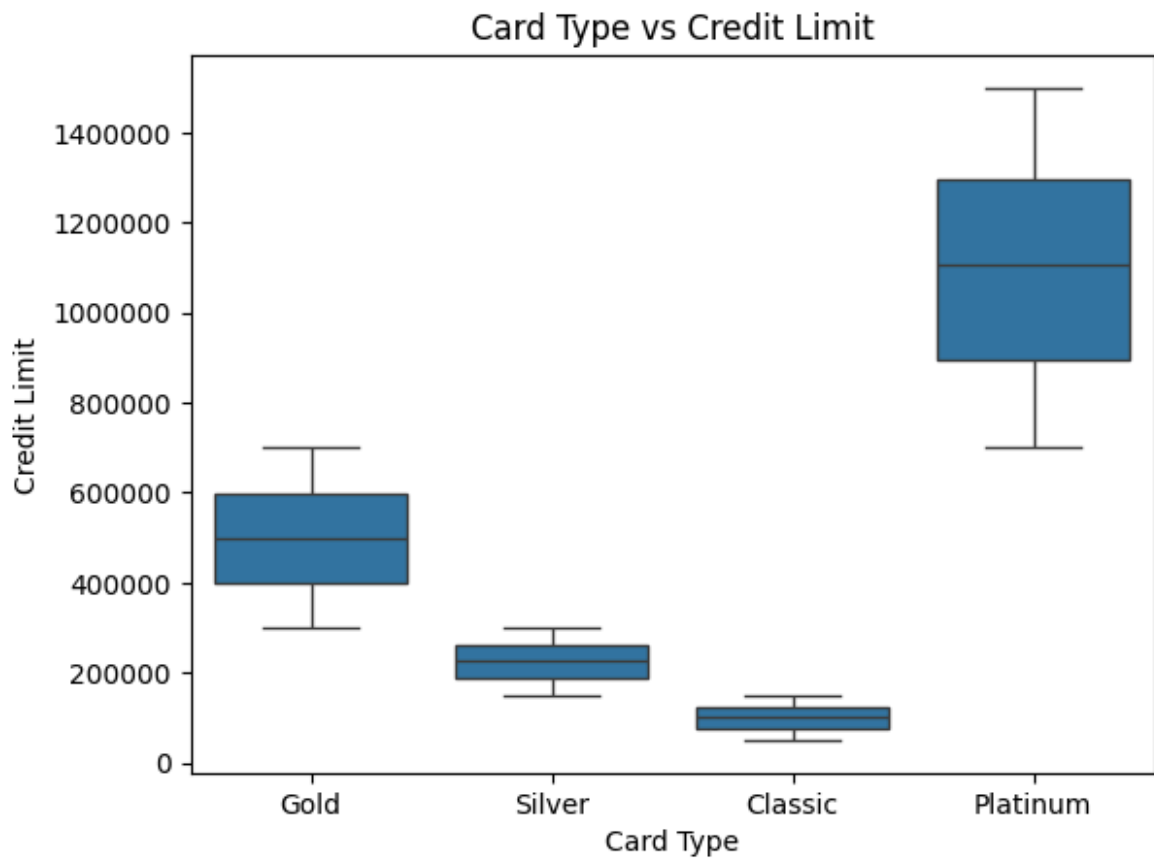
```
In [98]: plt.figure(figsize=(10,6))
sns.scatterplot(data=final_merge, x = 'Merchant_Rating', y = 'Transaction_Amount')
plt.title("Merchant Rating vs Transaction Amount")
```

```
plt.ticklabel_format(style='plain', axis = 'y')
plt.show()
```



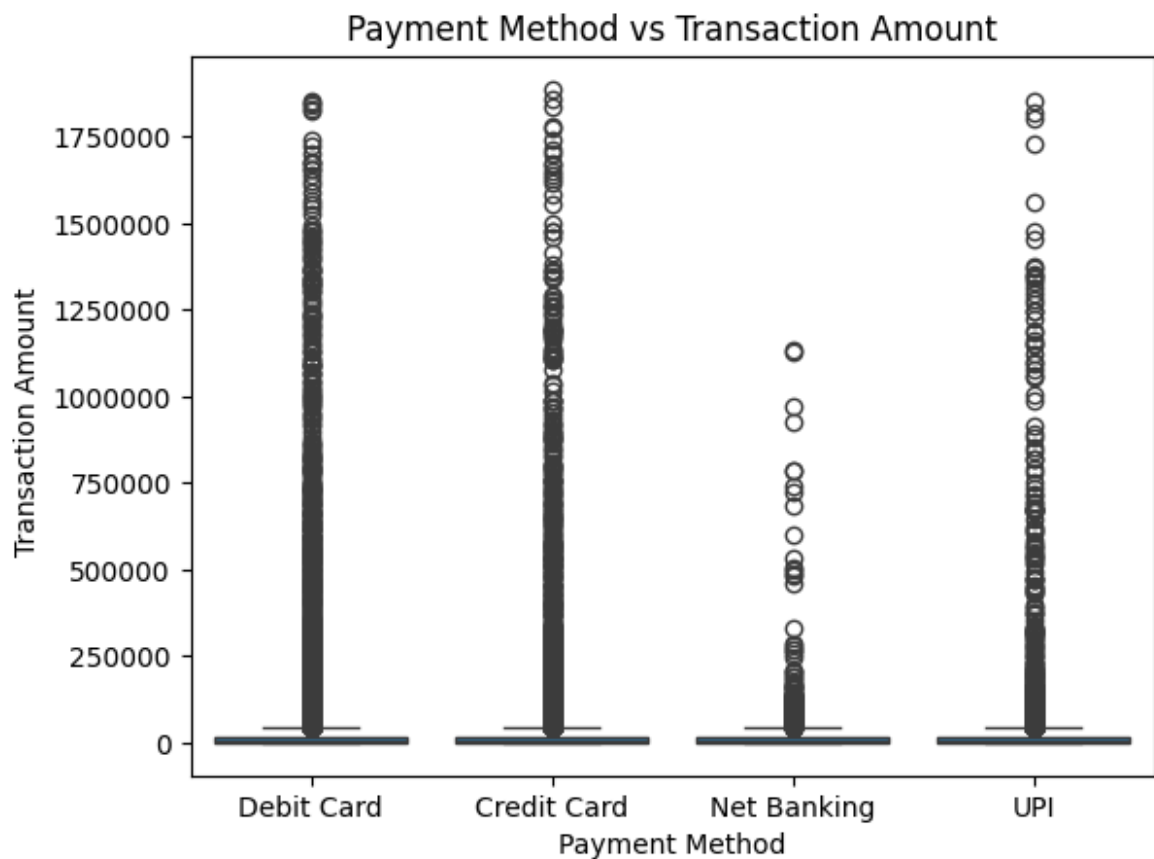
Categorical vs Numerical

```
In [99]: sns.boxplot(data=final_merge, x = 'Card_Type', y = 'Credit_Limit')
plt.title("Card Type vs Credit Limit")
plt.xlabel("Card Type")
plt.ylabel("Credit Limit")
plt.ticklabel_format(style='plain', axis= 'y')
plt.show()
```

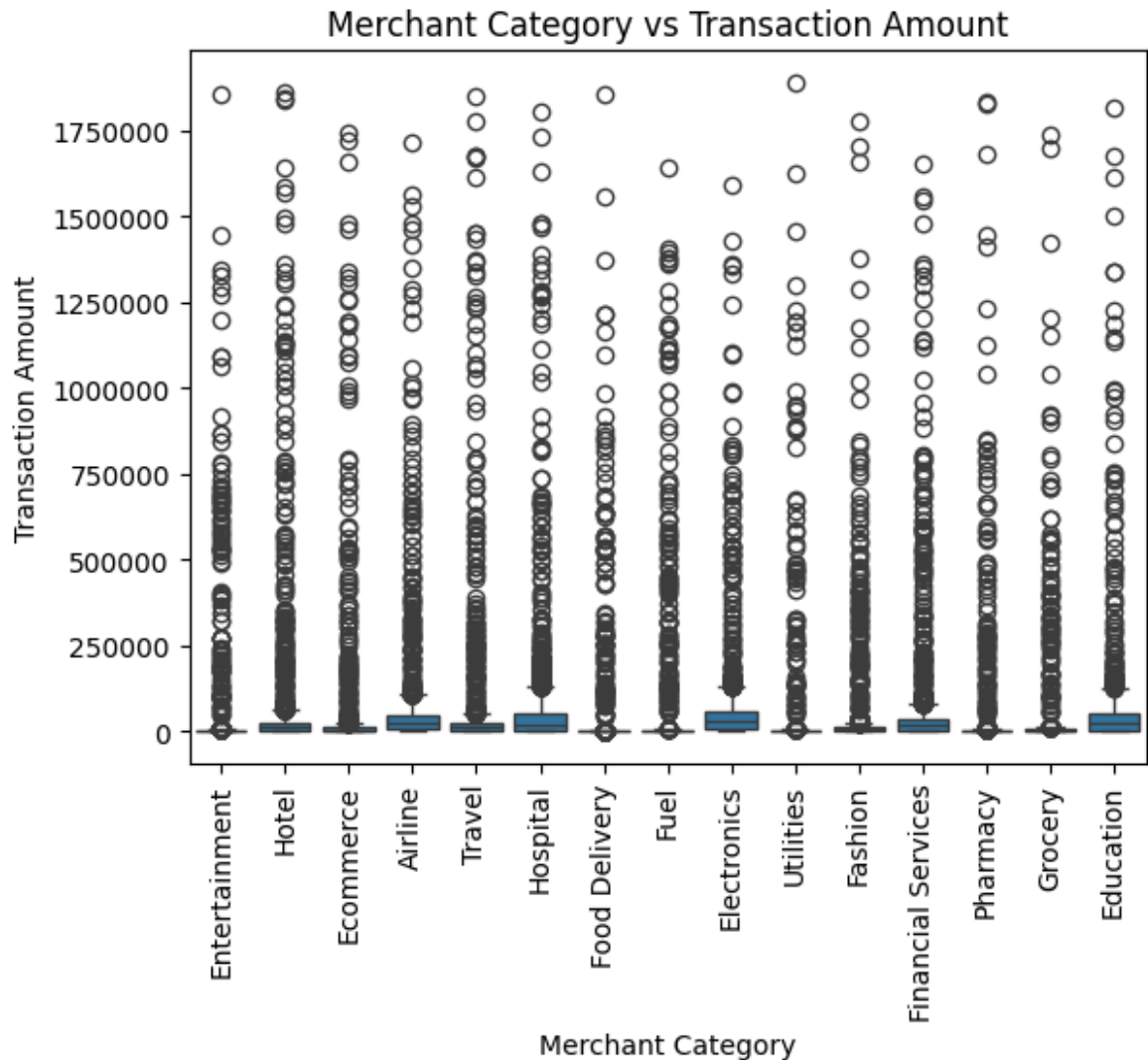


In [100...

```
sns.boxplot(data=final_merge, x = 'Payment_Method', y = 'Transaction_Amount')
plt.title("Payment Method vs Transaction Amount")
plt.xlabel("Payment Method")
plt.ylabel("Transaction Amount ")
plt.ticklabel_format(style='plain', axis= 'y')
plt.show()
```



```
In [101... sns.boxplot(data=final_merge, x = 'Merchant_Category', y = 'Transaction_Amount')
plt.title("Merchant Category vs Transaction Amount")
plt.xlabel("Merchant Category")
plt.ylabel("Transaction Amount")
plt.xticks(rotation = 90)
plt.ticklabel_format(style='plain', axis='y')
plt.show()
```

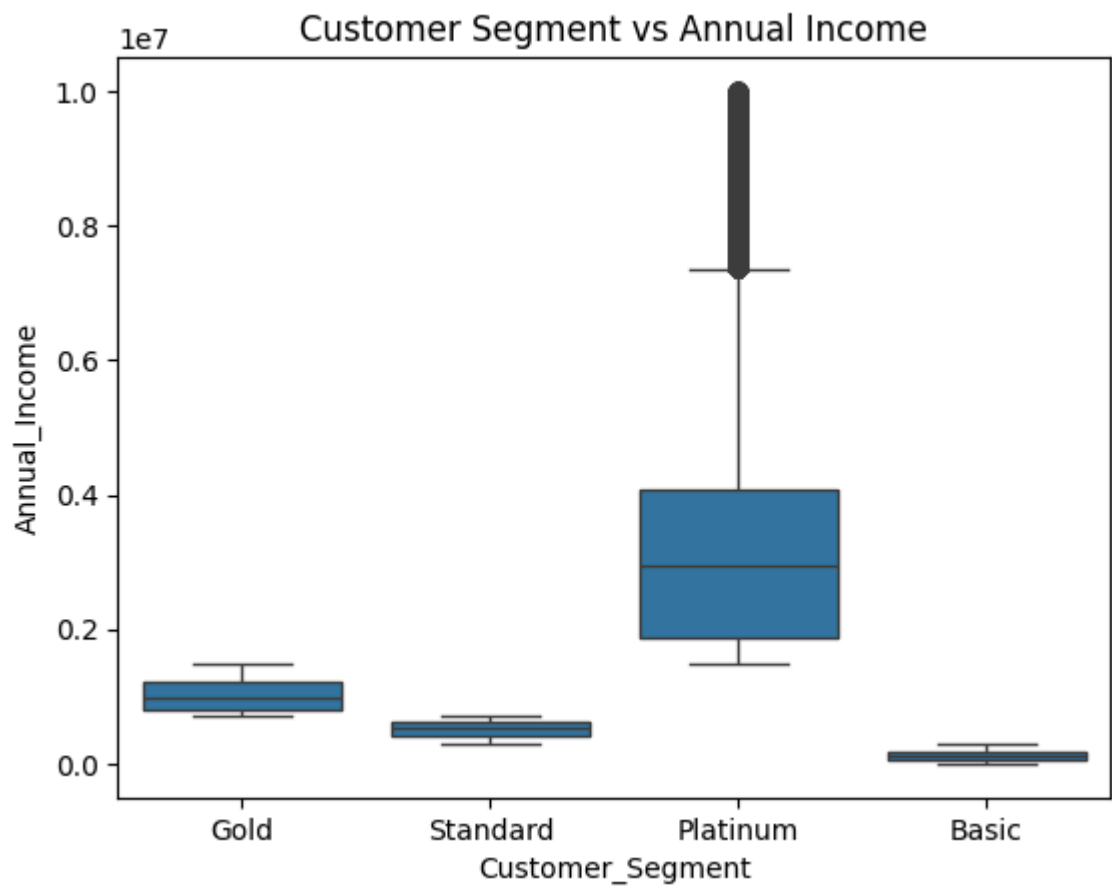


```
In [102... sns.boxplot(data=final_merge, x = 'Fraud_Flag', y = 'Transaction_Amount')
plt.title("Fraud Flag vs Transaction Amount")
plt.xlabel("Fraud Flag")
plt.ylabel("Transaction Amount")
plt.ticklabel_format(style='plain', axis='y')
plt.show()
```

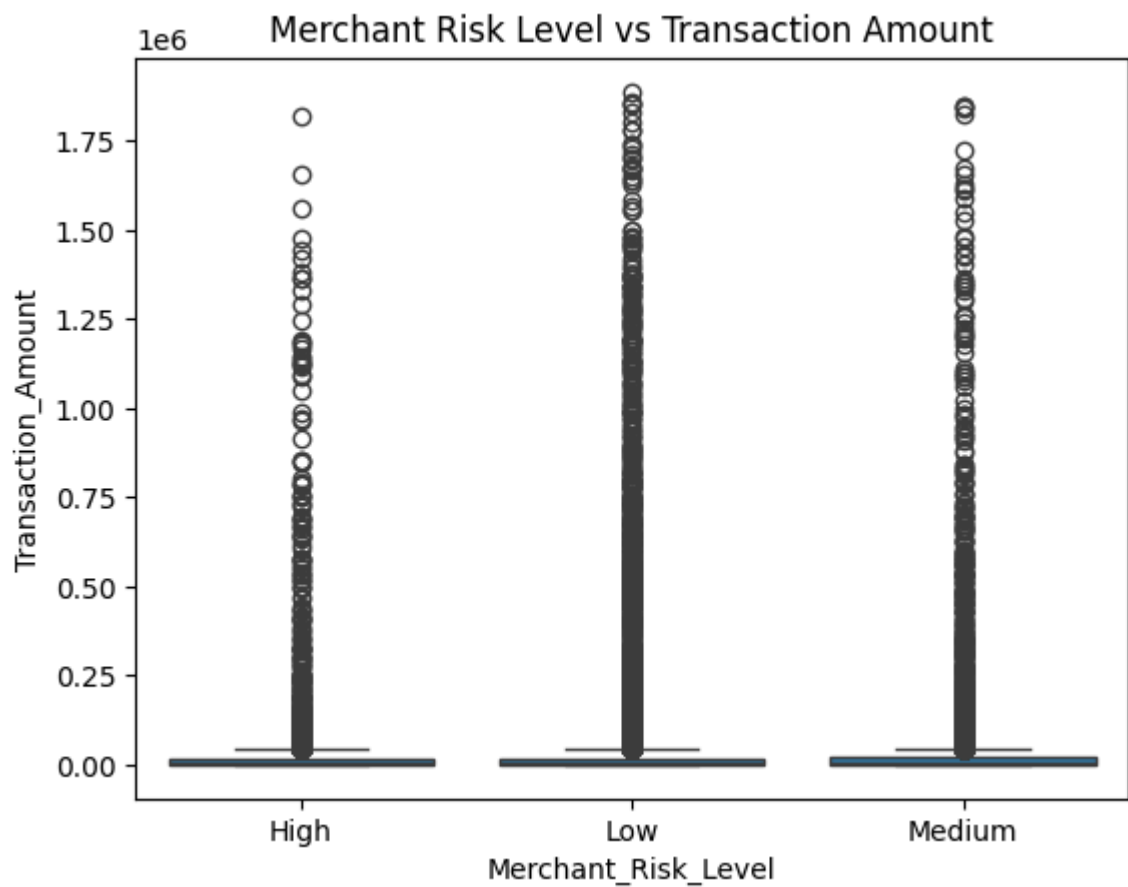


In [103...

```
sns.boxplot(data=final_merge, x='Customer_Segment', y='Annual_Income')  
plt.title("Customer Segment vs Annual Income")  
plt.show()
```

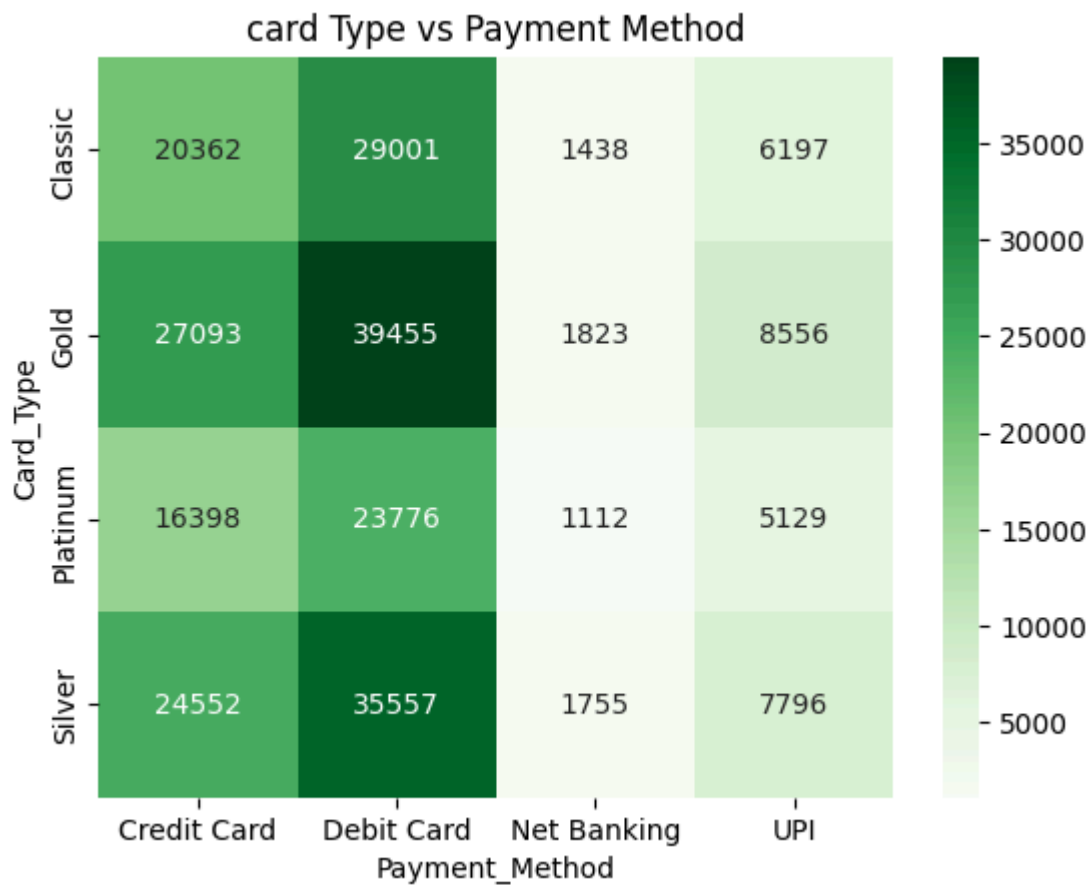


```
In [104... sns.boxplot(data=final_merge, x="Merchant_Risk_Level", y='Transaction_Amount')
plt.title("Merchant Risk Level vs Transaction Amount")
plt.show()
```



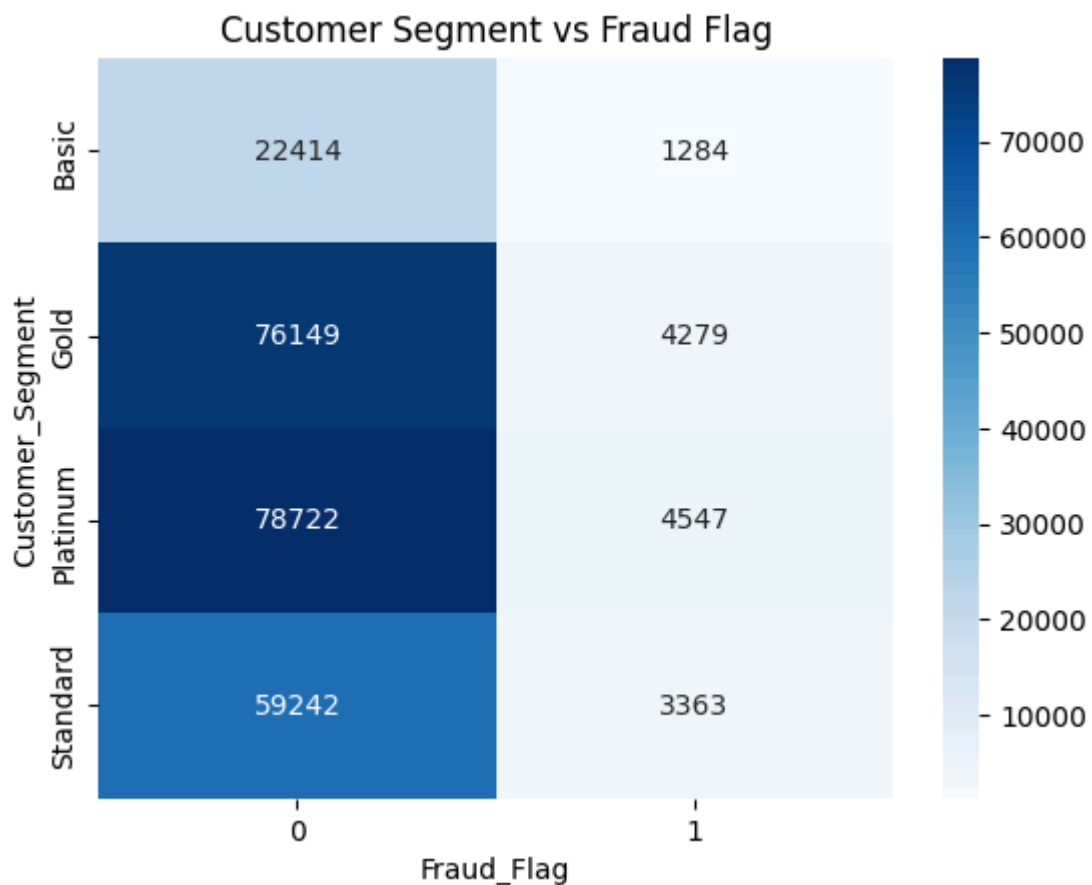
Categorical vs Categorical

```
In [105... ct = pd.crosstab(final_merge['Card_Type'], final_merge['Payment_Method'])
sns.heatmap(ct, annot=True, fmt = 'd', cmap='Greens')
plt.title("card Type vs Payment Method")
plt.show()
```

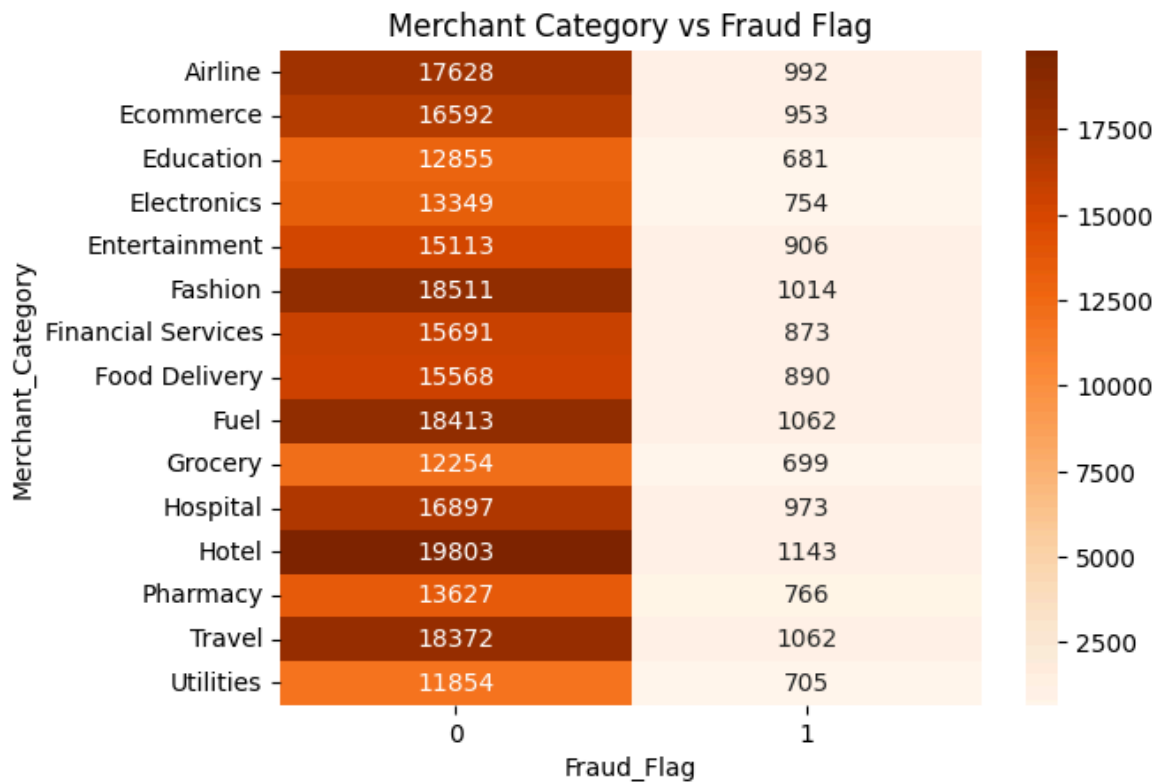


In [106...

```
ct = pd.crosstab(final_merge['Customer_Segment'], final_merge['Fraud_Flag'])
sns.heatmap(ct, annot=True, fmt='d', cmap='Blues')
plt.title("Customer Segment vs Fraud Flag")
plt.show()
```




```
In [107... ct = pd.crosstab(final_merge['Merchant_Category'], final_merge['Fraud_Flag'])
sns.heatmap(ct, annot=True, fmt='d', cmap='Oranges')
plt.title("Merchant Category vs Fraud Flag")
plt.show()
```

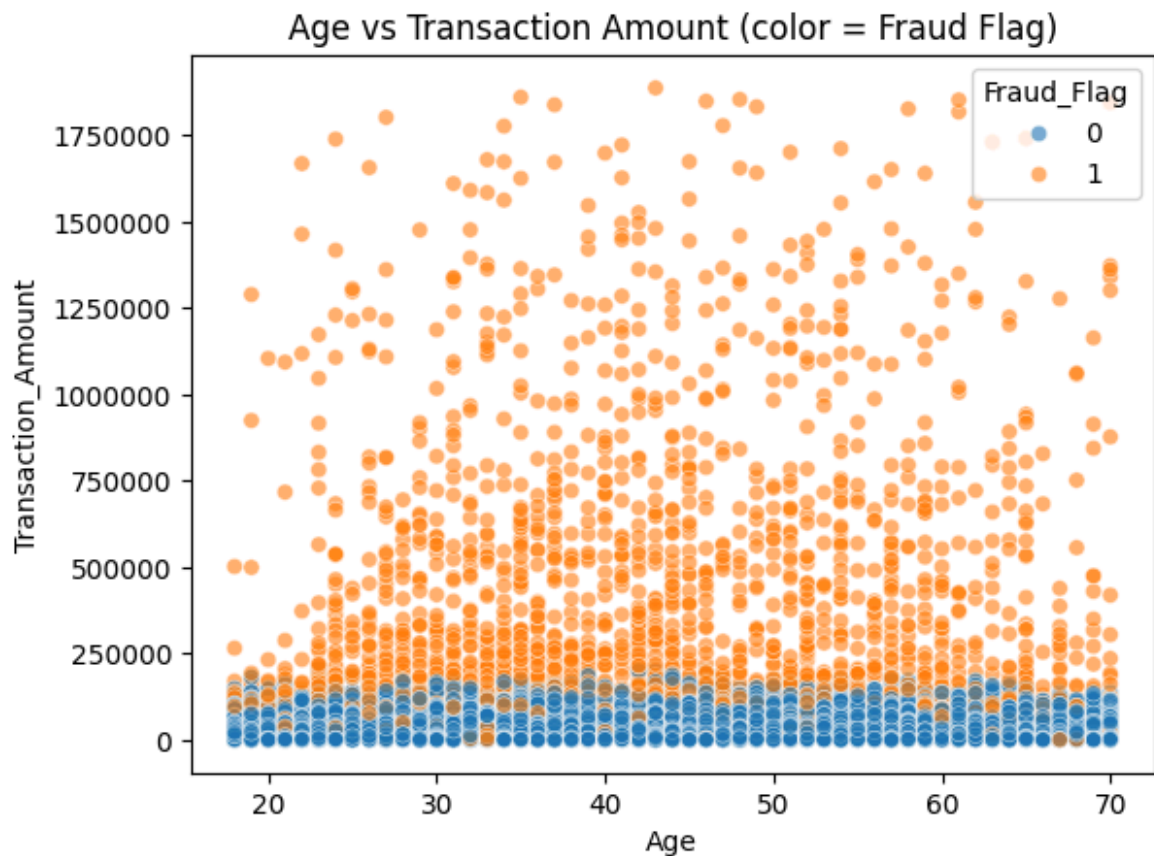


Multivariate Analysis

```
In [108... sns.scatterplot(data=final_merge,
                  x='Age',
                  y='Transaction_Amount',
                  hue='Fraud_Flag',
                  alpha=0.6)
plt.title("Age vs Transaction Amount (color = Fraud Flag)")
plt.ticklabel_format(style='plain', axis = 'y')
plt.show()
```

c:\Users\visha\AppData\Local\Programs\Python\Python314\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Creating legend with loc="best" can be slow with large amounts of data.

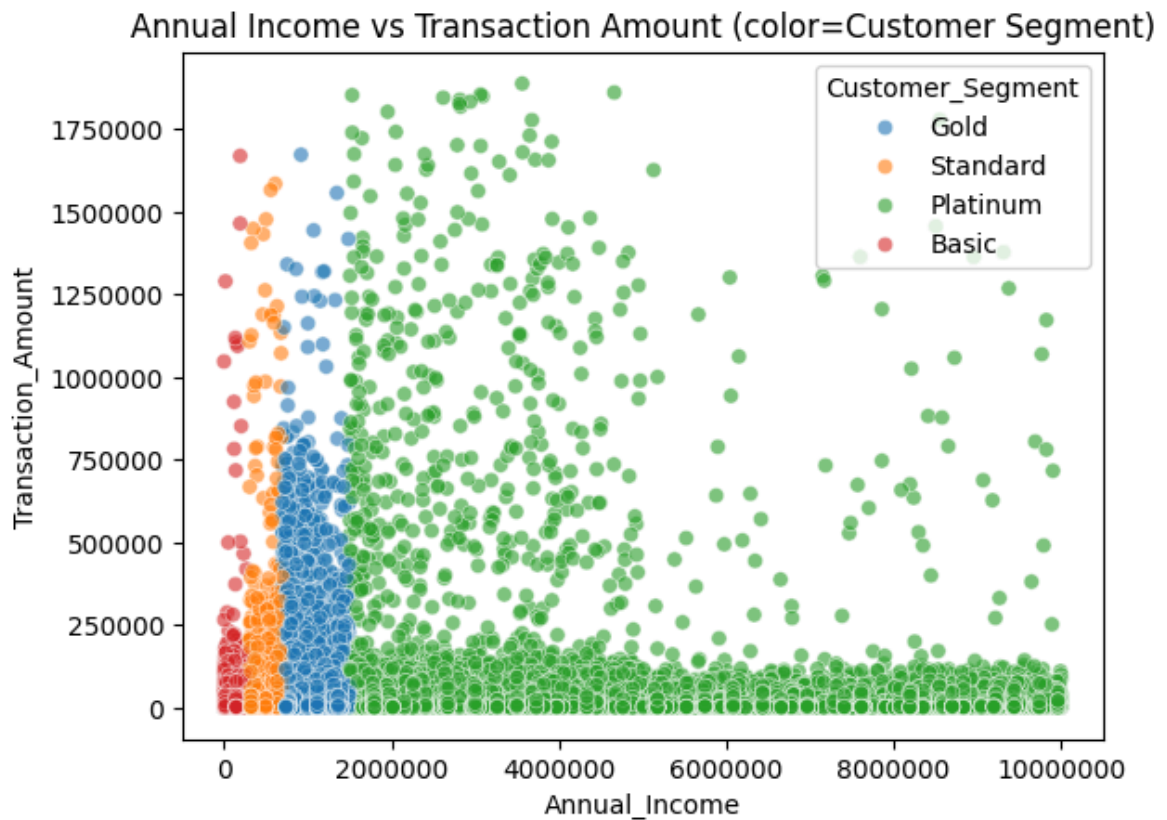
```
fig.canvas.print_figure(bytes_io, **kw)
```



In [109...

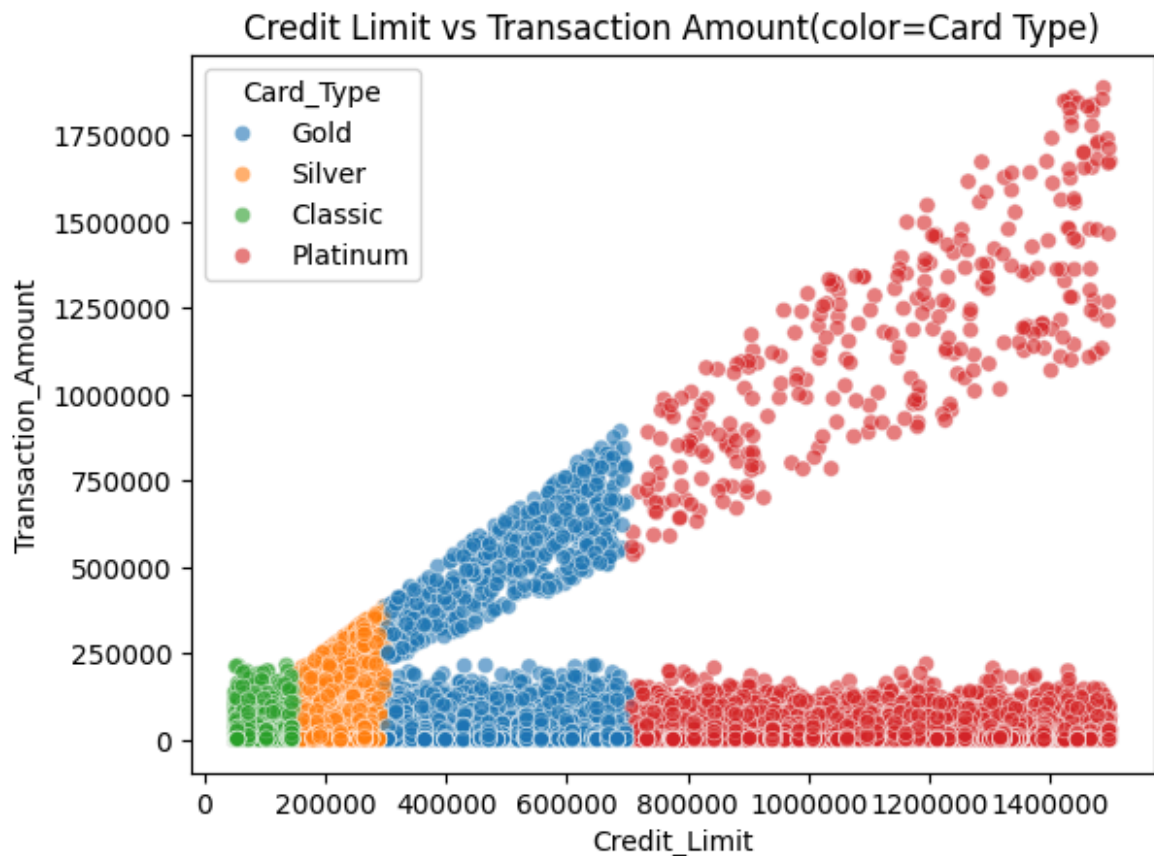
```
sns.scatterplot(
    data=final_merge,
    x='Annual_Income',
    y='Transaction_Amount',
    hue='Customer_Segment',
    alpha=0.6
)
plt.title("Annual Income vs Transaction Amount (color=Customer Segment)")
plt.ticklabel_format(style='plain', axis='x')
plt.ticklabel_format(style = 'plain', axis = 'y')
plt.show()
```

c:\Users\visha\AppData\Local\Programs\Python\Python314\Lib\site-packages\IPython\
\core\pylabtools.py:170: UserWarning: Creating legend with loc="best" can be slow
with large amounts of data.
fig.canvas.print_figure(bytes_io, **kw)



In [110...

```
sns.scatterplot(  
    data=final_merge,  
    x='Credit_Limit',  
    y='Transaction_Amount',  
    hue='Card_Type',  
    alpha=0.6  
)  
plt.title("Credit Limit vs Transaction Amount(color=Card Type)")  
plt.ticklabel_format(style='plain', axis='x')  
plt.ticklabel_format(style = 'plain', axis = 'y')  
plt.show()
```

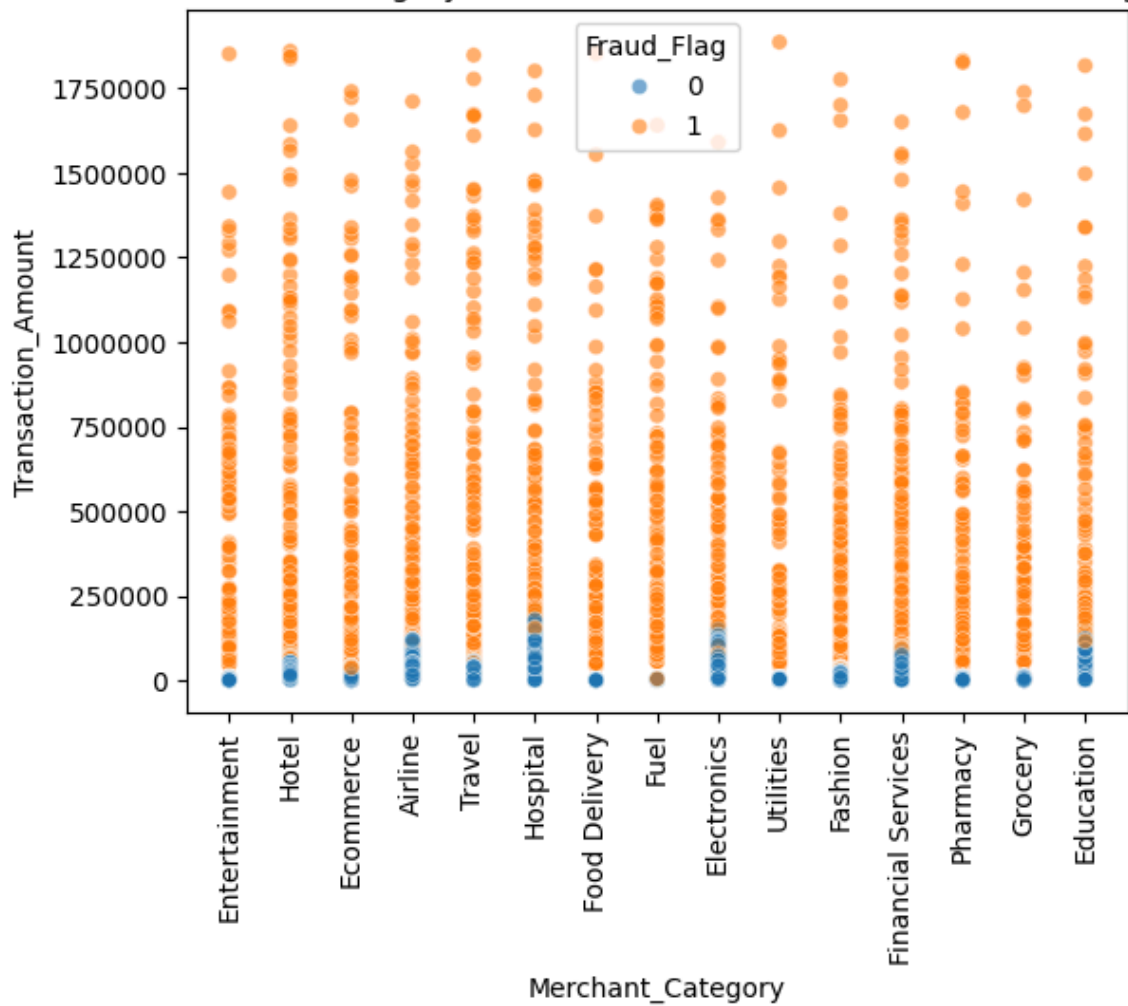


In [111...

```
sns.scatterplot(
    data=final_merge,
    x='Merchant_Category',
    y='Transaction_Amount',
    hue="Fraud_Flag",
    alpha=0.6
)
plt.title("Merchant Category vs Transaction Amount (color = Fraud Flag)")
plt.ticklabel_format(style='plain', axis = 'y')
plt.xticks(rotation=90)
plt.show()
```

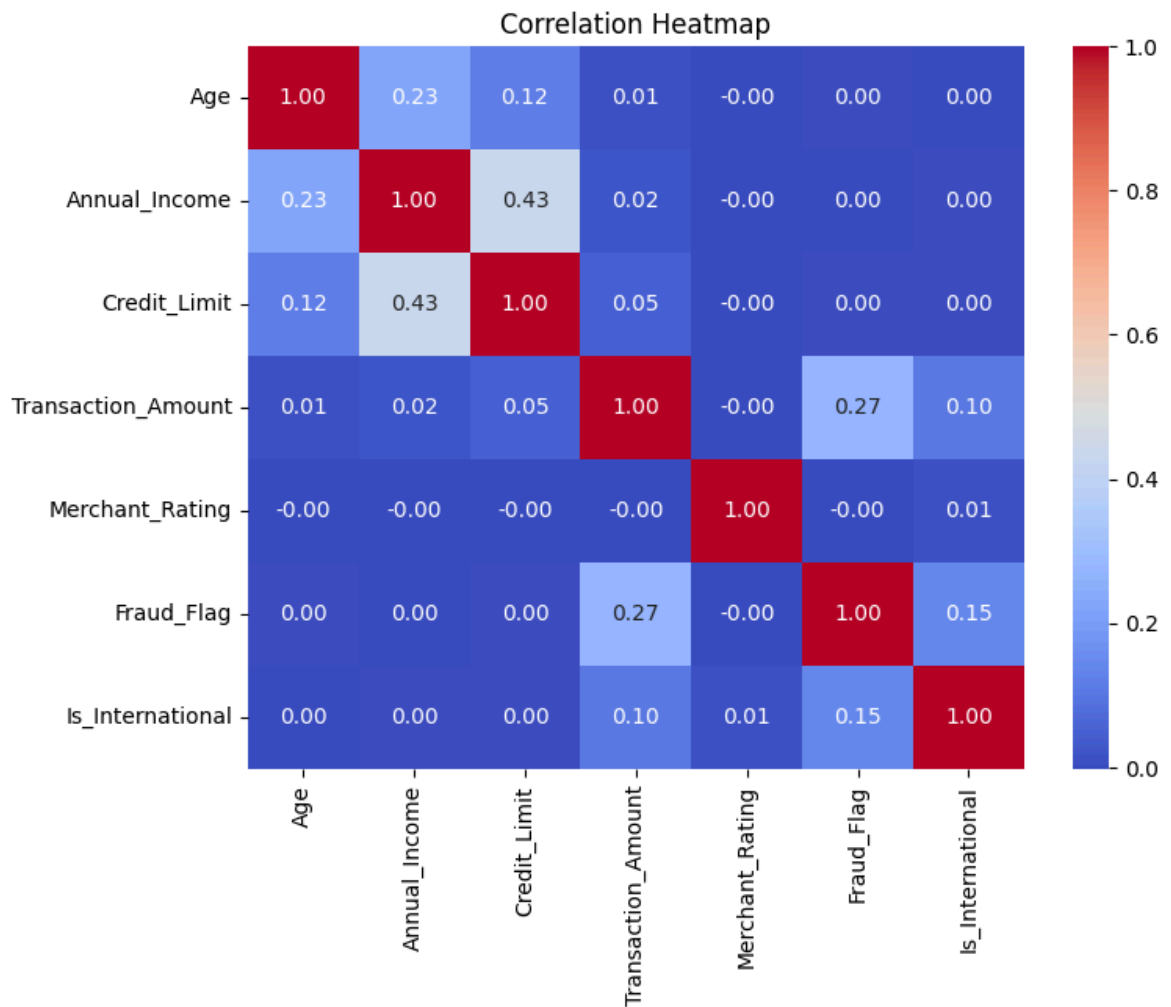
c:\Users\visha\AppData\Local\Programs\Python\Python314\Lib\site-packages\IPython\
\core\pylabtools.py:170: UserWarning: Creating legend with loc="best" can be slow
with large amounts of data.
fig.canvas.print_figure(bytes_io, **kw)

Merchant Category vs Transaction Amount (color = Fraud Flag)



In [112...

```
num_cols = [
    'Age',
    'Annual_Income',
    'Credit_Limit',
    'Transaction_Amount',
    'Merchant_Rating',
    'Fraud_Flag',
    'Is_International'
]
corr = final_merge[num_cols].corr()
plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, fmt = '.2f', cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```



In [113... corr

Out[113...

	Age	Annual_Income	Credit_Limit	Transaction_Amount	Merchant
Age	1.00	0.23	0.12	0.01	
Annual_Income	0.23	1.00	0.43	0.02	
Credit_Limit	0.12	0.43	1.00	0.05	
Transaction_Amount	0.01	0.02	0.05	1.00	
Merchant_Rating	-0.00	-0.00	-0.00	-0.00	
Fraud_Flag	0.00	0.00	0.00	0.27	
Is_International	0.00	0.00	0.00	0.10	

In [114...

```
# final_merge.to_csv("C:\\Users\\visha\\Downloads\\Datasets\\Financial Transacti
```